
Python multibody4everybody

Release v2.0.0

Alvaro Diaz Flores Caminero, Sahand Sabet

Oct 05, 2026

CORE MODULES

1	Executive Summary	2
1.1	Why a New Multibody Tool?	3
1.2	How Multibody4Everybody Works	4
1.3	Validation and Help for New Users	5
1.4	Symbolic or Numeric—Your Choice	5
2	Installation	7
2.1	Documentation	8
3	User documentation	9
3.1	How to Write an Example	9
3.2	Quick examples overview:	10
3.3	Detailed examples	25
3.4	Linear hydrodynamics examples	51
3.5	Flexible bodies (FSM)	58
4	Developer’s documentation	79
4.1	Project Layout	79
4.2	Public API summary	80
4.3	Multibody system creation workflow	82
4.4	Tests	82
4.5	API	86
	Index	139

EXECUTIVE SUMMARY

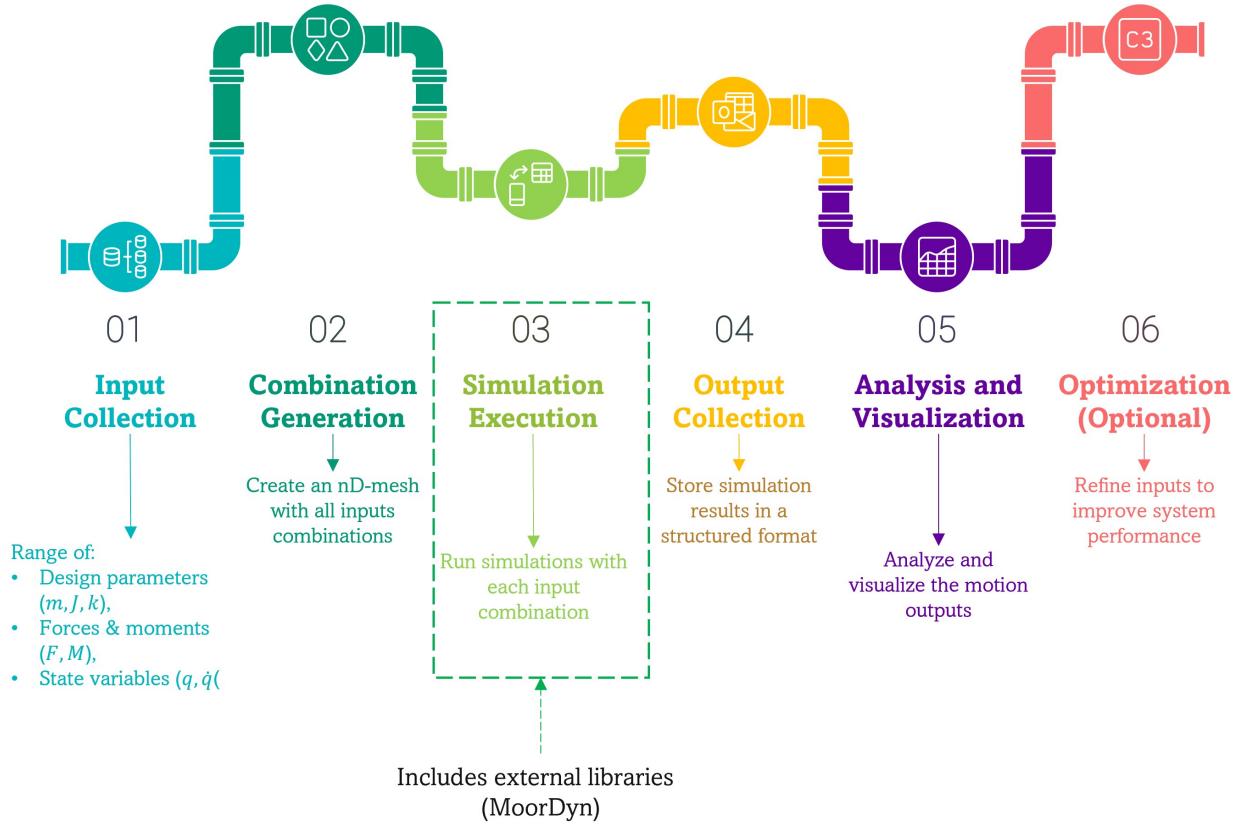
Welcome to Multibody4Everybody, a symbolic multibody dynamics engine designed to be accessible, efficient, and extensible. Multibody dynamics (MBD) allows us to model mechanical systems composed of interconnected rigid (or flexible) bodies.

These systems are common in robotics, biomechanics, aerospace, marine engineering, and many other fields. However, many existing tools for MBD are either closed-source, difficult to extend, or require advanced knowledge to use effectively.

This project was developed to provide a user-friendly and symbolic multibody framework that simplifies the modeling process while retaining full flexibility and mathematical transparency. Additionally, this code is an efficient tool for design exploration, controls, and optimization.

At a glance, this is what is possible with Multibody4Everybody design optimization:

Multibody Framework Simulation Process



1.1 Why a New Multibody Tool?

Several open-source MBD libraries already exist:

- Chrono is a powerful C++-based library with many add-on modules, but it does not provide symbolic equations of motion or use reduced-coordinate formulations.
- PyDy (Python Dynamics) uses Kane's method and offers symbolic equation generation—but it often requires deep familiarity with multibody theory and SymPy mechanics.
- MBDyn Uses a redundant-coordinate, monolithic differential-algebraic formulation, with kinematic constraints and reaction forces introduced through Lagrange multipliers. It is especially established in aerospace, rotorcraft and aeroelastic simulations.
- Simbody Uses minimal/generalized internal coordinates and Featherstone-style recursive rigid-body algorithms, giving roughly $O(n)$ computation for tree systems. Additional loop and user constraints restrict the motion to a constraint manifold.

- Exudyn Supports both minimal and redundant generalized-coordinate formulations, expressed as second-order ODEs coupled to algebraic constraint equations. For flexible bodies it supports formulations including ANCF, floating-frame-of-reference and reduced-order models.

In contrast, Multibody4Everybody is built for:

- Ease of use, even for those new to MBD
- Symbolic generation of the equations of motion (EOMs) for design and optimization
- Efficient simulation using minimal (joint) coordinates
- Plug-and-play interfaces for external forces, including libraries like MoorDyn

1.2 How Multibody4Everybody Works

To use the code, all you need is a single input file, native python or YAML file. In short, you define the kinematics of the system (a snapshot at its initial condition) and the forces and points where forces are applied.

This file contains structured information about your mechanical system:

1. Define the reference frame and bodies
 - Specify the location of the global origin (the y-axis is perpendicular to the working plane).
 - Define the bodies and how they are connected (parent-child relationships).
 - Define the center of mass (CG) of each body and the location of each joint relative to the CG (in the body-fixed frame).
 - Describe the type of joint connecting each body (e.g., revolute, prismatic, floating).
2. Body Points (Optional)
 - Add body points, defined in the body-fixed frame and relative to the CG of the corresponding body, which move with the body.
 - Add ground points, in the global frame, which remain fixed. These can later be used for applying forces or constraints.
3. External Forces (Optional)

- Apply forces or torques to the CG or to body points defined above.
- Add springs, dampers, or mooring lines using a consistent format. The springs and dampers can be linear or nonlinear.
- Mooring systems can be defined via MoorDyn, using the same interface style as springs and dampers.

4. Numerical Parameters

- Provide initial conditions (positions and velocities).
- Set body properties like mass and inertia.
- Define simulation parameters: gravity, total simulation time, and time step.

The numerical parameters step is needed to simulate the system over time. The parameters defined above could have been symbolic or numeric—once everything is defined, the code can generate and simulate the system, animate its motion, and optionally save it as a GIF.

1.3 Validation and Help for New Users

Each example included in the package contains the structure described above, along with:

- Automatic checks to validate that joints, points, and forces are defined consistently.
- Table displays for joints, points, and forces—allowing visual inspection of each section.
- Initial condition guidance, showing which variables require starting values, even if you're unfamiliar with the joint coordinate method.

For clarity, each example ends with a short reference section explaining the expected format for each field, including valid values and data types. This documentation is designed to help both new and advanced users define systems correctly.

1.4 Symbolic or Numeric—Your Choice

A key strength of this package is flexibility: any quantity you define can be symbolic (for symbolic derivations) or numeric (for direct simulation). You can start simple and build complexity as needed.

This introduction is intended to help you get started quickly and confidently. If you're new to multibody modeling, don't worry—the examples and built-in checks will guide you.

INSTALLATION

The goal of this chapter is to help the user install the python package as an external library that can be used no matter where it is installed. This is common practice in the open-source community. To proceed, the user must open a terminal at the location where they want to download the package for installation, typically the downloads folder.

To install the package without prior knowledge we recommend following option number one: CONDA option. This requires having conda installed in the computer. For how to install conda please visit: [conda installation](#) Once conda is installed proceed with the following steps.

Common step: to install using either conda or pip, first run on the terminal:

```
# Clone the repo and enter the directory
git clone https://github.com/Project-SEA-Stack/Python_multibody_dyamics.git
cd Python_multibody_dyamics
```

Option 1: then for CONDA environment creation and installation run:

```
# Create and activate the Conda environment
conda env create -f multibody_env.yaml
conda activate multibody_env
```

Option 2: for PIP installation in an already existing environment, run either option:

```
pip install .      # Option 1, for users
pip install -e .   # Option 2, for developers
```

2.1 Documentation

To build the documentation (if using Sphinx) make sure you have all dependencies installed:

```
pip install Sphinx
pip install sphinx-autodoc-typehints
pip install myst-parser
pip install furo
```

```
cd documentation
make html
```

USER DOCUMENTATION

This chapter aims to help understand the M4E 2D code usage. This is not the documentation for developers, but rather teaches how to define new examples to use the tool without knowledge of the math or what happens in the backend. What you can expect from this chapter is to learn how to define the multibody system either using numeric values or symbolic values and how to loop over multiple symbolic parameters.

You will learn how to define simple bodies connected by Float, Revolut, or Prismatic joints. You will learn how to understand the visualization of your defined example, what are the degrees of freedom associated with each joint briefly and how to animate your simulation (the cool part).

3.1 How to Write an Example

The goal of the examples section is to build knowledge in a dynamic fashion. For that reason, we have decided to increase the explanations step-by-step and the amount of information each example contains increases as you progress in this tutorial.

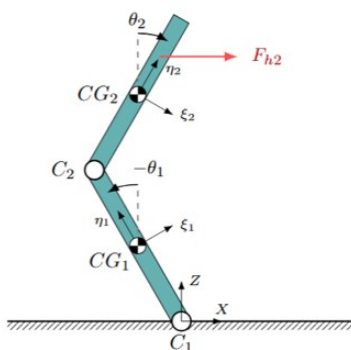
This section is organized into three main steps:

1. Quick examples overview explain how to set your example file without going into detailed explanation of what each variable means.
2. Detailed explanation examples, where bodies, points and forces definition are tackled in detailed.
3. Validation of the equations of motion (EOM) comparing results to a benchmark example.

3.2 Quick examples overview:

3.2.1 Example 1: Double pendulum

The double-pendulum example is a multibody dynamics benchmark, which we use to validate the framework. For the analytical solution refer to Reference to theoretical paper. The following image represents the sketch of a double pendulum following the notation from the publication above. This is example number 1 from the default examples.



1 - Define origin of inertial frame of reference (f.o.r)

This section allows for origin offset, but how to use it is tackled in the next example.

```
# Example 3
Reference_frame-Origin = np.array([0,0])
```

2 - Bodies definition

a. Symbols for bodies (optional)

If you'll define symbolic body variables, initialize them first. Otherwise, you can leave this lists empty :

```
# Bodies definition
dataNames = [] # Define symbolic variables
BodyDataSym = symvars_definition(dataNames, globals())
```

b. Joint connectivity

Define lists for all the fields below:

```
joints          = [[0, 1],[1, 2]]  # Joint connectivity: [parent,
child]
types           = ['R', 'R']       # Joint types: 'R' for revolute, 'P
→' for prismatic, 'F' for floating
parent_cg_to_joint = [[3, 0],[0,1]] # Vectors from parent's center-of-
→gravity (CG) to the joint location.
joint_to_child_cg  = [[0, 2],[0,4]] # Vectors from the joint to the
child's CG.
prismatic_direction = [[np.nan, np.nan],[np.nan, np.nan]] # For prismatic
joints, the direction vector; for others, [nan, nan] is used.
prismatic_direction = normalize_prismatic(prismatic_direction)
```

3 - Points definition

This section allows to define additional points on top of the CG points and joints. For now we leave it blank. Even if not used, these fields must be declared empty.

```
# Points definition
PointNames = [] # Define symbolic variables for force points (for
example, b1, b2, b3).
PointsSym   = symvars_definition(PointNames, globals())

# Define the structure for initial points.
Initial_Points = {}
# Ground points: these are fixed and given as [x, z] coordinates.
Initial_Points["GR"] = []

# Body-defined points ("BD"): we use a dictionary where each key (an
integer)
# corresponds to a body and the value is a list of points.
Initial_Points["BD"] = {}
```

4 - Forces definition

This section allows to define forces at specific location (CG, joints, additional points) and between bodies. For now we leave it blank. Even if not used, these fields must be declared empty.

```
# Forces definition
ForceNames = [] # Define symbolic variables for force points (for
example, b1, b2, b3).
ForceSym    = symvars_definition(ForceNames, globals())

# Define the structure for Force.
Force = {}
# Forces applied on body-defined points (PointsBD):
Force["PointsBD"] = []

# Forces applied at the center of gravity (CG):
Force["CG"] = []

# Tension springs: stored as a list of tuples: (connection, [l0,
stiffness]).
# Here "BD4_1" means (for example) the 1st point on body 4.
Force["TensionSpring"] = []

# Tension dampers: here we have two pairs.
# The first pair uses a constant damping coefficient.
# The second uses a lambda function to represent a nonlinear damping
coefficient.
Force["TensionDamper"] = []

# Torsion springs: here the first element is a list of parameters and the
second element is a lambda.
Force["TorsionSpring"] = []

# Torsion dampers:
Force["TorsionDamper"] = []

ForcesPointsSym = PointsSym + ForceSym
```

5 - Define numerical values

Now that joints, points, and forces are defined, you need to provide numerical values for:

- Initial conditions (positions/velocities)
- Mass & inertia of each body
- Gravity value (default 9.81)
- Gravity vector (percentage of “g” per body)
- Simulation time & timestep
- Animation options

a. Initial Conditions (ICs)

When bodies are initially defined, each body is given a body frame of reference which is parallel to the global frame of reference. The joint-coordinates (explained later) are initialized to zero at this initial configuration. This means that if we define our pendulum vertically it is not going to move, but otherwise it will and the initial conditions are defined as zero at that initial snapshot.

The second set of numerical values to initialized is `ForcesPointsNum` and `BodyDataNum`. These variables give a numerical quantity during time integration to quantities originally defined as symbolic. By default we always give it a value of 1 and they must always exist, even if unused.

```
# Initial conditions
# Create the JointSystem using the from_data class method.
joint_system      = JointSystem.from_data(joints, types, parent_cg_to_
→joint, joint_to_child_cg, prismatic_direction) # DO NOT MODIFY
Q, QD, _, NDOF, _ = joint_system.coordinate_finder() # DO NOT MODIFY

# If you are unsure what are the systems DOF run the example and you will
see on screen
ic                = np.zeros(2*sum(NDOF)) # Multiplied by 2 because is
position and velocity

# Parameters to loop over
ForcesPointsNum = np.ones(len(ForcesPointsSym))
BodyDataNum     = np.ones(len(BodyDataSym))
```

b. Time & Simulation parameters

Here we define values such as the mass and inertia of our bodies, the value of gravity (optional) and the percentage of gravity applied to each body (yes, not all bodies need to be subject to the same gravity value, imagine a body is floating in water and others are not). We also define for how long we want to run our simulation and how big the time between integration steps will be.

```
# Simulation
TimeStep      = 0.05
tspan         = 5.
g             = 9.81                # gravity
gVec          = np.ones((len(types),1)) # Percentage of gravity acting on
each body
m0            = np.ones(len(types))
J0            = np.ones(len(types))
```

c. Animation parameters Very straight forward, do you want to visualize the animation? Yes (1) or no(0)?. Do you want to save it as gif? None for no and string name as 'name.gif' if you do. Lastly, is the animation too slow? Just speed it up by specifying how many time steps to skip (natural numbers only) with the variable `plotTstep`.

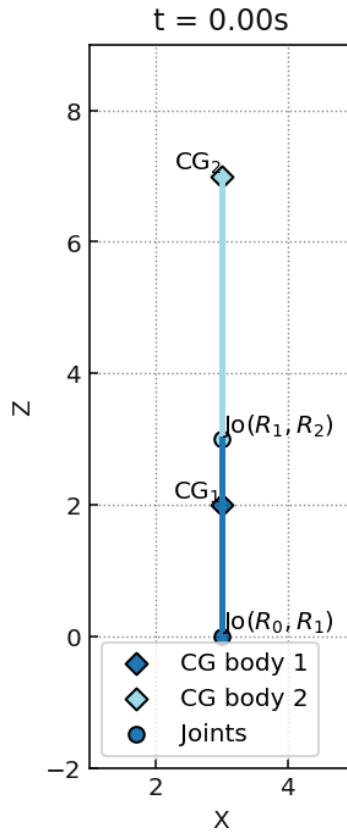
```
# Animation
animation_on   = 1      # Display animation:1, 0-otherwise
SaveMovieOn    = None   # Extension:gif Saves animation if given a string
as name, otherwise set as None
plotTstep      = 5      # how many times faster are we plotting w.r.t to
simulation time
```

6 - call from main

Having defined the example it only remains importing it in the main file a running it. To import the double flap example make sure the line in main where the Examples folders is, looks like this:

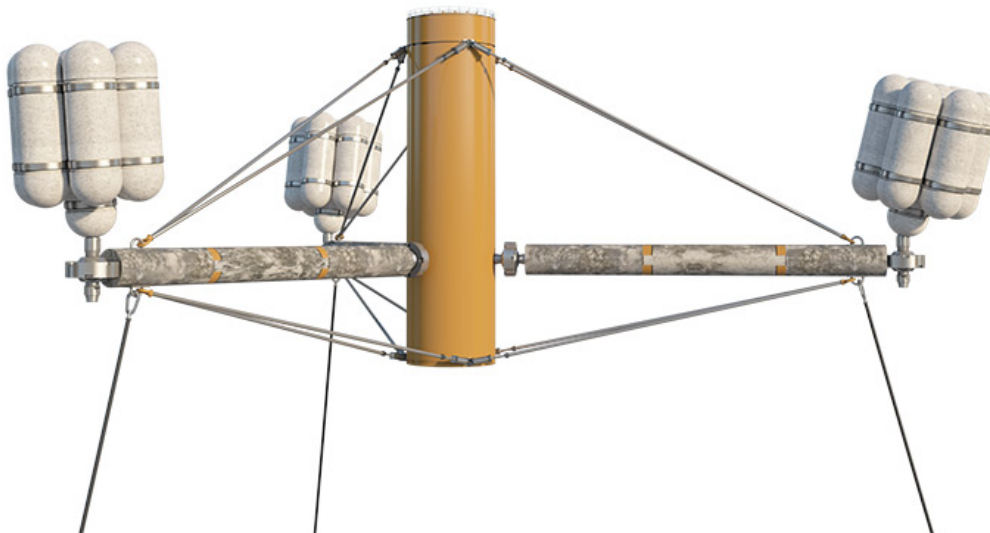
```
from Examples import _1_Double_pendulum as ex
```

Then just run the program! You should see a plot for the mechanical energy, which should be constant, and a plot with the initial snapshot. The initial snapshot should look like this:



3.2.2 Example 2: Spiderfloat

The Spiderfloat device (developed under ARPA-E ATLANTIS) is an offshore-wind platform designed to produce 1,200 GW of green energy. To see more, visit the [NREL Spiderfloat page](#). This example has reduced explanation on what each element to be defined is and focuses more on a general overview on how to set it up.



1 - Define origin of inertial frame of reference (f.o.r)

The user can choose the value of gravity and the origin of f.o.r which will influence how the bodies appear in the global frame of reference. If `Reference_frame_Origin` is provided other than `[0,0]`, then the origin will be located at `Reference_frame_Origin` from the point `[0,0]`. Then the coordinates `parent_cg_to_joint` for the float are measured from this `Reference_frame_Origin` point.

```
# Spiderfloat example
Reference_frame_Origin = np.array([10,5])
```

2 - Bodies definition

a. Symbols for bodies (optional)

If you'll define symbolic body variables, initialize them first. Otherwise, you can leave this lists empty :

```
# Bodies definition
dataNames    = []      # Define symbolic variables
                        # for force points (for example, b1, b2, b3).
BodyDataSym = symvars_definition(dataNames, globals())
```

b. Joint connectivity

Define lists for:

- `joints`: each entry is `[parent_body, child_body]`
- `types`: corresponding joint type for each pair ('R', 'P', or 'F')
- `parent_cg_to_joint`: vector from parent's CG to the joint location
- `joint_to_child_cg`: vector from joint to the child's CG
- `prismatic_direction`: direction of translation for prismatic joints (set to `[np.nan, np.nan]` for 'R' or 'F' joints).

```
# (a) Joint connectivity: [parent, child]
joints = [
    [0, 1],
    [1, 2],
    [1, 3],
    [2, 4],
    [3, 5]
]

# (b) Joint types: 'R' = Revolute, 'P' = Prismatic, 'F' = Floating
types = ['F', 'R', 'R', 'R', 'R']

# (c) Parent.CG_to_Joint (meters)
parent_cg_to_joint = [
    [0, 1],      # Parent = 0 to Joint 0
    [0.5, -2],   # Parent = 1 to Joint 1
    [-0.5, -2],  # Parent = 1 to Joint 2
    [4.5, 0],    # Parent = 2 to Joint 3
    [-4.5, 0]    # Parent = 3 to Joint 4
]

# (d) Joint_to_Child.CG (meters)
joint_to_child_cg = [
    [np.nan, np.nan], # For 'F' joint, use [nan, nan]
    [5, 0],           # Joint 1 to Child 2's CG
    [-5, 0],          # Joint 2 to Child 3's CG
    [0, 1.5],         # Joint 3 to Child 4's CG
    [0, 1.5]          # Joint 4 to Child 5's CG
]
```

(continues on next page)

(continued from previous page)

```

]

# (e) Prismatic direction vectors (only for 'P' joints; else use [nan,
nan])
prismatic_direction = [
    [np.nan, np.nan],
    [np.nan, np.nan],
    [np.nan, np.nan],
    [np.nan, np.nan],
    [np.nan, np.nan]
]

# Normalize the prismatic direction (no effect on [nan, nan])
prismatic_direction = normalize_prismatic(prismatic_direction)

```

Notes:

- *Order matters. Parent bodies should be already defined for every new joint you create. For example, if you haven't defined body 3 before, you can't say body 3 is parent of body 4
- For floating joints ('F'), set their *joint_to_child_cg* vectors to [np.nan, np.nan].

3 - Points definition

Points let you apply forces at specific locations. There are two categories:

- Ground points ("GR"): fixed in model coordinates. The coordinates are relative to `Reference_frame_Origin` to be consistent with the rest of the framework and other tools such as Project Chrono.
- Body-defined points ("BD"): move with their respective body

Initialize the point dictionaries `Initial_Points` (even if you don't use them):

```

# Symbolic point names (if needed):
PointNames = []
PointsSym = symvars_definition(PointNames, globals())

```

(continues on next page)

(continued from previous page)

```
# Initialize containers
Initial_Points = {}
Initial_Points["GR"] = [] # Ground points (list of [x, y] coordinates)
Initial_Points["BD"] = {} # Body points (dict: body_index + list of [x, y] for that body)
```

a. Define ground points

```
# Example: two ground points at (20, 0) and (0, 0)
Initial_Points["GR"] = [
    [20, 0],
    [0, 0]
]
```

b. Define body points

```
# Example: Body 1 has four points; Body 2 has two; Body 3 has two
Initial_Points["BD"][1] = [
    [0.5, -2.5],
    [-0.5, -2.5],
    [0.5, 5],
    [-0.5, 5]
]
Initial_Points["BD"][2] = [
    [4, 0],
    [4.5, 0]
]
Initial_Points["BD"][3] = [
    [-4, 0],
    [-4.5, 0]
]
```

Remember: Even if you don't need ground points or body points, you must still initialize:

```
Initial_Points["GR"] = []
Initial_Points["BD"] = {}
```

4 - Forces definition

Initialize symbolic force names (if needed):

```
ForceNames = []
ForceSym    = symvars_definition(ForceNames, globals())

Force = {}
```

Then define each force category. Even if a category has no entries, assign an empty list so that the “checks” run consistently (especially if you’re converting from MATLAB, which might implicitly remove empty variables).

a. Point forces applied to the body-defined points

```
# Each entry: [body_index, point_index, Fx, Fz, My]
Force["PointsBD"] = [
    # Example: apply a moment of 1 at Body 4's 1st point [4, 0, 0, 0, 1]
    (no forces here for Spiderfloat)
]
```

b. Forces applied to the body centers of gravity (CG)

```
# Each entry: [body_index, Fx, Fz, My]
Force["CG"] = [
    # Example: [2, 5, 0, 0] applies Fx=5 N on Body 2's CG
]
```

c. Tension springs

```
# Each entry is ((pointA, pointB), [l0, stiffness])
# pointA/pointB are strings like "BD1_0" = Body 1 Point 0, "GR0_1" =
Ground Point 1
Force["TensionSpring"] = [
    (("BD1_0", "BD2_0"), [7.0139, 1e0]),
    (("BD1_1", "BD3_0"), [7.0139, 1e0]),
    (("BD1_2", "BD2_1"), [11.8004, 1e-1]),
    (("BD1_3", "BD3_1"), [11.8004, 1e-1]),
    (("BD2_0", "GR0_0"), [3.0414, 1e-1]),
```

(continues on next page)

(continued from previous page)

```
(("GR0_1", "BD3_0"), [3.0414, 1e-1])
]
```

d. Tension dampers

```
# Each entry: ((pointA, pointB), damping_coefficient)
Force["TensionDamper"] = [
    (("GR0_0", "CG2_2"), 3),
    (("GR0_1", "CG3_3"), 3)
]
```

e. Torsion springs & dampers

If you have torsion springs or torsion dampers, add them—even if empty—to keep the structure consistent:

```
Force["TorsionSpring"] = [
    # ((bodyA, bodyB), [theta0, k_theta])
]

Force["TorsionDamper"] = [
    # ((bodyA, bodyB), c)
]
```

Key point:

- `PointsBD` and `CG` use lists of lists (each inner list = one force point).
- Springs and dampers use lists of tuples.

5 - Define numerical values

Now that joints, points, and forces are defined, you need to provide numerical values for:

- Initial conditions (positions/velocities)
- Mass & inertia of each body
- Gravity value
- Gravity vector (percentage of “g” per body)

- Simulation time & timestep
- Animation options

a. Initial Conditions (ICs)

You can either manually specify every initial position and velocity if you are already familiar with joint coordinates, or use `coordinate_finder()` to find all the positions and velocity that must be initialized.

We will explain later in more detail, but for now just know that when we have a float joint, the position coordinates (X,Z), must match `parent_cg_to_joint` corresponding to the float, and if it doesn't, the code will automatically adjust those initial conditions while displaying a warning.

```
# Build the JointSystem (DO NOT modify)
joint_system = JointSystem.from_data(
    joints,
    types,
    parent_cg_to_joint,
    joint_to_child_cg,
    prismatic_direction
)

# Extract coordinate info (DO NOT modify)
Q, QD, _, NDOF, _ = joint_system.coordinate_finder()
```

- NDOF is an array that gives the number of degrees of freedom for each joint.
- The total number of position variables is `sum(NDOF)`.
- Because we need both positions and velocities, the total IC length is `2 * sum(NDOF)`.

If you're unsure, initialize everything to zero:

```
# Zero initial conditions (positions + velocities)
ic = np.zeros(2 * sum(NDOF))
```

You can then modify `ic[...]` if you want specific starting position/velocities.

b. Symbolic Parameters to Loop Over (optional)

If you want to explore multiple force or body parameters, define arrays of symbolic values:

```
# For example, if ForceSym and BodyDataSym are nonempty:
ForcesPointsNum = np.ones(len(ForceSym)) # Start with ones
BodyDataNum      = np.ones(len(BodyDataSym)) # (modify later if needed)
```

At the moment, we initialize them to ones (no effect). You can expand this later to loop over parameter sets. This portion still requires an example

c. Time & Simulation Parameter

```
TimeStep = 0.05      # [s] integration interval
tspan     = 10.0     # [s] total simulation time (final time)
```

d. Gravity per Body

- `g`: global gravity constant (e.g., 9.81)
- `gVec`: an array of size `(n_bodies, 1)` specifying the percentage of gravity each body experiences. For example, if you want zero gravity on Body 2, set `gVec[1] = 0.0`.

```
g      = 9.81          # standard gravity [m/s²]
gVec = np.ones((len(types), 1)) # 100% gravity on each body
```

e. Mass & Inertia

Provide a 1-D NumPy array for each:

- `m0`: mass of each body
- `J0`: moment of inertia of each body (about its CG)

```
m0 = np.ones(len(types)) # Example: all masses = 1 kg
J0 = 2.0 * np.ones(len(types)) # Example: all inertias = 2 kg·m²
```

f. Animation Options

These flags control whether you see an on-screen animation and/or save a GIF:

- `animation_on`: 1 to display, 0 to skip
- `SaveMovieOn`: set to None (no file) or e.g. "Spiderfloat.gif"
- `plotTstep`: draw every `plotTstep`-th frame (for speed)

```

animation_on = 1      # 1 = show animation; 0 = skip
SaveMovieOn  = None   # "Spiderfloat.gif" to save, or None to skip
plotTstep    = 5      # Plot 1 frame every 5 simulation frames

```

6 - call from main

Having defined the example it only remains importing it in the main file a running it. To import the spiderfloat example make sure the line in main where the Examples folders is read looks like this:

```

from Examples import spiderfloat as ex

```

Then the multibody system is created with the lines:

```

# 1- Initialize the Multibody system
MBDsys = MbdSystem.from_example(ex)

```

The next step is to integrate over time. For that we need to define our variable containing all numeric values as:

```

# 3 - Define initial numerical values
mainNumVars = np.hstack((ex.ic, ex.ForcesPointsNum, ex.BodyDataNum))

```

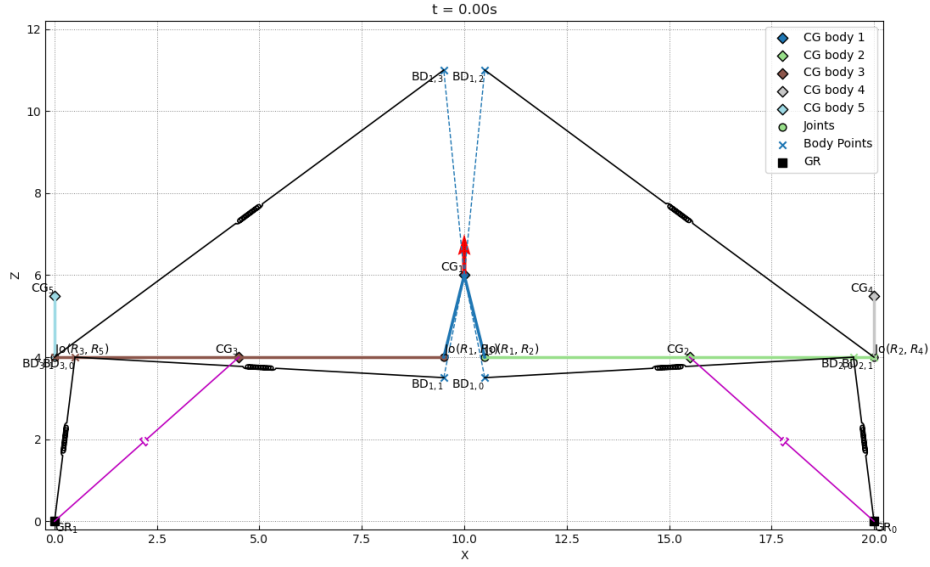
Then we can integrate the system

```

# 3 - Integrate over time
sol      = MBDsys.integrate(mainNumVars, ex.m0, ex.J0,
                           tspan=ex.tspan, dt=ex.TimeStep, external_
↳manager=None)

```

If everything went correctly, your first frame should look like this:



3.3 Detailed examples

3.3.1 Example 1: FOSWEC

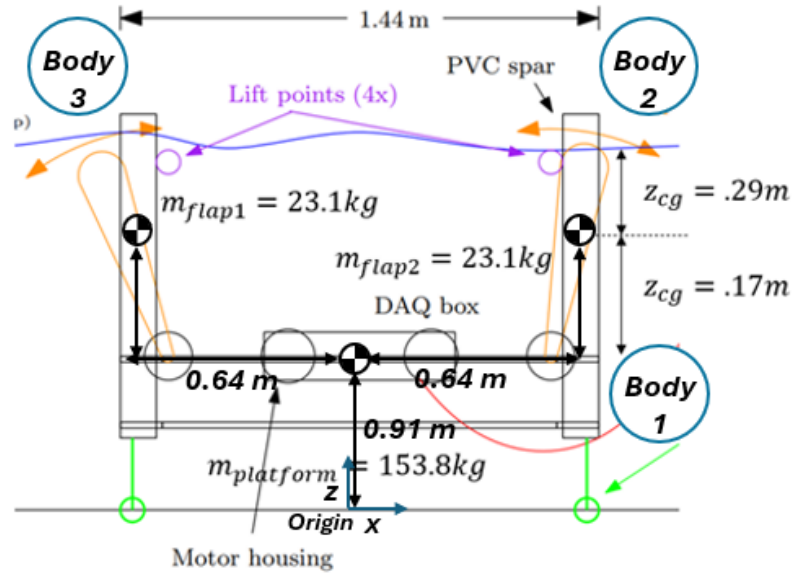
To model a multibody system, you need to define a kinematic sketch of the system at its initial condition. These are the steps we use to create the sketch of the system:

1. Establish the model's origin (Default is $[0,0]$)
2. Defining the joints of the system (starting with joint 1 which always connects body 1 to ground via prismatic, revolute, or floating joint).
3. Each joint has a parent and a child (for instance for joint 1, parent is always the ground and its child is always body 1). Then, for each joint, we define two vector: one vector connects the joint location to parent CG and the other vector connects the joint to child's CG. If we do this for all the joints, we have defined a kinematic sketch of the system.

There are a few exceptions:

- For instance, for joint 1, the parent is body 0 (ground) and it doesn't have a CG.
- CG of ground is the inertial frame. Basically, the parentCG to joint 1 shows the location of joint 1.

This example shows how the FOSWEC dynamics model can be created using our proposed code and how correct implementation is verified. Additionally we show the joints DOFs associated to the example. The equations of motion are provided symbolically using joint coordinates (reduced coordinates).



1 - Define origin of inertial frame of reference (f.o.r)

This time we define the origin of the frame of reference 0.91 units underneath the center of mass of body 1. The variable `Reference_frame-Origin` moves the model origin from the point `[0,0]` by the vector `Reference_frame-Origin`. Then, when setting the `parent_cg_to_joint` variable to `[0,0]` it is going to be coincident with `Reference_frame-Origin` because `parent_cg_to_joint` is a vector relative to the model frame.

```
Reference_frame-Origin = np.array([0,0.91])
```

2 - Bodies definition

This time we are presenting the result we achieve to aim when defining the bodies in table format before actual definition, and if the user feels comfortable it is recommended that you try it out before seeing the solution.

Bodies and joints table:

Joint	Connected Bodies	Joint Type	ParentCGtoJoint	JointtoChildCG	Prismatic	Direction
1	0 1	F	[0, 0]	NaN NaN		NaN NaN
2	1 2	R	[0.6400000000000000, 0]	[0, 0.1700000000000000]		NaN NaN
3	1 3	R	[-0.6400000000000000, 0]	[0, 0.1700000000000000]		NaN NaN

This table is automatically generated when running the example FOSWEC.py either as standalone or from main. To obtain such table we must define the bodies as follows

```
joints          = [[0, 1],[1, 2],[1,3]]
types           = ['F', 'R', 'R']
parent_cg_to_joint = [[0, 0],[0.64, 0],[-0.64, 0]]
joint_to_child_cg  = [[np.nan, np.nan],[0, 0.17],[0, 0.17]]
prismatic_direction = [[np.nan, np.nan],[np.nan, np.nan],[np.nan, np.nan]]
prismatic_direction = normalize_prismatic(prismatic_direction)
```

The key point between the bodies definition and how it is displayed as table is that one is the transposed of the other. This means that if we read the information of the code vertically, i.e. column by column, that information will correspond to each row of the table.

Each row of this table represents a kinematic joint. The entire kinematic structure is built by defining the joints and their associated vectors. The types of joints that can be modeled include: Floating Joint (F), Prismatic Joint (P), and Revolute Joint (R).

Let's go over the first row. Each joint has a parent body and a child body. Each row defines how the joint is connected to the child's center of gravity (CG) and the parent's CG. This is how the entire kinematic structure is defined.

- The first column indicates the parent of the joint, and the second column indicates the child.
- The parent of the first defined joint is always set to zero, which represents the ground. The child body, in this case, is body 1.
- The third column specifies the joint type. For body 1, which is connected to the ground, this is a floating joint (representing the platform's base).
- The vectors in the fourth and fifth columns represent the [x,z] components of the vector from the ground (reference frame) to the CG of the floating body.
- The sixth and seventh columns should be set to [NaN,NaN] for floating joints.
- The last two columns are used only for prismatic joints to define the initial direction of motion and should be set to [NaN,NaN] for all non-prismatic joints.

The second row of the table indicates that a revolute joint connects body 1 (the parent) to body 2 (the child). Note that body 1 was already defined in the first row.

- The third and fourth columns specify the vector from the CG of body 1 to the revolute joint. This joint is located at $[0.64, 0]$ meters relative to the CG of body 1.
- The fifth and sixth columns specify the vector from the revolute joint to the CG of body 2, which is $[0, 0.17]$.

In summary, this row defines the connection between the platform's base and the right flap.

The third row indicates that a revolute joint connects body 1 (the parent) to body 3 (the child).

- The third and fourth columns define the vector from the CG of body 1 to the revolute joint, which is located at $[-0.64, 0]$ meters.
- The fifth and sixth columns define the vector from the revolute joint to the CG of body 3, which is $[0, 0.17]$.

In summary, this row adds the left flap to the platform's base.

3 - Joint coordinates DOFs

The joint coordinates associated to this example are extracted at the initial conditions section, but for the sake of explaining a bit the coordinates associated with each type of joint we moved it here.

```
Q, QD, _, NDOF, _ = joint_system.coordinate_finder()
```

Running this line requires creating the `joint_system` but that is already in the line above in the code and it's not important at the moment. If we print on screen these coordinates we get a set of degrees of freedom with repeated subindices, sometimes. Each subindex, points to a different body. Therefore if a subindex is repeated it means that the joint type has multiple DOFs associated to a body. For our example it looks like this:

```
Q' = [ X_1, Z_1, Theta_1, Theta_2, Theta_3]
```

There are three subindices: 1, 2, 3. This means that we have three bodies. The first body was defined as a floating body or joint. Floating joints have in 2D three degrees of freedom, two translations: X, Z and one rotation θ . The second and third bodies are connected to the first one through revolute joints, and this joint have one rotational degree of freedom: θ . So

in summary, $[X_1, Z_1]$ show the position of the CG of the platform and $[\theta_1, \theta_2, \theta_3]$ show the pitch angle of the platform, the first flap, and the second flap, respectively.

4 - Points definition

To define springs and dampers associated to the the bodies, unless these points are either joints or the center of gravity, we must define them on top of the bodies definition. As mentioned in the previous example, if the bodies are fixed we define them as ground points, GR, whereas if this points are on a body, we define them as body points, BD. These points are always relative to their own frame of reference, the model frame defined by `Reference_frame_Origin` for GR, and the body frame for BD. The definition of the points for FOSWEC is summarized in the table below and is used to verify we have defined everything as we wanted.

Points table:

Body	PointType	PointID	Rel coordinates
0	GR	0	[0.75, -0.91]
0	GR	1	[-0.75, -0.91]
1	BD	0	[0.75, 0]
1	BD	1	[-0.75, 0]

- Column 1: body to which the point belongs. Ground is always associated to 0.
- Column 2: Point type
- Column 3: Order in which the point was defined in a body
- Column 4: x-coordinate relative to the CG
- Column 5: y-coordinate relative to the CG

This points are defined in using the following lines of code:

```
# Ground or fixed points
Initial_Points["GR"] = [[0.75, -0.91], [-0.75, -0.91]]
# Body associated points
Initial_Points["BD"] = {}
Initial_Points["BD"][1] = [[0.75, 0], [-0.75, 0]]
```

The variable `Initial_Points` is a dictionary with two entried:

- GR for ground points
- BD for body points

The entry **GR** is a list containing the relative coordinates of each point in an inertial frame of reference, defined by **Reference_frame_Origin**. On the other hand, the entry **BD** is another dictionary, which entries refer to the additional points each body has. Each dictionary entry, contains a list of points. In this example, only body 1 has associated points and it has two points. The coordinates entered for the **Initial_Points['BD']** are relative to the CG. Thus the first point in body 1 is located 0.75 units to the right of the body 1 CG.

After defining these popints, we can access them at the code. For instance point $[0.75, 0]$, the first point defined in body 1 is accessed as 'BD1_0' and point $[-0.75, 0]$ defined on body 1 also is accessed as 'BD1_1'

5 - Forces definition

For the forces definition section, we are going to introduce a new concept, defining symbolic forces. To define symbolic variables in section, we must define the variables as a list of strings in their corresponding section. Note: if a variable is repeated, define it only once. In this example we define the time as symbolic variable, **t**, and the springs stiffness, **k**. The string name is defined under the variable **ForceNames** and the function **symvars_definition** converts those names into variables and places them in the global workspace.

```
ForceNames = ['t', 'k']
ForceSym    = symvars_definition(ForceNames, globals())
```

The final result will look like this:

```
Forces table:
ForceType BodiesConnected PointType PointID ForceComponents SpringParameters DamperParameters
CG          1             CG        NA [0, 0.1*exp(-t), 0]          None             NaN
TensionSpring 0,1       GR,BD      0,0      None          [5, k]           NaN
TensionSpring 0,1       GR,BD      1,1      None          [5, k]           NaN
TensionDamper 0,1       GR,BD      0,0      None          None             1.0
TensionDamper 0,1       GR,BD      1,1      None          None             1.0
TorsionSpring 1,2        JO         NA      None          [0, 10]          NaN
TorsionSpring 1,3        JO         NA      None          [0, 10]          NaN
TorsionDamper 1,2        JO         NA      None          None             0.0
TorsionDamper 1,3        JO         NA      None          None             0.0
```

The explanation for each of the entries is divided into two groups. One point forces PointsBD and CG which are defined as lists of lists and the two points forces which are defined as list of tuples.

- PointsBD: [lists of lists] where each list corresponding to one point contains
 1. Body of application i, which corresponds to the dictionary entry in Initial_points.

2. Element in the list `j` of `Initial_points[i]`.
3. F_x : force in x-direction
4. F_z : force in z-direction
5. M_y : moment in y-direction

In this example `PointsBD` is defined as empty, but must always be defined for consistency with MATLAB code.

```
Force["PointsBD"] = []
```

- CG: [lists of lists] where each list corresponding to one point contains
 1. Body of application `i`, which corresponds to the dictionary entry in `Initial_points`.
 2. F_x : force in x-direction
 3. F_z : force in z-direction
 4. M_y : moment in y-direction

With the time variable we are going to define an exponential force at the CG of body 1, with only vertical component. This force is an exponential decay function $F_y = e^{-t}$.

```
Force["CG"] = [[1,0,0.1*sym.exp(-t),0]]
```

- TensionSpring: list of tuples where each tuple contains
 1. Tuple of strings that refer to the two points connected by the spring
 2. List of parameters containing [l_0 : undeformed length, k : stiffness constant if linear spring (will be clarified later)]

In this example we are connecting the first ground point, 'GRO_0', with the first point in body 1, 'BD1_0'. These points were already defined in the dictionary `Initial_Points`. Then we define the unstretched length, l_0 as 5, and the stiffness, k , will be symbolic.

```
Force["TensionSpring"] = [ (('GRO_0', 'BD1_0'), [5,k]),
                             (('GRO_1', 'BD1_1'), [5,k])
                           ]
```

- TensionDamper: list of tuples where each tuple contains
 1. Tuple of strings that refer to the two points connected by the damper

2. float containing c : damping coefficient if linear spring (will be clarified later)

Tension dampers are defined in a very similar manner to tension springs. The points are defined in the same manner, however, when defining the parameters, instead of passing a list we pass a float corresponding to the damping coefficient.

```
Force["TensionDamper"] = [ (('GRO_0', 'BD1_0'), 1),
                           (('GRO_1', 'BD1_1'), 1)]
```

- TorsionSpring: list of tuples where each tuple contains
 1. Tuple of strings or floats that refer to the two bodies connected by the damper
 2. list containing $[\theta_0$: undeformed angle, k_θ : stiffness]

Torsion springs follow the same structure as Tension Springs, however, instead of defining the points connected we defined the bodies connected as this springs are only conceived to be at the joints between bodies. These points can be defined as a **tuple** of two **floats** or two **strings**. The parameters are defined in the same fashion as tension springs.

```
Force["TorsionSpring"] = [ ((1,2), [0,10]),
                           ((1,3), [0,10])]
```

- TorsionDamper: list of tuples where each tuple contains
 1. Tuple of strings or floats that refer to the two bodies connected by the damper
 2. float containing c : damping coefficient if linear spring (will be clarified later)

Torsion dampers follow the same structure as Torsion springs to define the points and the same structure as tension dampers to define the damping coefficient.

```
Force["TorsionDamper"] = [ ((1,2), 0),
                           ((1,3), 0)]
```

6 - Initial Conditions

As opposed to frameworks using redundant coordinates (not using joint coordinates or Kane's method), the initial conditions will not be updated due to incompatibility issues. This means that you can insert any value and the tool will solve as you requested except for the position of the float joints. The float joints require the position of their CG to be defined and the coordinates represent the CG position itself, as opposed to other coordinates.

Therefore the initial position of the float coordinates must be equal to the corresponding `parent_cg_to_joint`, otherwise it will be automatically updated to the appropriate value while displaying an [Info] message on the screen.

```
ic = np.zeros(2*sum(NDOF)) # Will automatically be updated to [0,1]
```

7 - Simulation parameters

This step is needed to simulate the system over time. The parameters defined before, could have been symbolic or numeric, to simulate the system over time, you need numerical values.

In this section we must provide at least the time parameters and the mass and inertia properties. The gravity by default is set to 9.81 and applied to all bodies. We define the parameters in the following manner:

- **TimeStep**: specifies the integration time taken in forward integration
- **tspan**: defines the end-time of the integration
- **g**: it is the value of gravity's acceleration
- **gVec**: it is a vector that defines, for each body, a gravity multiplier, i.e.: a value of 2 in the first position would indicate that the first body is subject to 2 times the value of **g**.
- **m0**: it is a 0-D array specifying the mass of each body
- **J0**: it is a 0-D array specifying the second moment of inertia of each body

```
# Simulation
TimeStep    = 0.05
tspan       = 10.
g           = 0.                # gravity
gVec        = np.ones((len(types),1)) # Percentage of gravity acting on
each body
m0          = np.ones(len(types))
J0          = 2.*np.ones(len(types))
```

The animation parameters are not related to the integration but rather just visualization. They define:

- **animation_on**: whether we want to display animation or not. It is a boolean and takes values of 0 or 1.

- `SaveMovieOn`: if name is specified with `.gif` extension, it saves the animation in the root folder.
- `plotTstep`: it is used to speed up the animation and specifies every how many time steps to plot. It must be an integer.

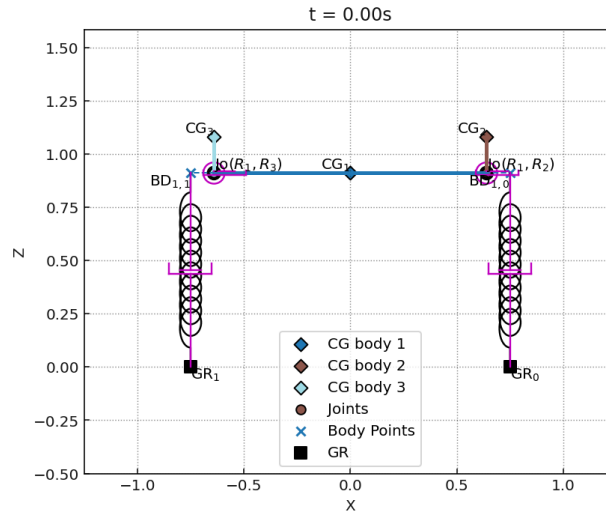
```
# Animation
animation_on      = 1          # Display animation:1, 0-otherwise
SaveMovieOn       = None       # Extension:gif Saves animation if given a
                                # string as name, otherwise set as None
plotTstep         = 5          # how many times faster are we plotting w.r.t
                                # to simulation time
```

8 - Simulation and results

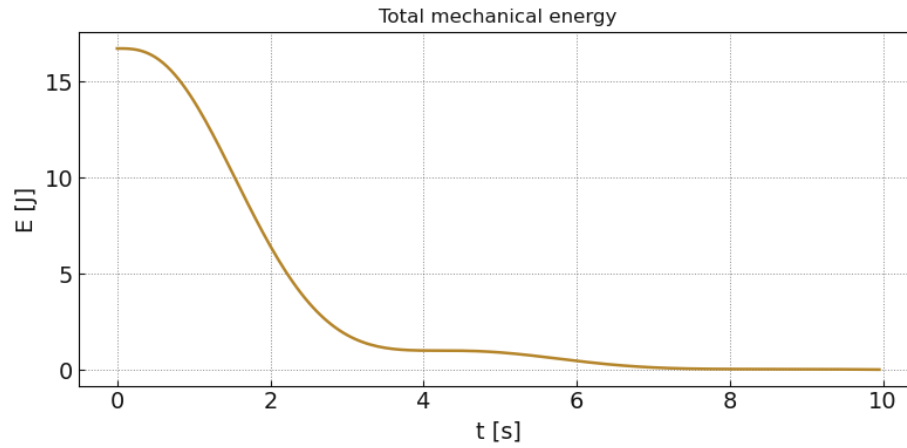
To run this example you only need to update one line in the main file. To import the FOSWEC example it must look like:

```
from Examples import FOSWEC as ex
```

Foswec initial configuration looks like this (set `tspan=0.05` to get this visualization):



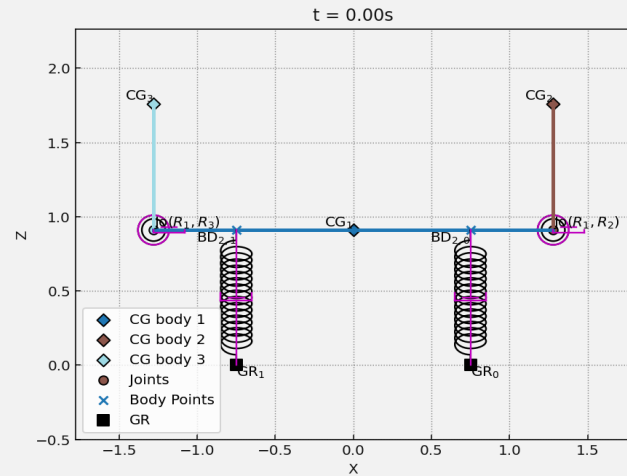
and the energy plot looks:



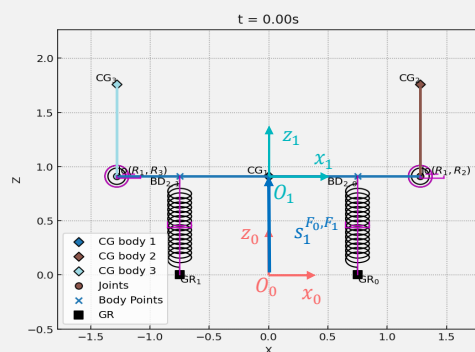
Step-by-step PPT

This portion contains the slides presenting this example. The slides with the animations are in TODO: ref to slides.

FOSWEC



Bodies definition: body 1



Connecting bodies

Joint type

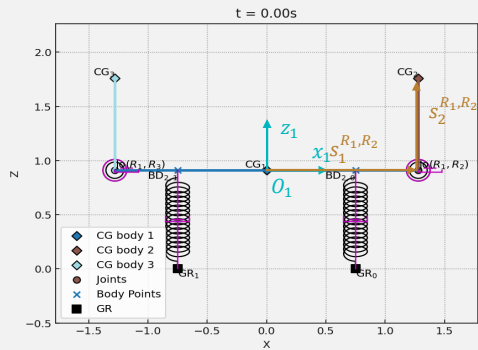
```
joints = [[0, 1], [1, 2], [1, 3]]
types = ['F', 'R', 'R']
parent_cg_to_joint = [[0, 0.91], [2*0.64, 0], [-2*0.64, 0]]
joint_to_child_cg = [[np.nan, np.nan], [0, 5*0.17], [0, 5*0.17]]
prismatic_direction = [[np.nan, np.nan], [np.nan, np.nan], [np.nan, np.nan]]
prismatic_direction = normalize_prismatic_directions(prismatic_direction)
```

$s_0^{F_0, F_1}$
'F' joints only
Parent cg to joint

Direction of allowed translation

Go to next slide to continue animation

Bodies definition: body 2



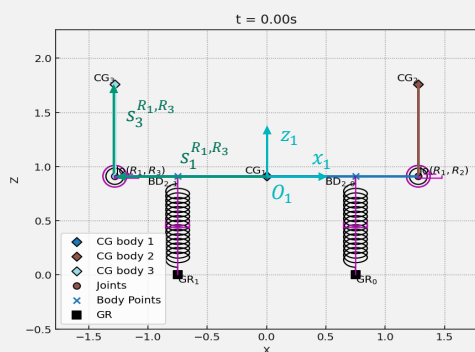
Connecting bodies

Joint type

```
joints = [[0, 1], [1, 2], [1, 3]]
types = ['F', 'R', 'R']
parent_cg_to_joint = [[0, 0.91], [2*0.64, 0], [-2*0.64, 0]]
joint_to_child_cg = [[np.nan, np.nan], [0, 5*0.17], [0, 5*0.17]]
prismatic_direction = [[np.nan, np.nan], [np.nan, np.nan], [np.nan, np.nan]]
prismatic_direction = normalize_prismatic_directions(prismatic_direction)
```

Go to next slide to continue animation

Bodies definition: body 3



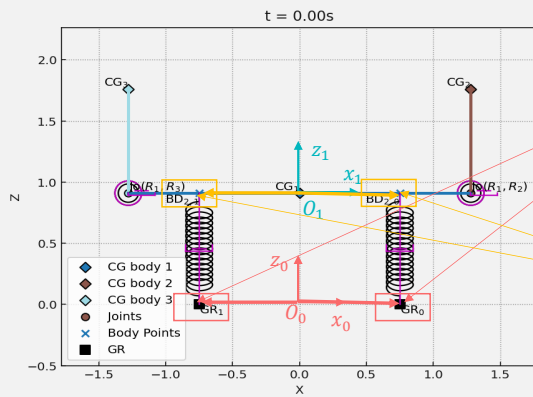
Connecting bodies

Joint type

```
joints = [[0, 1], [1, 2], [1, 3]]
types = ['F', 'R', 'R']
parent_cg_to_joint = [[0, 0.91], [2*0.64, 0], [-2*0.64, 0]]
joint_to_child_cg = [[np.nan, np.nan], [0, 5*0.17], [0, 5*0.17]]
prismatic_direction = [[np.nan, np.nan], [np.nan, np.nan], [np.nan, np.nan]]
prismatic_direction = normalize_prismatic_directions(prismatic_direction)
```

Go to next slide to continue animation

Points definition

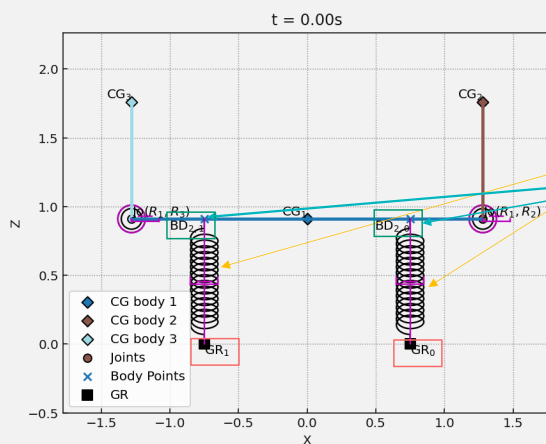


Initial_Points["GR"] = [[0.75, 0], [-0.75, 0]]

Initial_Points["BD"] = {}
Initial_Points["BD"][1] = [[0.75, 0], [-0.75, 0]]

Go to next slide to continue animation

Forces definition: tension springs

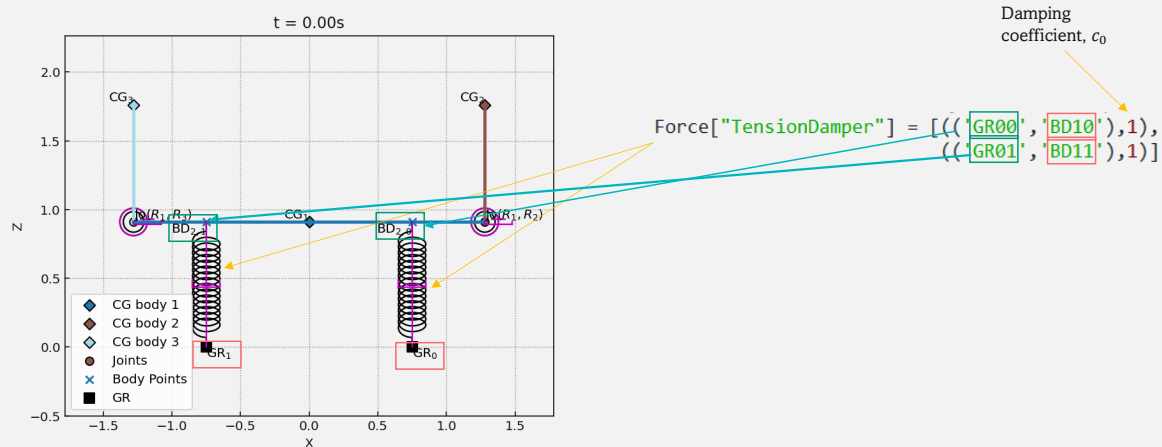


Force["TensionSpring"] = [((["GR00"], ["BD10"]), [5, k]),
((["GR01"], ["BD11"]), [5, k])]

Undeformed length, l_0 Stiffness, k

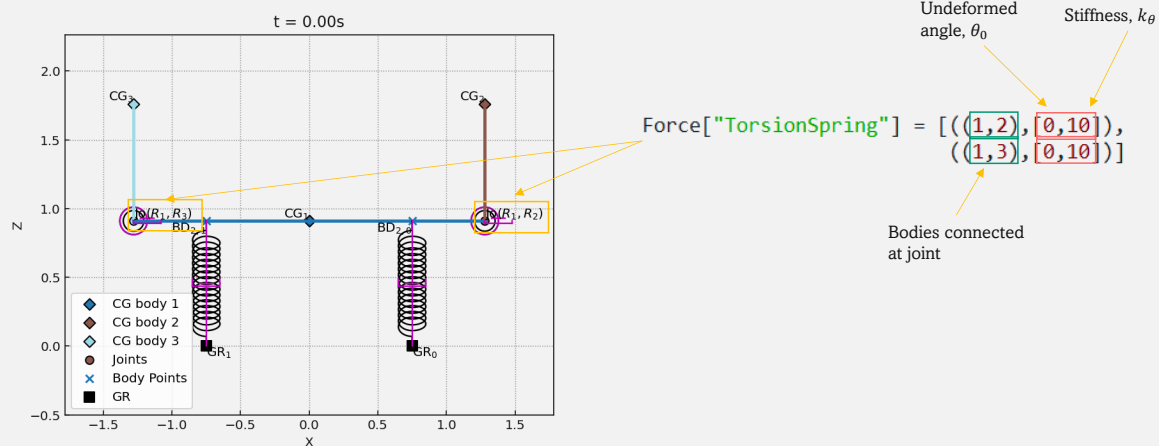
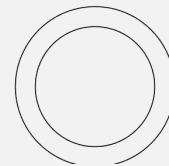
Go to next slide to continue animation

Forces definition: tension dampers



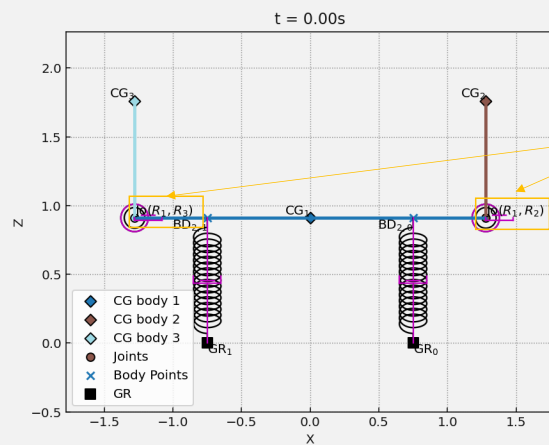
Go to next slide to continue animation

Forces definition: torsion springs



Go to next slide to continue animation

Forces definition: torsion dampers



Force["TorsionDamper"] = [((1,2),0),
((1,3),0)]

Damping
coefficient, c_θ

Bodies connected
at joint

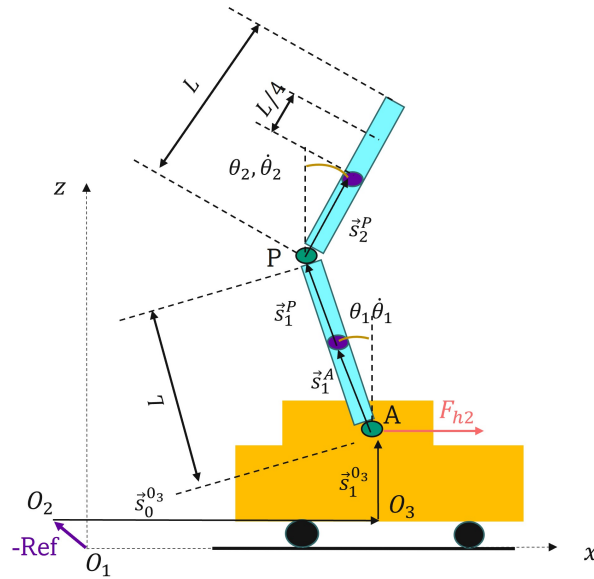
Go to next slide to continue animation

3.3.2 Example 2: Double pendulum on cart

In this example we want to model a well known example in multibody dynamics and now if interest in the machine learning community, double pendulum on a cart. This example is interesting from the control theory perspective, although we will not focus on stabilizing the pendulum on top of the cart but rather visualize the motion. For you is interesting also because you will learn how to model prismatic joints and a dig deeper in the origing offset. Additionally, you will define time dependent forces at the center of mass. This is example 4 of the default examples, except for the displace origin.

1 - Define origin of inertial frame of reference (f.o.r)

We want to model the base of the body to be above the ground 1 unit. One way to do so without over-complicating the vectors `parent_cg_to_joint` and `joint_to_child_cg` is to offset where the bodies definition starts. The way this offset works is a vector that goes from the ground origin to the model origin. In the figure below is a vector going from O_1 to O_2 . This feature may be useful when defining the prismatic direction as horizontal but we want some offset from the ground, if anything is parametric.



```
Reference_frame_Origin = np.array([-1,1])
```

Check the schematics to understand better what `Reference_frame_Origin` means. After the body definition, we will clarify a few points on what it means then to define `parent_cg_to_joint` for the bodies in contact with ground and `joint_to_child_cg`.

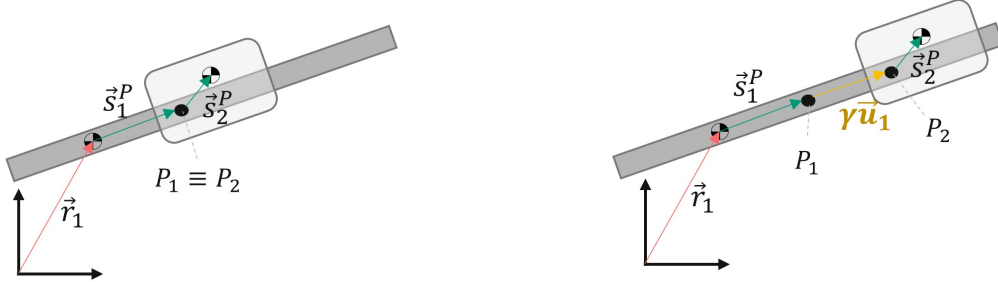
```

joints          = [[0, 1],[1, 2],[2,3]]
types           = ['P', 'R', 'R']
parent_cg_to_joint = [[3, 0],[0,0],[0,1]]
joint_to_child_cg  = [[0,1],[0, 2],[3,0]]
prismatic_direction = [[3, 0],[np.nan, np.nan],[np.nan, np.nan]]
prismatic_direction = normalize_prismatic(prismatic_direction)

```

By moving the reference frame to $[-1, 1]$, then we transformed `parent_cg_to_joint`, vector $\vec{s}_0^{O^3}$, to a horizontal line defined by vector $[3, 0]$. If we want to define this vector symbolically and then the prismatic direction for the cart as that same vector, offsetting the origin becomes handy.

A prismatic joint contains one degree of freedom, like revolut joints, but it is a translational DOF. To define a prismatic joint, in our framework, we need three points and a sliding direction. These points are the center of mass of the two connected bodies and the point of contact, P between the two bodies. In fact there are two points P, one belonging to the first body and another that belongs to the second one. However, initially they are defined in the same location (this can be changed at initial conditions). The sliding direction is defined by the vector $\vec{u}$ which is a unit vector and the magnitude of that vector, γ determines how far apart the two P points are apart as the simulation proceeds. This variable γ is our joint coordinate.



In our example, we have defined the first body as prismatic joint type. This means that we must define a prismatic direction associated to it. We can define it with whatever vector we want but then its norm is normalized using the function `normalize_prismatic` which lives under `multibody`.

Another interesting feature of prismatic joints is that they do not possess their own rotational degree of freedom, and therefore inherit the rotation of the previous body in the chain. In this case the previous body is the ground, hence the cart has no rotation at all and will move only horizontally.

In summary, this system has three bodies connected by prismatic and revolut joint. These joints contribute with 1 translational DOF (prismatic joint) and 2 rotational DOFs (1 per revolut joint). We name the translational DOFs with **S** and the rotational DOFs with **Theta**. Remember that the subindex shows which body they belong to. We can print the DOFs on screen by running the example script by itself. For a closer look you can check the following lines:

```
Q, QD, QDD, NDOF, _ = joint_system.coordinate_finder()
```

These variables will show you the positional DOFs, **Q**, as well as as the velocity and acceleration coordinates, **QD**, **QDD**, respectively. Lastly, **NDOF** summarizes the number of DOFs per joint, so in this case it would be a list showing `[1,1,1]`.

2 - Points definition

This system does not have forces applied outside from the CG, thus no point definition is required. However, the list for **GR** points and the dictionary for **BD** points must be initialized.

```
Initial_Points["GR"] = []
Initial_Points["BD"] = {}
```

3 - Forces definition

The forces definition involves the last learning point of this example. Define a time-dependant force at the CG of body 1. To define a time-dependant force, we must initialize it as a symbolic variable. Since we are using it in the force section we initialize it in this section. If it was used in multiple sections, it should only be initialized in the **FIRST** one. To define this symbolic variable, we define a string **"t"** in the list **ForceNames**.

```
ForceNames = ["t"]
ForceSym = symvars_definition(ForceNames, globals()) # DO NOT MODIFY
```

The second line, where **ForceSym** is defined, and which should not be touched, converts each element in the list into a **sympy Symbol** with the assumption of **real=True**. This is important because manipulation of this variable will involves taking into account that the variable is real. Check **Sympy Gotchas** to dig deeper into the concept.

Once the symbolic variables needed are defined, then we can proceed to define the force of interest. Remember all the force types must be defined, even as empty. We are interested in a force applied on the CG of body 1. Then, in the force dictionary, in the key: **"CG"** we can

create a list for each for applied in the CG. The elements on the list will specify to which body is the force applied to and the components of the force as F_x , F_z , M_y . In our case, we decided to apply a horizontal force with exponential decay. Note that this definitions must follow SymPy syntax.

The force definition lines look like this:

```
Force["PointsBD"]      = []
Force["CG"]            = [[1,sym.exp(-t),0,0]]
Force["TensionSpring"] = []
Force["TensionDamper"] = []
Force["TorsionSpring"] = []
Force["TorsionDamper"] = []

ForcesPointsSym = PointsSym + ForceSym
```

The last line in the code above groups all the symbolic variables defined in the points and forces section to create the numeric association and mapping latter on. It must exists but it is not of the users concern. It appears here cos it is necessary for the checks.

The remaining lines for numerical values and simulation are left for completeness but it requires no user input and they have been already explained.

4 - Numerical simulation and animation

Just a quick comment, we saw that this problem has 3 DOFs, one translational and two rotationals. Since we are solving a 2nd order ODE, it requires initial conditions for position and velocity, thus double of the number of DOFs.

```
# If you are unsure what are the systems DOF run the example and you will
see on screen
ic          = np.array([0,0,0,0,0,0]) # Multiplied by 2 because is
position and velocity

# Parameters to loop over
ForcesPointsNum = np.ones(len(ForcesPointsSym))
BodyDataNum     = np.ones(len(BodyDataSym))

# Simulation
```

(continues on next page)

(continued from previous page)

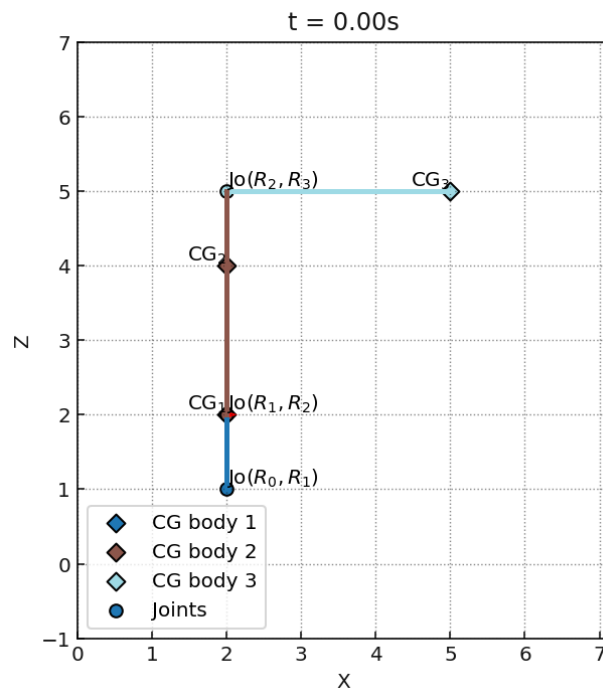
```

TimeStep      = 0.05
tspan         = 5.
g             = 0.                                # gravity
gVec          = np.ones((len(types),1))          # Percentage of gravity acting on
each body
m0            = np.ones(len(types))
J0            = 2.*np.ones(len(types))

# Animation
animation_on  = 0                                # Display animation:1, 0-otherwise
SaveMovieOn   = None                             # Extension:gif Saves animation if given a
string as name, otherwise set as None
plotTstep     = 5                                # how many times faster are we plotting w.r.t
to simulation time

```

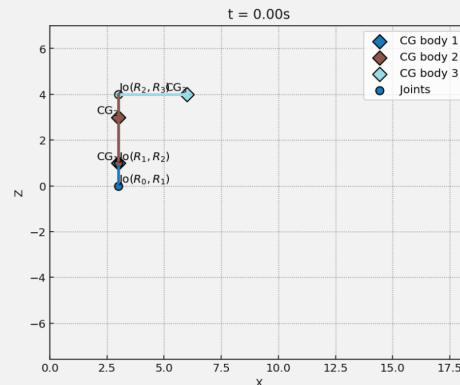
And after all that hard effort ... this is how it looks:



Step-by-step PPT

This portion contains the slides presenting this example. The slides with the animations are in TODO: ref to slides.

Double pendulum on cart



This example focuses on how to define the bodies in the system and the simulation



WPTO - NREL

Example initialization: do not modify

Do not modify:

- ❖ Defines time as symbolic variable and is used in the checks sections
- ❖ Define the dictionary for points and forces

```
# Do not modify
t = sym.symbols('t', real=True)
Initial_Points = {}
Force = {}
# End of Do not Modify
```



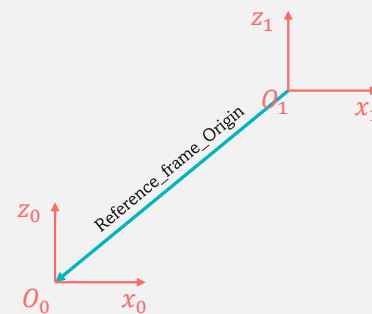
WPTO - NREL

Example initialization: user-defined

Gravity and origin:

- ❖ g is the gravity which can be set to any value, zero is applicable if floating in water
- ❖ `Reference_frame_Origin`: vector from absolute origin to origin of body 1
(important when first body is floating)

```
# Example 4
g = 9.81 # gravity
Reference_frame_Origin = np.array([1,-1])
Reference_frame_Origin = -Reference_frame_Origin
```



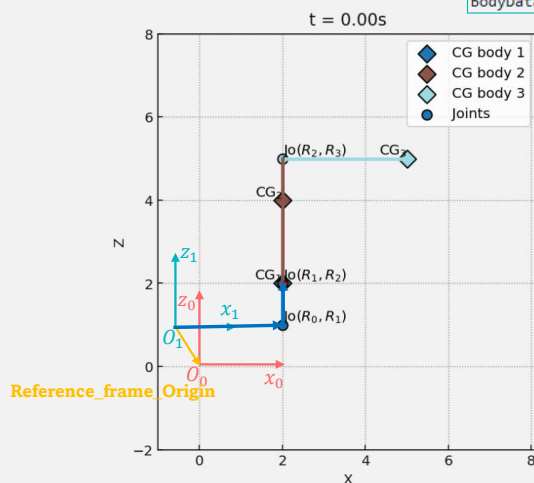
WPTO - NREL

3

Bodies definition: body 1

- Skip for now

```
# Bodies definition
dataNames = [] # Define symbolic variables for force points (for example, b1, b2, b3).
BodyDataSym = symVars_definition(dataNames, globals()) # DO NOT MODIFY
```



Connecting bodies

```
joints = [[0, 1], [1, 2], [2, 3]]
types = ['R', 'R', 'R']
parent_cg_to_joint = [[3, 0], [0, 0], [0, 1]]
joint_to_child_cg = [[0, 1], [0, 2], [3, 0]]
prismatic_direction = [[3, 0], [np.nan, np.nan], [np.nan, np.nan]]
```



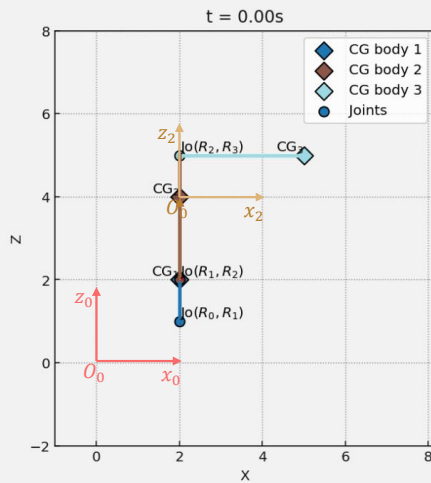
Go to next slide to continue animation



WPTO - NREL

4

Bodies definition: body 2



Connecting bodies

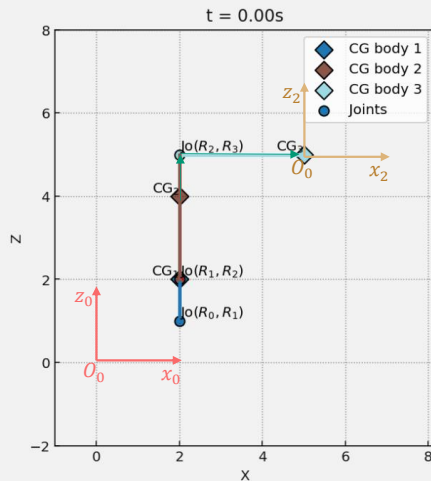
Joint type

```
joints = [[0, 1], [1, 2], [2, 3]]
types = ['P', 'R', 'R']
parent_cg_to_joint = [[3, 0], [0, 0], [0, 1]]
joint_to_child_cg = [[0, 1], [0, 2], [3, 0]]
prismatic_direction = [[3, 0], [np.nan, np.nan], [np.nan, np.nan]]
```

$S_1^{R_1, R_2}$
 $S_2^{R_1, R_2}$

Go to next slide to continue animation

Bodies definition: body 3



Connecting bodies

Joint type

```
joints = [[0, 1], [1, 2], [2, 3]]
types = ['P', 'R', 'R']
parent_cg_to_joint = [[3, 0], [0, 0], [0, 1]]
joint_to_child_cg = [[0, 1], [0, 2], [3, 0]]
prismatic_direction = [[3, 0], [np.nan, np.nan], [np.nan, np.nan]]
```

$S_2^{R_2, R_3}$
 $S_3^{R_2, R_3}$

Go to next slide to continue animation

Bodies definition: table display

Summary of how bodies were defined but organized by rows instead of columns

Bodies and joints table:

Joint	Connected	Bodies	Joint Type	ParentCGtoJoint	JointtoChildCG	Prismatic	Direction
1	0	1	P	3.000	0.000	0.000	1.000
2	1	2	R	0.000	0.000	0.000	2.000
3	2	3	R	0.000	1.000	3.000	0.000

1.000	0.000
NaN	NaN
NaN	NaN

Normalized direction

Simulation and Animation

Simulation

TimeStep = 0.05

tspan = 0.05

gVec = np.ones((len(types),1))

m0 = np.ones(len(types))

J0 = 2.*np.ones(len(types))

Factor to apply a different gravity to each body

Mass and Inertia of each body

Animation

animation_on

= 0

SaveMovieOn

= None

plotTstep

= 5

Boolean to display animation or not

Name with .gif extension if you want to save animation (requires animation_on = 1), None otherwise

Number of timeSteps to skip (integer) when displaying animation (Plots faster)

3.4 Linear hydrodynamics examples

Note

This is part of the M4E v2.0.

The linearization workflow allows a nonlinear multibody system to be coupled with linear frequency-domain hydrodynamics (radiation, diffraction, hydrostatics) computed with [Capytaine](#) through the [HydroLinearMCKF](#) adapter, plus any number of additional linear force contributions (mooring, viscous damping, PTOs, ...) through the same adapter interface. The three examples below build up this workflow from a minimal single-body system to a multibody system with several custom adapters and a frequency-domain impedance calculation. An example using Moorpy is also available in the [Github repository](#). Every linearization problem follows the following steps:

1. Define the M4E inputs (same as M4E v1.0)
2. Define the hydrodynamic inputs (new in M4E v2.0)
3. Import M4E and hydrodynamic inputs in main script (same as M4E v1.0)
4. Instantiate the multibody system (same as M4E v1.0)
5. Instantiate the linearization manager (new in M4E v2.0)
6. Register the linearization adapter(s) (similar to external forces manager in v1.0)
7. Solve the problem of interest (new in M4E v2.0):
 - Solve linear ODE system
 - Solve in frequency domain, FD

3.4.1 Example 1: Minimal linearized hydrodynamics (WaveBot)

This example is the smallest possible linearized hydrodynamics setup: a single body attached to ground through one prismatic joint (heave only), radiating and diffracting waves computed from a simple cylinder + cone mesh.

`Examples_linearization/wavebot/M4E_inputs.py` defines the multibody model:

```

joints          = [[0,1]]
types           = ['P']
parent_cg_to_joint = [[0,0]]
joint_to_child_cg = [[np.nan, np.nan]]
prismatic_direction = [[0,1]]
prismatic_direction = normalize_prismatic(prismatic_direction)

```

Examples_linearization/wavebot/hydro_inputs.py defines the wave conditions and the mesh, using `wecopttool` or using `numpy` to generate the frequency vector and the regular wave, and `pygmsh/gmsh` to build the cylinder-cone hull:

```

wavefreq      = 0.3 # Hz
nfreq         = 2
freq          = wot.frequency(wavefreq, nfreq, False) # False -> no zero
              frequency

amplitude     = 0.0625 # m
waves         = wot.waves.regular_wave(wavefreq, nfreq, wavefreq, amplitude,
phase=0, wavedir=0)

```

The mesh is packed into the `body_inputs` dictionary that `HydroLinearMCKF` expects, keyed by body index (matching the multibody body numbering), together with the mass/inertia used for the linearized rigid-body matrices:

```

body_inputs = {
    1: {
        "name": "wavebot",
        "mesh": mesh,
        "mesh_reference": "absolute",
        "inertia_diag": [m_wb, m_wb, m_wb, J_wb, J_wb, J_wb],
    },
}

```

To run this example, swap the “Example to import” lines at the top of the root `main_linearization.py` to point at the new folder:

```
##### Example to import #####
```

(continues on next page)

(continued from previous page)

```

from Examples_linearization.wavebot import M4E_inputs as ex
from Examples_linearization.wavebot.hydro_inputs import waves, body_
    inputs, freq, amplitude, m0, J0

```

The rest of the main file follows the same three steps used by every linearization example:

1. Build the multibody system and the equilibrium position:

```

MBDsys = MbdSystem.from_example(ex)

q0 = (MBDsys.ic - ex.ic)[:len(MBDsys.Q)] # Numerical values can be passed
mainNumVars = ex.ic.copy()

```

2. Create the LinearizationManager and register the hydrodynamics adapter:

```

LinManager = LinearizationManager(MBDsys, q0, mainNumVars, m0, J0, print_
    sym_matrices=False)

# Calculatiosn of added mass, radiation damping, hydrostatic stiffness and
excitation force terms in cartesian coordinates
hydroAdapter = HydroLinearMCKF(
    MBDsys,
    (mainNumVars, m0, J0),
    is_2D=False,
    omega_r=waves.omega.values,
    body_inputs=body_inputs,
    wave_amplitude=waves.attrs['Amplitude (m)'],
    equilibrium_pos=q0,
)

# Transformation of hydrodynamic terms from cartesian to joint coordinates

LinManager.register(hydroAdapter)

```

3. Compile an operating point (a driving wave frequency and, optionally, an amplitude ramp), then integrate the linearized equations of motion in time and plot the response:

```

omega0 = waves.omega.values[0]
LinManager.compile_operating_point(omega0, eq_tol=1e-6, eq_mode="warn",
ramp_T=30.0)

result = LinManager.integrate_linear_system(
    tspan=ex.tspan,
    dt=ex.TimeStep,
    method="RK45",
    solver_opts={'rtol': 1e-6, 'atol': 1e-9},
)

com_positions, com_velocities, angle_positions, _ = mbd.evaluate_
→trajectories(MBDsys, result, mainNumVars.copy())

```

`com_positions`, `com_velocities` and `angle_positions` are then plotted the same way as in the nonlinear examples above.

3.4.2 Example 2: Detailed multibody inputs and custom force adapters (FOSWEC)

This example uses the same 3-body FOSWEC platform-and-flaps kinematic structure introduced in “Detailed examples > Example 1: FOSWEC” above, this time coupled to linear hydrodynamics and two additional custom linear force adapters (mooring stiffness/damping and extra viscous damping). It is a standalone runnable file: `Examples_linearization/foswec/FOSWEC_main_linear.py`.

Note

The physical parameters used in `Examples_linearization/foswec/` (`M4E_inputs.py` and `hydro_inputs.py`) are not the same as the WEC-Sim-validated parameter set used by the regression tests in `tests/cases/linearization/foswec/` (see `test_foswec_rao.py` / `test_foswec_td.py`). If you need parameters that have been verified against WEC-Sim, use the copy under `tests/cases/linearization/foswec/` instead.

`M4E_inputs.py` defines the same joint topology as the nonlinear FOSWEC example (a floating platform with two revolute-jointed flaps), with its own CG heights and hinge offsets:

```

joints          = [[0, 1],[1, 2],[1, 3]]
types           = ['F', 'R', 'R']
parent_cg_to_joint = [[0, platformCG[2]],
                      [flap1CG[0],-platformCG[2]-flapHingeDepth],
                      [flap2CG[0],-platformCG[2]-flapHingeDepth]]
joint_to_child_cg = [[np.nan, np.nan],
                      [0,flapHingeDepth+flap1CG[2]],
                      [0,flapHingeDepth+flap2CG[2]]]
prismatic_direction = [[np.nan, np.nan],[np.nan, np.nan],[np.nan, np.nan]]
prismatic_direction = normalize_prismatic(prismatic_direction)

```

`hydro_inputs.py` builds a more detailed mesh than the WaveBot example (platform frame with cutout, four support columns, a DAQ box, and two flaps), and clips each body's mesh at the waterline with Capytaine before it is handed to `HydroLinearMCKF`:

```

fb_platform = cpt.FloatingBody(mesh=mesh_platform, name="platform")
fb_platform = fb_platform.keep_immersed_part()
mesh_platform_clipped = fb_platform.mesh

```

Doing this clipping in the input file (rather than inside `HydroLinearMCKF`) keeps the adapter mesh-agnostic and makes it explicit which part of each mesh is actually submerged and used for the BEM solve.

Custom adapters. Any additional linear force contribution — mooring, viscous damping, a PTO, etc. — can be registered alongside the hydrodynamics adapter as long as it satisfies the `LinearizationAdapter` protocol: a `.name` attribute, and a `.frequency_domain_MCKF(omega) -> (M, C, K, F)` method returning the mass, damping and stiffness matrix contributions plus a forcing dict with 'dc' and 'phasor' entries, all in the reduced (joint) coordinate ordering. `FOSWEC_main_linear.py` defines two such adapters:

```

class CustomMooring():
    name = "Mooring"

    def __init__(self):
        pass

    def frequency_domain_MCKF(self, omega):

```

(continues on next page)

(continued from previous page)

```

        M = np.zeros((9,9))
        K = np.diag([8e3, 2e5, 2e5, 0, 0, 0, 0, 0, 0])
        C = np.diag([8e2, 1e4, 1e4, 0, 0, 0, 0, 0, 0])
        F = {'dc': [0, 0, 0, 0, 0, 0, 0, 0, 0], 'phasor': np.
→zeros(9)} # Pre-tension

        return M, C, K, F

class CustomViscousDamping():
    name = "ViscousDamping"

    def __init__(self):
        pass

    def frequency_domain_MCKF(self, omega):
        M = np.zeros((9,9))
        K = np.diag([0, 0, 0, 0, 0, 0, 0, 0, 0])
        C = np.diag([0, 0, 0, 0, 0, 10, 0, 0, 10])
        F = {'dc': [0, 0, 0, 0, 0, 0, 0, 0, 0], 'phasor': np.
→zeros(9)} # Pre-tension

        return M, C, K, F

```

Both adapters are registered on the `LinearizationManager` in addition to the hydrodynamics adapter:

```

hydroAdapter = HydroLinearMCKF(
    MBDsys,
    (mainNumVars, m0, J0),
    is_2D=False,
    omega_r=waves.omega.values,
    body_inputs=body_inputs,
    wave_amplitude=waves.attrs['Amplitude (m)'],
    equilibrium_pos=q0,
)
mooringAdapter = CustomMooring()

```

(continues on next page)

(continued from previous page)

```
dampingAdapter = CustomViscousDamping()

LinManager.register(hydroAdapter)
LinManager.register(mooringAdapter)
LinManager.register(dampingAdapter)
```

From here the compile/integrate/plot steps are identical to Example 1.

3.4.3 Example 3: Frequency-domain analysis — computing the impedance matrix

The main file remains the same as Example 2 (`FOSWEC_main_linear.py`) — this example only adds a few lines after the `LinManager.register(...)` calls, before compiling an operating point. Instead of (or in addition to) integrating the linear system in time, the registered M , C , K , F contributions from every adapter can be assembled at any set of frequencies and solved directly in the frequency domain:

```
%% Define the impedance matrix
omega = waves.omega.values

# Instantiate the total linearized system with attributes M,C,K,F
FD_system = LinManager.assemble_frequency_domain(omega)

# Extract linearized matrices
M = FD_system.M
C = FD_system.C
K = FD_system.K
F = FD_system.Fhat

# Impedance matrix in joint coordinates
impedance_reduced = M * (1j * omega[:, None, None]) + C - 1j * K[None, :, :] * (1/omega[:, None, None])
```

`LinManager.assemble_frequency_domain(omega)` sums the M , C , K , F contributions from every registered adapter (hydrodynamics, mooring, damping, ...) at the given frequency ω . The matrix $Z = 1j\omega * M + C - 1j * K / \omega$ is the system's mechanical impedance (or dynamic stiffness) matrix in the reduced joint coordinates: it relates a complex

forcing phasor $\mathbf{F}_{\text{hat}}$ to the resulting complex response phasor $\mathbf{q}_{\text{hat}} = \mathbf{Z}^{-1} @ \mathbf{F}_{\text{hat}}$ at that frequency. Dividing the response amplitude by the wave amplitude gives the response-amplitude-operator (RAO) shown above.

This same impedance matrix is the starting point for computing quantities such as absorbed PTO power at each frequency (not shown here); see [main_linearization.py](#)'s frequency-domain block for a complete working reference.

3.5 Flexible bodies (FSM)

3.5.1 1 - What is FSM, and why it helps with BEM hydrodynamics

The Finite Segment Method (FSM) models a flexible member (a beam-like link that bends or stretches) as a chain of `n_seg` rigid segments connected by ordinary revolute ('R', bending) or prismatic ('P', axial) joints, each carrying a spring whose stiffness is derived from the member's own E , I/A , and length. As `n_seg` increases, this discrete chain converges to the real continuous beam (see the convergence plot in section 2) – but at any finite `n_seg` it is, numerically, just an ordinary M4E multibody system comprised of multiple rigid bodies: every segment is a plain rigid body with its own CG, mass, and inertia, connected by joints and springs you already know how to define (discussed in previous sections).

This is what makes FSM convenient for BEM-based hydrodynamics in particular. Computing genuinely distributed hydrodynamic loads over a continuously deforming body (projecting pressure onto mode shapes, or numerically integrating pressure over a deforming wetted surface) is slow and intricate with a BEM solver. FSM sidesteps this entirely: because the member is already a chain of rigid segments, BEM results (calculated for each segment) can generate the applied force and moments to the CG of the system. As a result, FSM can be used for quickly exploring various designs in low-to-mid fidelity. Additional external forces can be added using the external-force-adaptor machinery that existing multibody example in this documentation already uses (see the “Linear hydrodynamics examples” section above). A flexible body built with FSM is, from the hydrodynamics adapter's point of view, just several more rigid bodies to apply forces to – nothing new to build.

3.5.2 2 - Derivation: bending and axial segments

Each segment of length $dx = L/n_seg$ is treated as a short, straight, rigid link. Deformation is NOT distributed inside a segment. This is the classical finite segment method (FSM): a flexible member is replaced by a chain of rigid bodies joined by kinematic joints, following

Nikravesh, Chung & Benedict (1983) and Connelly & Huston (1994a,b), and Nikravesh (2018). The two deformation modes:

- Bending (`deformation_mode="bending"`): each joint is an ‘R’ joint, and its spring represents the beam’s curvature between the two neighboring segments.
- Axial (`deformation_mode="axial"`): each joint is a ‘P’ joint, and its spring represents the member’s own stretch/compression. An axial member needs one extra “phantom” body at the free tip (a massless anchor) – unlike bending’s free tip (which naturally carries zero moment), a free axial tip still needs a joint to terminate the chain.

References – Nikravesh, P. Planar Multibody Dynamics: Formulation, Programming with MATLAB(R), and Applications; CRC Press: Boca Raton, FL, USA, 2018. doi:10.1201/b22302 – Connelly, J.; Huston, R. “The dynamics of flexible multibody systems: A finite segment approach – I. Theoretical aspects.” Comput. Struct. 1994, 50, 255-258. doi:10.1016/0045-7949(94)90300-X – Connelly, J.D.; Huston, R.L. “The dynamics of flexible multibody systems: A finite segment approach – II. Example problems.” Comput. Struct. 1994, 50, 259-262. doi:10.1016/0045-7949(94)90301-8 – Nikravesh, P.; Chung, I.; Benedict, R. “Plastic hinge approach to vehicle crash simulation.” Comput. Struct. 1983, 16, 395-400. doi:10.1016/0045-7949(83)90178-5

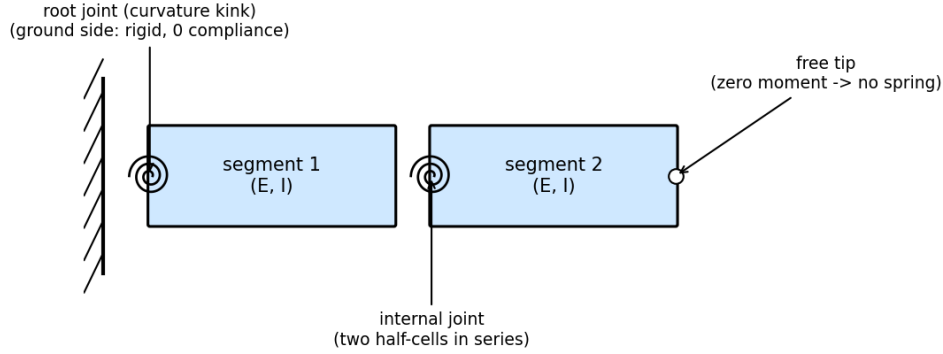
2.1 - Bending: from Euler-Bernoulli curvature to a half-cell spring

Start from the ordinary Euler-Bernoulli relation between curvature and bending moment, $\kappa = M/(EI)$. Treat one segment of length `e11` as carrying a constant moment `M` along its own length (reasonable once `e11=dx` is small). The relative end-to-end rotation it accumulates is then simply curvature times length:

$$\theta = \kappa \ell = \frac{M \ell}{EI} \implies k_{\text{segment}} = \frac{M}{\theta} = \frac{EI}{\ell}, \quad C_{\text{segment}} = \frac{1}{k_{\text{segment}}} = \frac{\ell}{EI}.$$

FSM does not place this whole-segment spring at one of the segment’s ends – it splits the segment’s own compliance in half and assigns one half-cell (length `dx/2`) to each of the two joints bounding it, giving a half-cell compliance of `dx/(2EI)` per side. A joint’s actual stiffness then depends on what is on its two sides (automatically assigned by the code):

Bending: half-cell compliance (root clamp vs. internal joint vs. free tip)



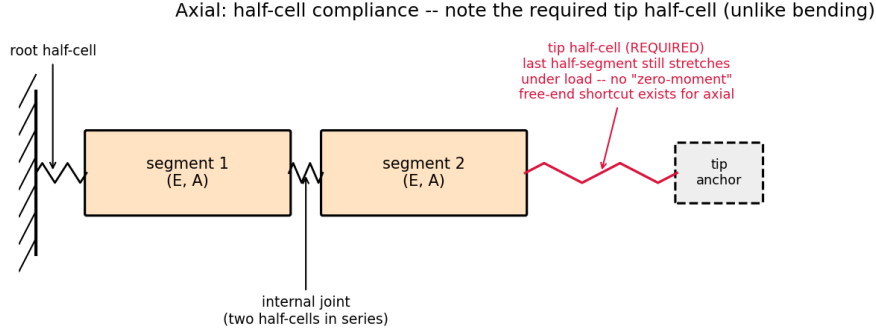
- Clamped root (`flex_clamp_on=True`): whatever is mounted on the OTHER side of this row's first joint – literal ground, an ordinary rigid body, or even another flexible member's own (already-expanded) terminal body – contributes zero EXTRA rotational compliance at that one joint, so only segment 1's own half-cell remains in the chain: $C_{\text{root}} = dx/(2EI)$, i.e. $k_{\text{root}} = 2EI/dx$. “Clamped” here describes this joint's local stiffness, not that the mounting point is fixed in space – a member clamped to a moving rigid hub still moves with that hub.
- Internal joint (between two flexible segments): the SAME bending moment passes through both half-cells one after the other, so their compliances add in series (rotations add for a shared moment, exactly like two springs in series): $C_{\text{joint}} = dx_1/(2E_1I_1) + dx_2/(2E_2I_2)$, i.e. $k = C_{\text{joint}}^{-1}$ – for uniform segments ($dx_1=dx_2=dx$, $E_1I_1=E_2I_2=EI$) this collapses to the familiar $k = EI/dx$.
- Free tip: Euler-Bernoulli's own free-end boundary condition is zero moment, $M=0$, at the tip – there is nothing left to split into a spring, so the chain simply ends with no joint stiffness there (zero stiffness, zero moment).
- Pinned root (mounted via a free hinge): also a zero-moment boundary condition at that joint, same as a free tip – no spring.

2.2 - Axial: the same half-cell idea, from Hooke's law

The axial case follows an identical derivation, with Hooke's law $\sigma = E \epsilon$ in place of the curvature relation. A uniform bar of length ℓ under axial force F stretches by $\delta = F \ell / (EA)$, so:

$$\delta = \frac{F \ell}{EA} \implies k_{\text{segment}} = \frac{F}{\delta} = \frac{EA}{\ell}, \quad C_{\text{segment}} = \frac{\ell}{EA}.$$

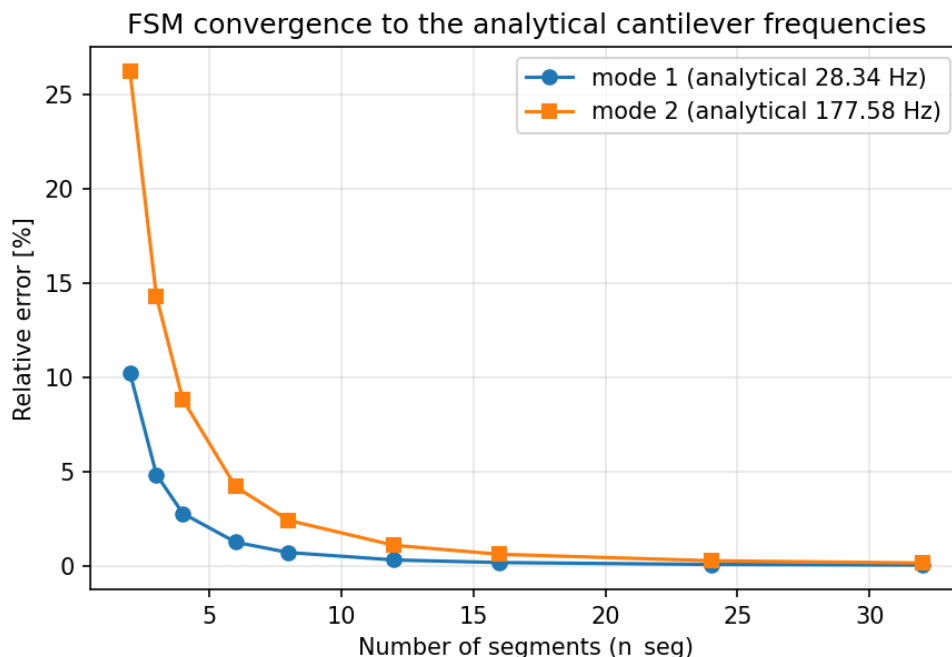
Splitting this into two half-cells of length $dx/2$, each with compliance $dx/(2EA)$, and combining them exactly as before (ground-side root = one half-cell only; internal joint = two half-cells in series) reproduces the bending case term-for-term with EA replacing EI : $k_{root} = 2EA/dx$ and $k_{joint} = (dx_1/(2E_1A_1) + dx_2/(2E_2A_2))^{-1}$ ($=EA/dx$ for a uniform member).



The one genuine difference from bending is at the free tip. Bending's free end has a natural zero-moment boundary condition that lets the chain simply stop. Axial deformation has no such shortcut: force equilibrium only requires the NET force carried past the tip to vanish, but the last half-segment (length $dx/2$) is still being stretched by whatever force IS transmitted through it, right up to the very end – there is no point along a stretched bar where the local elongation “naturally” goes to zero the way curvature does at a free end. FSM therefore terminates every axial chain with one more explicit half-cell spring, $k_{tip} = 2EA/dx$, connecting the last real segment to a massless “phantom” tip-anchor body. This is also why an axial member of n_{seg} segments reserves $n_{seg} + 1$ real bodies in Table 1, while a bending member of n_{seg} segments only needs n_{seg} .

2.3 - Convergence to the continuous beam

Because this is just the standard finite-element “half-springs in series” idea, refining the mesh (increasing n_{seg}) makes the discrete FSM chain converge to the real continuous Euler-Bernoulli cantilever beam. The figure below sweeps n_{seg} for a steel cantilever ($L=1$ m, $E=200$ GPa, $I=1e-8$ m⁴) and compares FSM's first two bending frequencies against the exact analytical clamped-free beam solution:



Both modes converge cleanly (roughly first-order in `n_seg`), confirming FSM is not just a qualitative approximation – with enough segments it reproduces the real continuous beam to whatever accuracy you need.

3.5.3 3 - How this is done in M4E: Table 1, Table 2, and the generator

Declaring a flexible member in M4E uses the SAME body/joint lists every rigid example in this documentation already uses (`joints`, `types`, `parent_cg_to_joint`, `joint_to_child_cg`, `prismatic_direction`, `m0`, `J0`) – this is Table 1 – plus one extra column, `flex_bd`:

```
# Table 1: one row per body, same convention as any rigid example, plus
flex_bd.
joints          = [[0, 1]]          # ground -> body 1
types           = ['R']             # mounting joint type (hinge
here)
parent_cg_to_joint = [[0.0, 0.0]]
joint_to_child_cg  = [[L / 2.0, 0.0]] # -> the WHOLE member's own CG
(its midpoint)
prismatic_direction = [[np.nan, np.nan]]
flex_bd          = [1]              # nan/None = rigid; flexible row
```

(continues on next page)

(continued from previous page)

```

-> its OWN body index
m0                = [None]                # ignored for a flexible row
(mass comes from Table 2)
J0                = [None]

```

`flex_bd` is `nan/None` for an ordinary rigid row, and for a flexible row it must equal that row's own child body index – in the example above, body 1 is flexible. This is checked: declaring the wrong value raises an error.

Table 2 then declares each flexible member's own physical properties, one row per flexible body (an example with multiple flexible bodies is also discussed in this documentation), with `flex_bds` listing all flexible bodies discussed in table 1:

```

# Table 2: one row per FLEXIBLE member, flex_bds says which body it is.
flex_bds          = [1]                  # Table 2 row 0 -> body 1
flex_n_seg        = [8]                  # number of rigid segments
flex_L            = [1.0]                # member length [m]
flex_E            = [200e9]              # Young's modulus [Pa] -- scalar OR
per-segment list
flex_A            = [1e-4]               # cross-sectional area [m^2]
flex_I            = [1e-8]               # second moment of area [m^4]
flex_rho          = [7800.0]             # material density [kg/m^3]
flex_deformation_mode = ["bending"]      # "bending" or "axial"
flex_clamp_on     = [True]               # True: clamped root: 2EI/dx; False:
pinned root: free hinge

```

`flex_E`, `flex_A`, `flex_I`, and `flex_rho` each accept either a single scalar (uniform member) or a list of `n_seg` values (one per segment) – a member with variable properties is declared simply by passing `flex_E/flux_I` as per-segment lists instead of scalars, e.g. `flex_E = [[20e9, 17e9, 15e9, 12e9, 10e9, 8e9]]` for a 6-segment member that is stiffest at the root and softest at the tip. Nothing else about the declaration changes.

Notice that Table 1's `m0/J0` are not filled in for a flexible row – you do NOT need to supply mass/inertia yourself. Each segment's mass and inertia are derived automatically from Table 2's own `A`, `rho`, and the segment length $dx = L/n_seg$, using the standard uniform-density lumped-segment formulas:

$$m_i = \rho_i A_i dx, \quad J_i = \frac{m_i dx^2}{12}$$

(the second formula is the usual slender-rod inertia about its own CG, $m L^2/12$, applied to one segment of length dx). This happens for every real segment of every flexible member – E/I only ever affect the joint stiffnesses (section 2), not the mass. If you need to override this (e.g. matching a published lumped-mass model exactly rather than deriving it), Table 2 accepts optional `flex_m/flex_J` (scalar or per-segment list, same broadcasting rule as `flex_E/flex_I`) that bypass the formula above for that member.

Once both tables are declared, `mb.generate_flex_model_file(...)` expands them into a complete, standalone example file – the same flat-list convention every hand-written example already uses – that you can import from `main.py` and run exactly like any other previously defined rigid example (integrate, plot, animate):

```
import multibody as mb

output_path = "Examples_flexible_FSM/_generated_flex_cantilever.py"
mb.generate_flex_model_file(
    joints, types, parent_cg_to_joint, joint_to_child_cg, prismatic_
    ↪direction,
    flex_bd, m0, J0,
    flex_bds, flex_n_seg, flex_L, flex_E, flex_A, flex_I, flex_rho,
    flex_deformation_mode, flex_clamp_on,
    output_path,
    ic=ic, gVec=np.zeros(n_bodies), tspan=2.0, TimeStep=0.005,
    overwrite=True,
)
```

The generated file is an ordinary, hand-editable `.py` example that includes all the segments and corresponding stiffness values: everything below a `USER ADDITIONS` marker is yours to extend (e.g. append your own hydrodynamic/mooring forces), and re-running the generator only ever overwrites the part above that marker. Point `main.py`’s “Example to import” at the generated file to run it, same as any other example in this documentation.

Starting from `FSM_table_generator.py` instead of a blank page. Rather than writing a new Table 1/Table 2 declaration from scratch, the repository root’s `FSM_table_generator.py` is already a working library of examples – labeled `EXAMPLE A` through `EXAMPLE N` – each a self-contained Table 1/Table 2 declaration followed by its own `mb.generate_flex_model_file(...)` call. Only a few blocks are left active (uncommented) at any one time; the rest are commented out on purpose, so they can be copied, uncommented, and adapted. Between them they already cover most of the topologies discussed in this

documentation and a few more besides: a single pinned or clamped member with a free tip (**EXAMPLE A/C/G**), a clamped member plus a rigid tip payload (**EXAMPLE D**), two flexible links joined through real rigid hub bodies plus a tip payload (**EXAMPLE E**), a translating rigid cart carrying a clamped beam, with or without a tip mass (**EXAMPLE F/I**), a mixed axial + bending chain (**EXAMPLE J** – the same topology as Example 2 below), a floating ('F', 3-DOF) body connected to an axial or a bending member (**EXAMPLE K/L**), a chained two-flexible-link double pendulum (**EXAMPLE B**), a tapered (per-segment E/I) cantilever with its own hand-computed stiffness check (**EXAMPLE M**), and a rigid hub sandwiched between a uniform and a tapered bending member (**EXAMPLE N** – this is Example 5 below, verbatim). **EXAMPLE H** and **EXAMPLE I** go a step further: they reproduce published validation cases (Subedi et al. 2021; Franco et al. 2018) exactly, with the resulting natural-frequency comparison recorded right in the script's own comments – **EXAMPLE H** is the same model used for Example 4 below.

To build a new example, copy whichever block is topologically closest to what you need, comment out whichever block is currently active (every block reuses the same top-level variable names – `joints`, `types`, `flex_bds`, etc. – so only one should be uncommented at a time), then edit the Table 1/Table 2 values and initial conditions. Running the script directly (`python FSM_table_generator.py`) regenerates the corresponding file(s) under `Examples_flexible_FSM/` and/or prints the tables, so a new declaration can be sanity-checked before it is wired into a full simulation via `main.py`.

3.5.4 4 - Examples

Example 1: Flexible OSWEC (single hinged flap)

A single bottom-hinged flap – the same topology as the rigid OSWEC example earlier in this documentation – but the flap itself is a flexible bending member (`N_SEG=8`) instead of one rigid body:

```
N_SEG, L_FLAP, EI = 8, 1.0, 40.0    # bending stiffness (lumped E*I,
illustrative)

# Table 1: one row per body
joints          = [[0, 1]]
types           = ['R']
parent_cg_to_joint = [[0.0, 0.0]]    # hinge sits at the ground
origin
joint_to_child_cg  = [[L_FLAP / 2.0, 0.0]] # -> the WHOLE flap's own CG
```

(continues on next page)

(continued from previous page)

```

(its midpoint)
prismatic_direction = [[np.nan, np.nan]]
flex_bd             = [1]
m0                  = [None]
J0                  = [None]

# Table 2: one row per FLEXIBLE member
flex_bds            = [1]
flex_n_seg          = [N_SEG]
flex_L              = [L_FLAP]
flex_E              = [EI]
flex_A              = [1.0]
flex_I              = [1.0]
flex_rho            = [1.0]
flex_deformation_mode = ["bending"]
flex_clamp_on       = [False] # pinned -- free rotation at the hinge

```

```

----- Table 1 (bodies/joints, pre-expansion) -----

```

```

+-----+-----+-----+-----+-----+-----+-----+-----+
↪--++-----+
| Body | Parent | Type | p2j          | j2c          | pris_dir    | m0  | J0
| flex_bd |
+=====+=====+=====+=====+=====+=====+=====+=====+
| 1    | 0      | R    | [0.00, 0.00] | [0.50, 0.00] | [nan, nan] | nan |
nan | 1      |
+-----+-----+-----+-----+-----+-----+-----+-----+
↪--++-----+

```

```

----- Table 2 (flexible-member properties) -----

```

```

+-----+-----+-----+-----+-----+-----+-----+-----+
↪---+
| flex_bd | n_seg | L    | E    | A    | I    | rho  | deform_mode | clamp_
↪on |
+=====+=====+=====+=====+=====+=====+=====+=====+
| 1      | 8     | 1.00 | 40.00 | 1.00 | 1.00 | 1.00 | bending      | False

```

(continues on next page)

(continued from previous page)

```
|
+-----+-----+-----+-----+-----+-----+-----+-----+
↪---+
```

`generate_flex_model_file(...)` expands Table 1's single row into a complete, standalone .py file – `Examples_flexible_FSM/_generated_flex_oswec.py` – the actual input `MbdSystem` builds from. Here is that generated file's core content (imports/checks/animation settings trimmed for brevity – everything else below is copied verbatim):

```
# Examples_flexible_FSM/_generated_flex_oswec.py (auto-generated, excerpt)

joints          = [[0, 1], [1, 2], [2, 3], [3, 4], [4, 5], [5, 6], [6,
7], [7, 8]]
types           = ['R', 'R', 'R', 'R', 'R', 'R', 'R', 'R']
parent_cg_to_joint = [[0.0, 0.0], [0.0625, 0.0], [0.0625, 0.0], [0.0625,
0.0],
                    [0.0625, 0.0], [0.0625, 0.0], [0.0625, 0.0], [0.
↪0625, 0.0]]
joint_to_child_cg = [[0.0625, 0.0], [0.0625, 0.0], [0.0625, 0.0], [0.
↪0625, 0.0],
                    [0.0625, 0.0], [0.0625, 0.0], [0.0625, 0.0], [0.
↪0625, 0.0]]
prismatic_direction = [[float('nan'), float('nan')]] * 8   # all 'R'
joints -- no prismatic axis

Force = {}
Force["TorsionSpring"] = [
    ([1, 2], [0.0, 320.0]),
    ([2, 3], [0.0, 320.0]),
    ([3, 4], [0.0, 320.0]),
    ([4, 5], [0.0, 320.0]),
    ([5, 6], [0.0, 320.0]),
    ([6, 7], [0.0, 320.0]),
    ([7, 8], [0.0, 320.0]),
]   # (connection, [theta_0, k]) -- no entry for body 0->1: pinned root,
zero stiffness
```

(continues on next page)

(continued from previous page)

```
m0 = [0.125, 0.125, 0.125, 0.125, 0.125, 0.125, 0.125, 0.125]
J0 = [0.00016276041666666666] * 8
```

main.py prints the same joints/types/... lists above as Table 3 (the expanded, what-MbdSystem-actually-uses table – the SAME call it makes for any rigid example, just with 8 rows instead of 1):

```
----- Table 3 (expanded, what MbdSystem actually uses) -----
+-----+-----+-----+-----+-----+-----+-----+-----+
↪-----+
| Body | Parent | Type | p2j          | j2c          | pris_dir    | m0   |
J0   |
+=====+=====+=====+=====+=====+=====+=====+=====+
| 1    | 0      | R    | [0.00, 0.00] | [0.06, 0.00] | [nan, nan] | 0.12 |
0.00 |
| 2    | 1      | R    | [0.06, 0.00] | [0.06, 0.00] | [nan, nan] | 0.12 |
0.00 |
| 3    | 2      | R    | [0.06, 0.00] | [0.06, 0.00] | [nan, nan] | 0.12 |
0.00 |
| 4    | 3      | R    | [0.06, 0.00] | [0.06, 0.00] | [nan, nan] | 0.12 |
0.00 |
| 5    | 4      | R    | [0.06, 0.00] | [0.06, 0.00] | [nan, nan] | 0.12 |
0.00 |
| 6    | 5      | R    | [0.06, 0.00] | [0.06, 0.00] | [nan, nan] | 0.12 |
0.00 |
| 7    | 6      | R    | [0.06, 0.00] | [0.06, 0.00] | [nan, nan] | 0.12 |
0.00 |
| 8    | 7      | R    | [0.06, 0.00] | [0.06, 0.00] | [nan, nan] | 0.12 |
0.00 |
+-----+-----+-----+-----+-----+-----+-----+-----+
↪-----+
```

Each segment is $dx = L_FLAP/N_SEG = 0.125$ m long, so $p2j = j2c = [dx/2, 0] = [0.0625, 0]$ (rounds to 0.06 above) – and $m0/J0$ are exactly the auto-derived values from section 3's formula: $m_i = \rho * A * dx = 1.0 * 1.0 * 0.125 = 0.125$ kg and $J_i = m_i * dx^2 / 12 = 1.6276e-4$ kg*m² (rounds to 0.00 at 2-decimal display).

Table 3 itself only covers bodies/joints – the actual spring stiffnesses are the `Force["TorsionSpring"]` list shown in the generated-file excerpt above. Only 7 springs for 8 bodies – there is NO entry for body 0 -> 1 because `flex_clamp_on=False` (pinned root): section 2.1’s free-tip/pinned-root rule applies at the root here too, so that joint carries zero stiffness, exactly as declared. Every internal spring’s stiffness matches the uniform member formula from section 2.1, $k = EI/dx = 40.0/0.125 = 320.0 \text{ N*m/rad}$ – confirming the auto-generated values directly against the hand formula.

Example 2: Mixed axial + bending chain, multiple bodies

A single chain combining both deformation modes and several rigid + flexible bodies: a rigid hub, an AXIAL flexible member, a BENDING flexible member, and a rigid tip payload (ground -> hub('R') -> axial member('P') -> bending member('R') -> payload('R')). Both members use `flex_clamp_on=True`: the axial member is clamped to the rigid hub, and the bending member is clamped to the axial member’s own (already-expanded) tip-anchor body – which, LOCALLY at that one new joint, is just another plain rigid body contributing zero extra compliance, even though the tip-anchor itself is free to move (since it’s being stretched by the axial spring upstream):

```
HUB_HALF, L_AX, L_BEND, N_AX, N_BEND = 0.05, 0.8, 0.6, 4, 4
EA, RHO_AX = 1.0e6, 2.0      # soft, illustrative axial stiffness
EI, RHO_BEND = 50.0, 1.0     # soft, illustrative bending stiffness (lumped
EI)

# Table 1: one row per body (hub, axial member, bending member, payload)
joints          = [[0, 1], [1, 2], [2, 3], [3, 4]]
types           = ['R', 'P', 'R', 'R']
parent_cg_to_joint = [[0.1, 0.0], [HUB_HALF, 0.0], [L_AX / 2.0, 0.0], [L_
→BEND / 2.0, 0.0]]
joint_to_child_cg  = [[HUB_HALF, 0.0], [L_AX / 2.0, 0.0], [L_BEND / 2.0,
0.0], [0.0, 0.0]]
prismatic_direction = [[np.nan, np.nan], [1.0, 0.0], [np.nan, np.nan], [np.
→nan, np.nan]]
flex_bd          = [np.nan, 2, 3, np.nan] # hub (rigid), axial member,
bending member, payload (rigid)
m0               = [0.05, None, None, 0.2]
J0               = [0.001, None, None, 0.001]
```

(continues on next page)

(continued from previous page)

Table 2: one row per FLEXIBLE member (axial, bending)

```

flex_bds          = [2, 3]
flex_n_seg        = [N_AX, N_BEND]
flex_L            = [L_AX, L_BEND]
flex_E            = [EA, EI]
flex_A            = [1.0, 1.0]
flex_I            = [1.0, 1.0]
flex_rho          = [RHO_AX, RHO_BEND]
flex_deformation_mode = ["axial", "bending"]
flex_clamp_on     = [True, True]

```

----- Table 1 (bodies/joints, pre-expansion) -----

```

+-----+-----+-----+-----+-----+-----+-----+
↪+-----+-----+
| Body | Parent | Type | p2j          | j2c          | pris_dir      | m0   |
J0    | flex_bd |
+=====+=====+=====+=====+=====+=====+=====+=====+=====+
| 1    | 0      | R    | [0.10, 0.00] | [0.05, 0.00] | [nan, nan]    | 0.05 |
0.00 | nan    |
| 2    | 1      | P    | [0.05, 0.00] | [0.40, 0.00] | [1.00, 0.00] | nan  |
nan  | 2.00   |
| 3    | 2      | R    | [0.40, 0.00] | [0.30, 0.00] | [nan, nan]    | nan  |
nan  | 3.00   |
| 4    | 3      | R    | [0.30, 0.00] | [0.00, 0.00] | [nan, nan]    | 0.20 |
0.00 | nan    |
+-----+-----+-----+-----+-----+-----+-----+
↪+-----+-----+

```

----- Table 2 (flexible-member properties) -----

```

+-----+-----+-----+-----+-----+-----+-----+
↪+-----+
| flex_bd | n_seg | L    | E          | A    | I    | rho  | deform_mode |
clamp_on |
+=====+=====+=====+=====+=====+=====+=====+=====+=====+

```

(continues on next page)

(continued from previous page)

2	4	0.80	1000000.00	1.00	1.00	2.00	axial	
True								
3	4	0.60	50.00	1.00	1.00	1.00	bending	
True								
+-----+-----+-----+-----+-----+-----+-----+-----+-----+								
↪-----+								

Note row 2's `n_seg=4` reserves 5 real bodies (4 segments + 1 phantom tip anchor) once expanded – exactly the axial free-tip half-cell requirement from section 2.2.

Example 3: Flexible FOSWEC (floating base + two hinged flaps)

A floating base ('F', 3 DOF) with two bottom-hinged flaps – the same topology as the rigid FOSWEC example earlier in this documentation – but both flaps are flexible bending members instead of rigid bodies:

```

N_SEG, L_FLAP, EI = 6, 1.0, 40.0  # bending stiffness (lumped E*I,
illustrative)
M_FLOAT = 0.5                      # floating base mass

# Table 1: one row per body (floating base, flap1, flap2)
joints      = [[0, 1], [1, 2], [1, 3]]
types       = ['F', 'R', 'R']
parent_cg_to_joint = [[0.0, 0.0], [-0.5, 0.0], [0.5, 0.0]]
joint_to_child_cg  = [[np.nan, np.nan], [L_FLAP / 2.0, 0.0], [L_FLAP / 2.
↪0, 0.0]]
prismatic_direction = [[np.nan, np.nan]] * 3
flex_bd            = [np.nan, 2, 3]
m0                 = [M_FLOAT, None, None]
J0                 = [0.05, None, None]

# Table 2: one row per FLEXIBLE member (flap1, flap2)
flex_bds          = [2, 3]
flex_n_seg        = [N_SEG, N_SEG]
flex_L            = [L_FLAP, L_FLAP]
flex_E            = [EI, EI]

```

(continues on next page)

(continued from previous page)

```

flex_A          = [1.0, 1.0]
flex_I          = [1.0, 1.0]
flex_rho        = [1.0, 1.0]
flex_deformation_mode = ["bending", "bending"]
flex_clamp_on   = [False, False] # pinned hinges

```

```
----- Table 1 (bodies/joints, pre-expansion) -----
```

```

+-----+-----+-----+-----+-----+-----+-----+-----+
↪-----+-----+
| Body | Parent | Type | p2j          | j2c          | pris_dir    | m0    |
J0    | flex_bd |
+=====+=====+=====+=====+=====+=====+=====+=====+
| 1    | 0      | F    | [0.00, 0.00] | [nan, nan]   | [nan, nan]  | 0.50 |
0.05 | nan    |
| 2    | 1      | R    | [-0.50, 0.00] | [0.50, 0.00] | [nan, nan]  | nan   |
nan  | 2.00   |
| 3    | 1      | R    | [0.50, 0.00]  | [0.50, 0.00] | [nan, nan]  | nan   |
nan  | 3.00   |
+-----+-----+-----+-----+-----+-----+-----+-----+
↪-----+-----+

```

```
----- Table 2 (flexible-member properties) -----
```

```

+-----+-----+-----+-----+-----+-----+-----+-----+
↪-----+
| flex_bd | n_seg | L    | E    | A    | I    | rho  | deform_mode | clamp_
↪on |
+=====+=====+=====+=====+=====+=====+=====+=====+
| 2      | 6     | 1.00 | 40.00 | 1.00 | 1.00 | 1.00 | bending     | False
|
| 3      | 6     | 1.00 | 40.00 | 1.00 | 1.00 | 1.00 | bending     | False
|
+-----+-----+-----+-----+-----+-----+-----+-----+
↪-----+

```

Example 4: Flexible member with a rigid child

A flexible member doesn't need to end in a free tip – a rigid body (a payload, sensor, or tool) can be attached at a flexible member's end, exactly like any other rigid-to-rigid joint. This example reproduces a published result: Subedi, Tyapin & Hovland (2021), “Dynamic Modeling of Planar Multi-Link Flexible Manipulators,” *Robotics* 10(2):70 – link 3 with a 2 kg tip payload (rigid payload attached via a very stiff torsional spring added after generation, since M4E has no literal “weld” joint). The linearized FSM bending frequencies match the paper's own published values closely:

Mode	FSM [Hz]	Published [Hz]	Error
1	4.88	4.90	0.32%
2	63.15	63.97	1.28%

This example builds link 3 as an ordinary FSM bending member (`N_SEG=10`, clamped root) with the 2 kg payload welded on via a very stiff torsional spring (`k = 1000 * k_root`), then gets its frequencies the same way any linearized M4E system does:

1. Build the `MbdSystem` from the generated file (identical to any other example).
2. Linearize it about the straight, undeformed equilibrium – `numeric_linearize()` returns the joint-space mass matrix `M0` and stiffness matrix `K0`.
3. Solve the generalized eigenvalue problem $K0 \mathbf{x} = \omega^2 M0 \mathbf{x}$ (`scipy.linalg.eigh(K0, M0)`), drop any near-zero eigenvalue, and convert the first two nonzero ones to frequencies: `f = sqrt(eigenvalue)/(2*pi)`.

The resulting FSM frequencies (4.88 Hz, 63.15 Hz) differ from the paper's own values by only 0.32%/1.28% – ordinary FSM discretization error (`N_SEG=10` segments standing in for the continuous beam), the same convergence trend demonstrated by section 2.3's sweep, not a modeling mismatch.

```
L3, EI3, RH03, JP, MP, N_SEG = 1.5, 2.4114e3, 0.7425, 3.2e-4, 2.0, 10
```

```
# Table 1: one row per body (link3, rigid tip payload)
```

```
joints          = [[0, 1], [1, 2]]
types           = ['R', 'R']
parent_cg_to_joint = [[0.0, 0.0], [L3 / 2.0, 0.0]]
joint_to_child_cg  = [[L3 / 2.0, 0.0], [0.0, 0.0]]
```

(continues on next page)

(continued from previous page)

```

prismatic_direction = [[np.nan, np.nan]] * 2
flex_bd             = [1, np.nan]           # row 2 is rigid (the payload)
m0                  = [None, MP]
J0                  = [None, JP]

# Table 2: one row per FLEXIBLE member (link3)
flex_bds            = [1]
flex_n_seg          = [N_SEG]
flex_L              = [L3]
flex_E              = [EI3]
flex_A              = [1.0]
flex_I              = [1.0]
flex_rho            = [RH03]
flex_deformation_mode = ["bending"]
flex_clamp_on       = [True]

```

The rigid payload itself is attached AFTER generation, directly in the generated file's own Force["TorsionSpring"] list – it can't be part of Table 1/Table 2 since it references real, post-expansion body numbers (10, 11) that only exist once the generator has run:

```

dx = L3 / N_SEG
k_root = 2.0 * EI3 * 1.0 / dx           # SAME half-cell root formula as
section 2.1
k_weld = 1000.0 * k_root                # 1000x stiffer -- approximates
a rigid weld

ex.Force["TorsionSpring"] = list(ex.Force["TorsionSpring"]) + [
    ([N_SEG, N_SEG + 1], [0.0, k_weld])    # body 10 (last beam segment) -
    →> body 11 (payload)
]

```

No dampers are used in this example (Force["TorsionDamper"]/Force["TensionDamper"] stay empty).

```

----- Table 1 (bodies/joints, pre-expansion) -----

```

```

+-----+-----+-----+-----+-----+-----+-----+-----+

```

(continues on next page)

(continued from previous page)

↪-----+							
Body	Parent	Type	p2j		j2c		pris_dir
J0	flex_bd						m0
+=====+							
1	0	R	[0.00, 0.00]		[0.75, 0.00]		[nan, nan]
nan	1.00						nan
2	1	R	[0.75, 0.00]		[0.00, 0.00]		[nan, nan]
0.00	nan						2.00
+-----+							
↪-----+							
----- Table 2 (flexible-member properties) -----							
+-----+							
↪-----+							
flex_bd	n_seg	L	E	A	I	rho	deform_mode
clamp_on							
+=====+							
1	10	1.50	2411.40	1.00	1.00	0.74	bending
							True
+-----+							
↪-----+							

Example 5: Two flexible bodies sharing one rigid hub (beam with variable properties)

Table 1/Table 2 are not limited to a single flexible member – this example declares TWO independent flexible bending members (`link1`, `link2`) joined by one ordinary rigid hub body in between (`link1 -> hub (rigid) -> link2`), with `link2`'s `E` and `I` both varying per-segment (tapered) while `link1` stays uniform, to show the two styles side by side. Not validated against any paper – it is only a structure demo:

```
L1, N1, E1, I1 = 0.6, 4, 70e9, 2e-6          # link1: uniform E, I
L2, N2 = 0.5, 5
E2_vals = [200e9, 170e9, 140e9, 110e9, 80e9] # link2: tapered E
(stiffer at root)
I2_vals = [5e-6, 4e-6, 3e-6, 2e-6, 1e-6]     # link2: tapered I
(larger cross-section at root)
```

(continues on next page)

(continued from previous page)

```

# Table 1: one row per body (link1, hub, link2)
joints      = [[0, 1], [1, 2], [2, 3]]
types       = ['R', 'R', 'R']
parent_cg_to_joint = [[0.0, 0.0], [L1 / 2.0, 0.0], [0.0, 0.0]]
joint_to_child_cg = [[L1 / 2.0, 0.0], [0.0, 0.0], [L2 / 2.0, 0.0]]
prismatic_direction = [[np.nan, np.nan]] * 3
flex_bd     = [1, np.nan, 3]          # hub (row 2) is rigid
m0          = [None, 0.15, None]
J0          = [None, 0.0008, None]

# Table 2: one row per FLEXIBLE member (link1, link2)
flex_bds    = [1, 3]
flex_n_seg  = [N1, N2]
flex_L      = [L1, L2]
flex_E      = [E1, E2_vals]          # link1: scalar, link2: per-
→segment list
flex_I      = [I1, I2_vals]          # same -- link2 is the
tapered one
flex_A      = [1.0, 1.0]
flex_rho    = [2700.0, 2700.0]
flex_deformation_mode = ["bending", "bending"]
flex_clamp_on = [True, True]

```

```

----- Table 1 (bodies/joints, pre-expansion) -----

```

```

+-----+-----+-----+-----+-----+-----+-----+-----+
→-----+-----+
| Body | Parent | Type | p2j          | j2c          | pris_dir    | m0    |
J0    | flex_bd |
+=====+=====+=====+=====+=====+=====+=====+=====+=====+
| 1    | 0      | R    | [0.00, 0.00] | [0.30, 0.00] | [nan, nan] | nan   |
nan   | 1.00   |
| 2    | 1      | R    | [0.30, 0.00] | [0.00, 0.00] | [nan, nan] | 0.15  |
0.00  | nan    |
| 3    | 2      | R    | [0.00, 0.00] | [0.25, 0.00] | [nan, nan] | nan   |

```

(continues on next page)

(continued from previous page)

```

nan | 3.00 |
+-----+-----+-----+-----+-----+-----+-----+-----+
↪-----+-----+
----- Table 2 (flexible-member properties) -----
+-----+-----+-----+-----+-----+-----+-----+-----+
↪-----+-----+-----+-----+-----+-----+-----+-----+
↪-----+-----+-----+-----+-----+-----+-----+-----+
| flex_bd | n_seg | L      | E
| A      | I
deform_mode | clamp_on |
+=====+=====+=====+=====+=====+=====+=====+=====+=====+
| 1      | 4      | 0.60 | 700000000000.00
| 1.00 | 2.00e-06
bending      | True      |
| 3      | 5      | 0.50 | [200000000000.00, 170000000000.00, 140000000000.
↪00, 110000000000.00, 80000000000.00] | 1.00 | [5.00e-06, 4.00e-06, 3.00e-
↪06, 2.00e-06, 1.00e-06] | 2700.00 | bending      | True      |
+-----+-----+-----+-----+-----+-----+-----+-----+
↪-----+-----+-----+-----+-----+-----+-----+-----+
↪-----+-----+-----+-----+-----+-----+-----+-----+

```

Row 1 (`link1`) is uniform (E/I are plain scalars); row 3 (`link2`) is tapered (E/I are per-segment lists), and row 2's hub body does not appear in Table 2 at all since it is rigid.

3.5.5 5 - Adding forces or external modules (hydrodynamics, linearization, mooring)

A flexible-body example generated via `generate_flex_model_file` is, once built, an ordinary M4E example module with real body numbers, `Force`, and `Initial_Points`. User can define custom forces and points on the bodies as discussed before. Additionally, every external-coupling mechanism already covered elsewhere in this documentation applies directly, with no FSM-specific extra step:

- BEM/linear hydrodynamics: couple radiation/excitation forces at each segment's own CG, the same way the “Linear hydrodynamics examples” section does for ordinary rigid bodies – see in particular the Detailed multibody inputs and custom force adapters

example for the exact `ExternalForcesManager`/adapter pattern.

- Linearization: `linearize_mbd/LinearizationManager` operate on the expanded (Table 3) body/joint lists exactly like any rigid system – see the Frequency-domain analysis example above for computing RAOs/impedance.
- Mooring (MoorDyn): attach mooring lines to any real body number (including a flexible member's own segments) the same way the MoorDyn examples in the developer documentation do.

In every case, the only FSM-specific detail to remember is that a flexible member's real body numbers are only known AFTER `generate_flex_model_file` expands Table 1/Table 2 – open the generated file (or its printed Table 3, see `multibody.tables.bodies_table`) to find the real body number you want to attach a force/mooring line to, then add it past the `USER ADDITIONS` marker, exactly like the tip-payload pattern used in Example 4 above.

DEVELOPER'S DOCUMENTATION

The developer's documentation provides guidance for anyone who wants to extend or maintain the Multibody4Everybody codebase. It describes the internal architecture, public API layout, and validation routines that ensure numerical and symbolic correctness. Developers can use this section to understand how the core modules interact, where to implement new functionality, and how to run the built-in validation and energy comparison tests to verify changes.

For a detailed overview of the v1.0 capabilities please refer to [M4E publication](#).

4.1 Project Layout

The following shows the structure of the `multibody` package:

```
multibody/  
  checks/  
    body_check.py  
    points_forces_check.py  
  ext_forces_manager/  
    coupling_main.py  
    moordyn_adapter.py  
    hydro_adapter.py  
    external_class_template.py  
    moordyn_writer.py  
  multibody_core/  
    _joints_helpers.py  
    joints_system.py
```

(continues on next page)

(continued from previous page)

```

kinematics_transformation.py
mbd_system.py
points_forces_definition.py
rigid_body_integrator.py
_springs_dampers_helpers.py
symvars_py.py
post_process.py
systems_energy.py
plotting/
    plotting_main.py
    plotting_helper.py
linearization/
    linearize_eom.py
    linearization_main.py
    linear_integrator.py
    hydro_linear_mckf.py
    _moorpy_schema.py          # MoorPy input schema and validation (no
MoorPy import)
    _moorpy_system_builder.py  # Constructs MoorPy System at q0
    moorpy_linear_mckf.py      # MoorPy LinearizationAdapter (optional
dep.)
tables/
    points_forces_table.py
yaml_parser/
    yaml_adapter.py
    simulation_parser.py
validation/
    automated_validation.py
    energy_and_ode_comparison.py

```

4.2 Public API summary

The following symbols are exposed by default when importing the multibody package:

```

from multibody import (
    # High-level interface
    MbdSystem,          # MultiBody dynamics class
                        # with all symbolic information
    integrate_dynamics, # Integration in time
    ExternalForcesManager, # Enables coupling external libraries
                        # providing forces to the MBD code

    # Core classes & routines
    JointSystem,        # Creates a class containing all bodies
                        # information, connectivity and variables
    normalize_prismatic, # Ensures the vectors of prismatic joints
                        # are unit length
    VelocityTransformation, # Computes Velocity transformation matrix and
its
                        # derivative, CG positions and joint positions
    points_force_finder, # Assembles the generalized force vector
                        # and points-forces mapping
    systems_energy,      # Computes system energy, assembles mass matrix
    symvars_definition,  # Creates the symbolic variables defined in the
example
    evaluate_trajectories, # Post-processes integration results

    # Linearization & YAML
    linearize_mbd,        # Linearizes EOM about equilibrium
    load_yaml_as_example, # Loads *.model.yaml + *.simulation.yaml

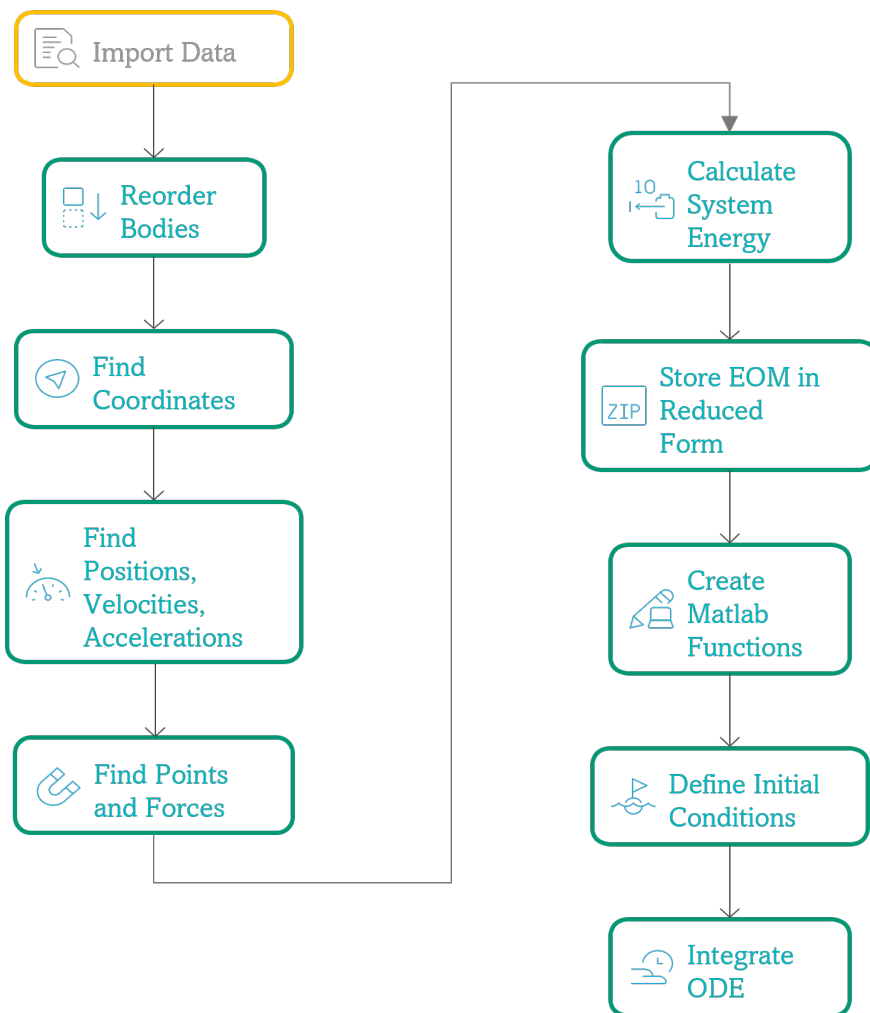
    # Submodules (namespaces)
    plot,                 # Quick access to plotting tools
    checks,               # Quick access to checking tools
    tables,               # Quick access to table-display tools
)

```

4.3 Multibody system creation workflow

The following workflow outlines the main steps followed by the multibody code. This is not necessary

Multibody Dynamics Simulation Workflow



4.4 Tests

The test suite lives entirely under `tests/` and is driven by `pytest`. All configuration is in `pyproject.toml` (`testpaths = ["tests"]`). Truth data (reference outputs) are committed alongside the source so no external tools or network access are required.

4.4.1 Test layout

```

tests/
  metrics.py                # Shared NRMSE helpers
  cases/
    catalog.py              # Shared MATLAB case registry (cases 1-8)
    matlab/                 # Symbolic kinematics + energy tests
      test_automated_validation.py
      test_energy_and_ode.py
      truth/
        symbolic/           # Example{1-8}.json (MATLAB reference)
        energy/             # Em_example{1-8}.mat
    chrono/                 # Chrono regression tests (9 cases)
      test_chrono_cases.py
      catalog.py
      runner.py
      truth/<case_id>/       # t_chrono.npy, r_chrono.npy, v_chrono.
→npy
  yaml/                     # YAML-parser regression tests (7 cases)
    test_yaml_cases.py
    catalog.py
    runner.py
    models/                 # *.model.yaml + *.simulation.yaml
    truth/<case_id>/         # trajectory.csv (PyChrono reference)
  linearization/            # Linearization tests
    test_lin_vs_nonlinear.py # 3 cases: lin  NL for small IC
    test_foswec_rao.py       # 6 cases: RAO vs WEC-Sim (slow)
    test_foswec_td.py        # 1 case:  time-domain vs WEC-Sim
(slow)
    test_moorpy_schema.py    # 33 cases: schema validation, no
MoorPy needed
    test_manager_adapter.py  # 24 cases: manager protocol + MoorPy
smoke
    test_moorpy_full.py      # 39 cases: point mapping, DOF
reduction, coupling
    catalog.py
    runner.py

```

(continues on next page)

(continued from previous page)

```

foswec_runner.py
foswec/           # FOSWEC example inputs + BEM data
truth/           # RAO_matlab.txt, results_wecSim.mat

```

4.4.2 Suite descriptions

tests/cases/matlab/

Validates symbolic kinematics and energy conservation for eight canonical examples against MATLAB-generated reference data.

- `test_automated_validation` — checks CG positions, joint locations, and the $R / \dot{R}$ matrices for Examples 1–8 against `.json` files in `truth/symbolic/`. A non-zero error means a kinematic formula has changed.
- `test_energy_and_ode` — integrates each system and compares the total mechanical energy trace against `.mat` files in `truth/energy/`. Typical RMSE values are zero for conservative cases and below 10^{-5} for the flexible/slider cases. The tolerance is 10^{-4} .

tests/cases/chrono/

Runs nine M4E multibody models and compares position and velocity trajectories against PyChrono simulation results stored as `.npz` files. Tolerances: NRMSE 1 % (position) and 1.5 % (velocity).

tests/cases/yaml/

Verifies the YAML parser by loading seven models from `.model.yaml` + `.simulation.yaml` pairs, integrating them, and comparing against PyChrono reference trajectories stored as `trajectory.csv` files. Tolerances: NRMSE 1×10^{-3} (position) and 1×10^{-3} (velocity).

tests/cases/linearization/

Three test files covering the linearization workflow:

- `test_lin_vs_nonlinear` — integrates each model twice (full nonlinear and linearized about the rest configuration) and asserts that position NRMSE 5×10^{-3} and velocity NRMSE 10^{-2} .
- `test_foswec_rao` (slow) — computes the time-domain RAO at five wave frequencies via the `LinearizationManager` and compares against a WEC-Sim MATLAB

reference stored in `truth/RAO_matlab.txt`. Platform DOF tolerances are 3 %; flap DOFs 20 %.

- `test_foswec_td` (slow) — integrates the full FOSWEC system in the time domain and checks position NRMSE against the WEC-Sim reference trajectory in `truth/results_wecSim.mat`.
- `test_moorpy_schema` — 33 tests covering schema dataclasses, ID validation, and reference checking. Does not require MoorPy.
- `test_manager_adapter` — 24 tests covering `LinearizationManager` matrix transforms (`Mq`, `Cq`, `Kq`), `detect_space`, DC-force handling, and a MoorPy smoke test verifying adapter instantiation and output shapes.
- `test_moorpy_full` — 39 tests covering point mapping, 6→3 DOF reduction, absence of double H-transform, cross-body coupling, mean force ordering, and end-to-end real-MoorPy smoke tests.

4.4.3 Running the tests

From the repository root:

```
# Fast suite only (excludes slow hydrodynamic tests, ~40 s)
pytest -m "not slow"

# Full suite including slow tests
pytest

# A single sub-suite
pytest tests/cases/matlab/
pytest tests/cases/chrono/
pytest tests/cases/yaml/
pytest tests/cases/linearization/

# Verbose output
pytest -v
```

The `slow` marker is applied to any test that takes more than a few seconds (currently the FOSWEC RAO and time-domain tests). Deselect them during routine development with `-m "not slow"`.

4.5 API

4.5.1 MultibodyCore package

This package contains the core multibody-dynamics engine: system setup, kinematics and symbolic equation generation. It also creates lambdified expressions for performance. The core element of this package is the `MBDsystem` class. The class is initialized:

- `from_example` [method]: reads an example where all the joints and connectivity is defined

<i><code>multibody.MbdSystem</code></i>	High-level multibody system combining symbolic model generation and numerical simulation.
<i><code>multibody.JointSystem</code></i>	Ordered collection of <code>Joint</code> objects plus convenience utilities for symbolic multibody modelling.
<i><code>multibody.normalize_prismatic</code></i>	Normalize a list of 2D prismatic-axis vectors into unit-length Sympy Matrices.
<i><code>multibody.VelocityTransformation</code></i>	Compute kinematic position and velocity transformations for a planar multibody system.
<i><code>multibody.points_force_finder</code></i>	Reads the points and forces defined in the dictionaries <code>Initial_Points</code> and <code>Forces</code> and parses them into the <code>MBDsystem</code> class.
<i><code>multibody.systems_energy</code></i>	Computes the total mechanical energy of the multibody system.
<i><code>multibody.symvars_definition</code></i>	Converts each string in the list to <code>sympy.Symbol</code> and saves it in the global workspace

`MbdSystem`

```
class MbdSystem(joints, types, parent_cg_to_joint, joint_to_child_cg,
                  prismatic_direction, Initial_Points=<factory>, Force=<factory>,
                  Reference_frame_Origin=<factory>, ForcesPointsSym=<factory>,
                  BodyDataSym=<factory>, g=9.81, gVec=None, ic=None)
```

Bases: `object`

High-level multibody system combining symbolic model generation and numerical simulation.

Constructs the full symbolic model (kinematics, constraints, forces, energies), compiles these to numeric functions, and provides an integration routine.

Parameters

- **joints** (*list[tuple[int, int]]*) – Parent-child connectivity for each joint.
- **types** (*list[str]*) – Joint types ('R', 'P', 'F', etc.).
- **parent_cg_to_joint** (*list*) – Vectors from parent body CG to joint locations.
- **joint_to_child_cg** (*list*) – Vectors from joint to child body CGs.
- **prismatic_direction** (*list*) – Unit direction vectors for prismatic joints.
- **Initial_Points** (*dict*) – Definitions of grounded and body-specific points.
- **Force** (*dict*) – Force definitions (CG forces, point forces, springs, dampers).
- **Reference_frame_Origin** (*tuple/list*) – Global origin coordinates.
- **ForcesPointsSym** (*list[sym.Symbol]*) – Symbolic variables for force-point magnitudes.
- **BodyDataSym** (*list[sym.Symbol]*) – Symbolic variables for body data (mass, inertia).
- **ic** (*np.ndarray*) – Initial conditions array for generalized coordinates and speeds.
- **g** (*float/sym.Symbol*) – Gravitational acceleration constant (default 9.81).
- **gVec** (*np.ndarray*) – Per-body gravity scale factors (default ones).

joints: *list[tuple[int, int]]*

```
types: list[str]

parent_cg_to_joint: list

joint_to_child_cg: list

prismatic_direction: list

Initial_Points: dict

Force: dict

Reference_frame-Origin: list | tuple | ndarray

ForcesPointsSym: list[Symbol]

BodyDataSym: list[Symbol]

g: float | Symbol = 9.81

gVec: ndarray = None

ic: ndarray | None = None

joint_system: JointSystem

Q: list

QD: list

QDD: list

NDOF: int

Body_col: list

Pos = None

JointLoc = None

R = None

RD = None

Vel = None
```

```
Acc = None

paths = None

graph = None

Points_All = None

Force_All = None

Energy = None

M = None

m = None

J = None

Pos_func = None

JointLoc_func = None

R_func = None

RD_func = None

Vel_func = None

Acc_func = None

Right_side_func = None

ReducedM_func = None

Energy_func = None

BDpoints_func: dict

CGpoints_func = None

GRpoints_func = None

BDvel_func: dict

CGvel_func = None
```

t_update: list[int]

classmethod from_example(ex)

Create an MbdSystem from a user-provided example module.

The example module must define: joints, types, parent_cg_to_joint, joint_to_child_cg, prismatic_direction, Initial_Points, Force, Reference_frame_Origin, ForcesPointsSym, BodyDataSym, ic, g (optional), and gVec (optional).

Return type

MbdSystem

Parameters

ex (*ModuleType*)

integrate(mainNumVars, m_vals, J_vals, *, tspan, dt, rtol=1e-08, atol=1e-08, algorithm='RK45', external_manager=None)

Numerically integrate the equations of motion $\mathbf{Q} = \mathbf{M}^{-1} \cdot \text{RHS}$.

Parameters

- **mainNumVars** (*Sequence[float]*) – Flattened initial values for [Q, QD, force-point parameters, body data].
- **m_vals** (*Sequence[float]*) – Mass values per body.
- **J_vals** (*Sequence[float]*) – Inertia values per body.
- **tspan** (*float/tuple*) – End time or (start, end) for integration.
- **dt** (*float*) – Time step for output.
- **rtol** (*float*) – Solver tolerances.
- **atol** (*float*) – Solver tolerances.
- **solver** (*str*) – SciPy ODE solver name (e.g., “RK45”).
- **external_manager** – Optional force manager for external coupling.

Returns

Result object with time points t and solution y.

Return type

OdeResult

JointSystem

class JointSystem

Bases: `object`

Ordered collection of `Joint` objects plus convenience utilities for symbolic multibody modelling.

A `JointSystem` stores every joint in the mechanism, keeps them sorted by their child-body index, and exposes helper methods that bridge the gap between the tabular data a user typically writes and the richer objects required by the rest of the multibody package.

Notes

Ground is body 0. All other bodies are numbered consecutively starting at 1; joints are sorted so that the k -th entry in *joints* always corresponds to body k .

All vectors are 2-D column vectors expressed in the global X-Z plane (right-handed with Y pointing out of the screen).

joints

List of `Joint` instances, sorted by child index.

Type

`list[Joint]`

The following are generated by `:meth:`coordinate_finder``

Q, QD, QDD

System-wide symbolic coordinates, velocities, and accelerations.

Type

`list[sympy.Symbol]`

NDOF

Degrees of freedom contributed by each joint (1 for 'R'/'P', 3 for 'F').

Type

`list[int]`

Body_col

Map from joint $\rightarrow$ global DOF index(es). Useful when assembling matrices.

Type

list[list[int]]

Examples

```
>>> import sympy as sym
>>> import numpy as np
>>> from multibody import JointSystem, normalize_prismatic
>>>
>>> joints = [[0, 1], [1, 2]]
>>> types = ['F', 'R']
>>> pcg2j = [[0.0, 0.0], [1.5, 0.0]]
>>> j2ccg = [[np.nan, np.nan], [-1.0, 0.0]]
>>> pris_d = normalize_prismatic([[np.nan, np.nan],
...                               [np.nan, np.nan]])
>>>
>>> sys = JointSystem.from_data(joints, types, pcg2j, j2ccg, pris_d)
>>> sys.display_table()
>>> Q, QD, QDD, NDOF, Body_col = sys.coordinate_finder()
```

add_joint(parent, child, joint_type, parent_cg_to_joint, joint_to_child_cg, prismatic_direction=None)

Create a Joint instance and add it to the system.

classmethod from_data(joints, types, parent_cg_to_joint, joint_to_child_cg, prismatic_direction)

Build a JointSystem from arrays/lists corresponding to MATLAB inputs.

display_table()

Displays the joint system in a table format.

coordinate_finder()

Associates each joint with its corresponding symbolic coordinate variables for positions (Q), velocities (QD), and accelerations (QDD), based on the joint type. The joints are sorted by their child body number so that the DOF labeling index (used in the symbolic variable names) matches the child-body number.

Joint types:

- 'R' (Revolute): 1 DOF (rotational).

- ‘P’ (Prismatic): 1 DOF (translational).
- ‘F’ (Floating in 2D): 3 DOFs (X, Z, Theta).

Returns

List of symbolic variables for joint positions. QD (list): List of symbolic variables for joint velocities. QDD (list): List of symbolic variables for joint accelerations. DOF_list (list): List containing the DOF count for each joint. Body_col (list of lists): Each sublist contains the global DOF indices for the corresponding joint.

Return type

Q (list)

`multibody.normalize__prismatic`

`normalize__prismatic(prismatic_direction)`

Normalize a list of 2D prismatic-axis vectors into unit-length Sympy Matrices.

For each vector in `prismatic_direction`, this function:

1. Converts it to a `sympy.Matrix`.
2. Checks that it contains no nan entries and that its Euclidean norm is non-zero.
3. If valid, divides the vector by its norm to produce a unit-length axis.
4. Otherwise, returns `Matrix([nan, nan])` for that entry.

Parameters

prismatic_direction (*Sequence[Sequence[float or sympy.Expr]]*) – Iterable of 2-element sequences specifying the local axis direction for each prismatic joint. Non-prismatic or uninitialized joints should be passed as `[nan, nan]`.

Returns

A list of 2×1 `sympy.Matrix` objects, each of unit length (for valid inputs) or `[nan, nan]` if the original vector was invalid or zero-length.

Return type

List[`sympy.Matrix`]

VelocityTransformation

class VelocityTransformation(joint_system, Reference_frame_Origin)

Bases: object

Compute kinematic position and velocity transformations for a planar multibody system.

The VelocityTransformation class takes a populated *JointSystem* and a 2×1 reference-frame origin, then builds the global position vector **Pos**, the velocity-transformation matrix **R**, its time derivative **RD**, and a tracking matrix **R_track** for every body in the mechanism. It provides separate methods to handle joints grounded to the base (parent==0) and to propagate kinematics through non-grounded bodies.

Parameters

- **joint_system** (*JointSystem*) – Instance that supplies joint topology, symbolic coordinates, velocities, accelerations, DOF counts, and body-to-global index mappings.
- **Reference_frame_Origin** (*array-like or sympy.Matrix*) – 2×1 vector locating the global frame origin. If not already a *sympy.Matrix*, it will be converted internally.

Joints

List of [parent, child] index pairs for each joint.

Type

list[list[int]]

Type

Joint type codes ('R', 'P', 'F') in the same order as **Joints**.

Type

list[str]

ParentCGtoJoint

Vectors from each parent's center of gravity to its joint.

Type

list[sympy.Matrix]

JointtoChildCG

Vectors from each joint to its child's center of gravity.

Type
list[sympy.Matrix]

prismatic_direction

Unit-length axis vectors for prismatic joints, or [nan, nan] otherwise.

Type
list[sympy.Matrix]

NBodies

Number of moving bodies (ground is index 0, so maximum child index).

Type
int

Q, QD, QDD

Symbolic position, velocity, and acceleration variables extracted from joint_system.

Type
list[sympy.Symbol]

NDoF

Degrees of freedom contributed by each joint (1 for revolute/prismatic, 3 for floating).

Type
list[int]

Body_col

Maps each body (by index-1) to its global coordinate indices in Q, QD, QDD.

Type
list[list[int]]

Reference_frame_Origin

2×1 sympy vector representing the global frame origin after conversion.

Type
sympy.Matrix

Examples

```

>>> from sympy import nan
>>> from multibody.MultibodyCore.joints_system import JointSystem
>>> from multibody.MultibodyCore.kinematicsTransformation import
VelTransformation
>>> # assume `jsys` is a prebuilt JointSystem and `origin` a 2x1
vector
>>> vt = VelTransformation(jsys, origin)
>>> Pos, JointLoc, R, RD, R_track = vt.compute_grounded()
>>> Pos, JointLoc, R, RD, R_track = vt.compute_non_grounded(Pos,
JointLoc, R, RD, R_track)

```

compute_grounded()

Original method: returns (Pos, JointLoc, R, RD, R_track) for joints/bodies grounded to 0. This remains unchanged, so your call to ‘compute_grounded()’ still works.

compute_non_grounded(Pos, JointLoc, R, RD, R_track)

After calling compute_grounded() to get initial (Pos, JointLoc, R, RD, R_track), call this method to fill in the rest for bodies that have a non-zero parent.

Returns updated (Pos, JointLoc, R, RD, R_track).

multibody.points_force_finder

```

points_force_finder(Joints, NBodies, Pos, Q, QD, Initial_Points, Force, JointLoc,
    pris_body_to_parent_map, Body_col, Vel,
    Reference_frame-Origin)

```

Reads the points and forces defined in the dictionaries Initial_Points and Forces and parses them into the MBDsystem class.

Parameters

- **data** (*list*) – Contains fields ‘joints_system.Joint’ (list of [parent, child] pairs) and ‘Type’ (list of joint types).
- **NBodies** (*int*) – Number of bodies in the system (1-based indexing for bodies).

- **Pos** (*sympy.Matrix*) – Global positions of all bodies stacked as [x1, z1, theta1, x2, z2, theta2, ...].
- **Q** (*list of sympy.Symbol*) – The symbolic position coordinates (e.g. [Theta_1, S_2, ...]).
- **QD** (*list of sympy.Symbol*) – The symbolic velocity coordinates (e.g. [ThetaD_1, SD_2, ...]).
- **Initial_Points** (*dict*) –
Must contain:
 - "GR": list of ground points (each [x, z]).
 - "BD": dict keyed by body number -> list of points [xRel, zRel], possibly symbolic.
- **Force** (*dict*) –
Dictionary of forces. May contain:
 - "PointsBD", "CG", "TensionSpring", "TensionDamper", "TorsionSpring", "TorsionDamper".
- **JointLoc** (*list of 2D coordinates*) – The location of each joint, consistent with data['Joints'] ordering.
- **pris_body_to_parent_map** (*list or dict*) – Info for prismatic joints' actual parent angles, as returned by the velocity routines.
- **Body_col** (*list of lists*) – For each body (1-based), the column indices in Q/QD that belong to it. E.g. Body_col[i-1] is a list of DOF indices for body i.
- **Vel** (*sympy.Matrix*) – Global velocities, parallel to Pos (stacked as [xD1, zD1, thetaD1, xD2, zD2, thetaD2, ...]).

Returns

- **Points_All** (*dict*) – Dictionary storing the computed positions for all relevant points (BD, GR, CG, JO).
- **Force_All** (*dict*) – Dictionary storing the computed force contributions for each body in each force category.

- `SpringPotentialEnergy` (`sympy.Expr`) – Total spring potential energy from tension/torsion springs.

`multibody.systems_energy`

`systems_energy`(NBodies, Q, QD, R, CGpos, g, gVec)

Computes the total mechanical energy of the multibody system. Correct only for linear springs and dampers for now.

Parameters

- **NBodies** (*int*) – Number of bodies in the system.
- **Q** (*list of sympy.Symbol*) – Generalized coordinates and velocities.
- **QD** (*list of sympy.Symbol*) – Generalized coordinates and velocities.
- **R** (*sympy.Matrix*) – Velocity transformation matrix.
- **CGpos** (*sympy.Matrix*) – Center-of-gravity positions for each body (rows: bodies, columns: x, z).
- **g** (*sympy.Symbol or float*) – Gravity constant.
- **gVec** (*list-like or sympy.Matrix*) – Per-body gravity-direction factors (e.g. 1 or 0).

Returns

- **Em** (`sympy.Expr`) – Total mechanical energy (kinetic + potential).
- **M** (`sympy.Matrix`) – Block-diagonal mass/inertia matrix.
- **m** (`sympy.Matrix`) – Mass vector for each body.
- **J** (`sympy.Matrix`) – Moment-of-inertia vector for each body.

`multibody.symvars_definition`

`symvars_definition`(varNames, scope_globals)

Converts each string in the list to `sympy.Symbol` and saves it in the global workspace

Parameters: - varNames: list of strings, each a variable name (e.g., ['x', 'y', 'z'])

Returns: - symbolicVars: sympy.Matrix column of symbolic variables - Also assigns each symbol into the global namespace (like assignin('base', ...))

4.5.2 Plotting package

This package is accessed through `multibody.plot` and it allows plotting figures using `plot_multibody_system` method or animations using `animate_multibody`. The method `plt_Template` just has a set of predefined colors that work well with slides and have a light and dark backgrounds. None of these are fully black or fully white.

<code>multibody.plotting.plot_multibody_system</code>	Draw the current configuration of a planar multibody system.
<code>multibody.plotting.animate_multibody</code>	Create and optionally save an animation of the multibody simulation.
<code>multibody.plotting.plt_template</code>	Apply light or dark styling to a Matplotlib Axes to match MATLAB's PlotUtil.

`multibody.plotting.plot_multibody_system`

plot_multibody_system(main_num_vars, MBD, *, xfm=None, t_val=None, frame_bounds=None, dark_mode=False, ax=None, new_figure=True)

Draw the current configuration of a planar multibody system.

Renders bodies' centers of gravity, joints, body-defined points, ground points, forces (CG forces, point forces), springs, dampers, and optional mooring lines—all on a single static plot.

Parameters

- **main_num_vars** (*np.ndarray*) – Numeric array of values for every symbolic variable in `MBD.mainSymVars`. If `MBD.t_update` is not `None` and `t_val` is provided, the corresponding entry in this array is overwritten with `t_val`.
- **MBD** (*object*) – A multibody-dynamics container with attributes:
 - `mainSymVars`, `CGpoints_func`, `BDpoints_func`, `GRpoints_func`, `JOpoints_func`
 - `Force` (dict of force definitions)

- `t_update` (index of time variable or `None`)
- **`xfm`** (*optional*) – Transformation/adaptor for external libraries (e.g. MoorPy) used when plotting mooring lines.
- **`t_val`** (*float or None, optional*) – Current time in seconds. If provided, used to update the time symbol in `main_num_vars` and shown in the plot title.
- **`frame_bounds`** (*tuple of 4 floats or None, optional*) – Axis limits as (xmin, xmax, ymin, ymax). If `None`, limits are computed from the union of all plotted points with a 10% padding.
- **`dark_mode`** (*bool, optional*) – If `True`, applies dark-mode styling. Default is `False`.
- **`ax`** (*plt.Axes or None, optional*) – The Axes to draw on. If `None` or `new_figure` is `True`, a new Figure/Axes pair is created.
- **`new_figure`** (*bool, optional*) – If `True`, always create a fresh Figure/Axes even if `ax` is provided. Default is `True`.

Return type

`tuple[Figure, Axes]`

Returns

- `fig` (`plt.Figure`) – The Figure containing the multibody plot.
- `ax` (`plt.Axes`) – The Axes with all drawn elements.

`multibody.plotting.animate_multibody`

`animate_multibody`(`tvec`, `y`, `MBD`, `mainNumVars`, `frame_bounds=None`,
`save_path=None`, `fps=20`, `loop=False`, `xfm=None`)

Create and optionally save an animation of the multibody simulation.

Uses `plot_multibody_system` internally to render each frame.

Parameters

- **`tvec`** (*array-like, shape (n_frames,)*) – Time values for each frame.
- **`y`** (*array-like, shape (n_states, n_frames)*) – Simulation state array; rows are state variables, columns are time steps.

- **MBD** (*object*) – Multibody-dynamics container used by `plot_multibody_system`.
- **mainNumVars** (*array-like*) – Template array of numeric values for all symbolic variables. Updated each frame with `y` and (if needed) `t`.
- **frame_bounds** (*tuple of 4 floats, optional*) – Axis limits passed to the first call of `plot_multibody_system`.
- **save_path** (*str or Path, optional*) – If provided, path to write out the animation (via `PillowWriter`).
- **fps** (*int, optional*) – Frames per second for playback and saving. Default is 20.
- **loop** (*bool, optional*) – If True, the animation will loop when replayed. Default is False.
- **xfm** (*optional*) – Transformation/adaptor for external plotting (e.g. `MoorPy`).

Returns

`anim` – The `FuncAnimation` object, which can also be used to save or display the animation outside this function.

Return type

`matplotlib.animation.FuncAnimation`

`multibody.plotting.plt_template`

plt_template(`ax`, `dark=False`)

Apply light or dark styling to a Matplotlib Axes to match MATLAB's `PlotUtil`.

Adjusts background color, tick parameters, title and axis label colors, grid appearance, and the default color cycle.

Parameters

- **ax** (*plt.Axes*) – The target Axes to style.
- **dark** (*bool, optional*) – If True, use dark-mode colors; otherwise use light-mode. Default is False.

Return type

None

4.5.3 Checks package

This package checks if the example file is defined correctly. In particular, it verifies if the bodies/joints connectivity is valid, if the joint types are acceptable and if based on the joint type the position vectors are correct. Maybe add the wrong examples as documentation.

<i>multibody.checks.bodies</i>	Validate a planar multibody joint table for structural and kinematic consistency.
<i>multibody.checks.points_forces</i>	Validate point and force definitions for a planar multibody mechanism.

`multibody.checks.bodies`

bodies(joints, types, parent_cg_to_joint, joint_to_child_cg, prismatic_direction)

Validate a planar multibody joint table for structural and kinematic consistency.

This function mirrors MATLAB's `table_check`, performing the following checks:

1. Ground Connection
 - The first joint must connect to ground (parent index 0).
2. Table Completeness and Correctness
 - All input lists/arrays must have the same length.
 - Child body indices must form a continuous sequence (no gaps).
 - Each joint's parent must already appear as a body (no parentless children).
3. Ground-Related Checks
 - Every floating ('F') joint must attach directly to ground.
 - For each joint type:
 - F: `joint_to_child_cg` must be `[nan, nan]`.
 - R: `parent_cg_to_joint` must be fully defined (no nan).
 - P: `joint_to_child_cg` must be fully defined (no nan).

4. Prismatic-Direction Consistency

- For prismatic ('P') joints: direction vector must be normalized (unit length) and not contain nan.
- For non-prismatic joints: prismatic direction must be [nan, nan].

Parameters

- **joints** (*array-like of shape (n_joints, 2)*) – Parent-child index pairs for each joint (ground = 0).
- **types** (*sequence of str*) – Joint type codes for each joint: 'R' = revolute, 'P' = prismatic, 'F' = floating.
- **parent_cg_to_joint** (*sequence of array-like*) – Vectors from each parent body's center of gravity to its joint location.
- **joint_to_child_cg** (*sequence of array-like*) – Vectors from each joint to its child body's center of gravity.
- **prismatic_direction** (*sequence of array-like*) – Direction vectors for prismatic joints; other entries should be [nan, nan].

Returns

If all checks pass, prints a success message: "Table validation successful: No errors found."

Return type

None

Raises

ValueError – If any validation step fails, with a specific error message indicating:

- First joint is not grounded. - Inconsistent input lengths. - Missing or non-continuous body indices. - Parentless child definition. - Floating joint not grounded. - Incorrect nan usage in CG vectors for a given joint type. - Prismatic direction not normalized or missing when required, or erroneously provided for non-prismatic joints.

`multibody.checks.points_forces`

points_forces(joints, types, Initial_Points, Force, t=t)

Validate point and force definitions for a planar multibody mechanism.

Performs these checks in order:

1. Adds missing keys in Force $\rightarrow$ ensures lists for 'PointsBD', 'CG', 'TensionSpring', 'TensionDamper', 'TorsionSpring', 'TorsionDamper'.
2. Adds missing keys in Initial_Points $\rightarrow$ ensures lists for 'BD', 'GR'.
3. Validates each entry in Force['PointsBD']:
 - Must be list-like with 2 elements.
 - Body and point indices are integers within valid ranges.
 - Referenced BD point exists and is not empty.
4. Validates each entry in Force['CG']:
 - Must be list-like with 2 elements.
 - Body indices are within [1, nBodies].
5. Validates TensionSpring and TensionDamper entries:
 - Entries come in (pointStr, param) pairs.
 - pointStr parsed via `check_point_string`.
 - param passes `check_parameter_has_no_function_of_t`.
 - Callable param is converted to symbolic form.
6. Validates TorsionSpring and TorsionDamper entries:
 - Same pairing and time-dependence rules.
 - Checks that any numeric/symbolic iterable has valid body indices.

Parameters

- **joints** (*array-like of shape (n_joints, 2)*) – List of [parent, child] index pairs.
- **types** (*sequence of str*) – Joint type codes ('R', 'P', 'F') matching joints.

- **Initial_Points** (*mapping or object*) –

Must supply:

- Initial_Points['BD']: body-defined point lists
- Initial_Points['GR']: ground reference point list

- **Force** (*dict-like*) –

Force definitions with keys:

'PointsBD', 'CG', 'TensionSpring', 'TensionDamper', 'TorsionSpring', 'TorsionDamper'

- **t** (*sympy.Symbol*) – The symbolic time variable for detecting unwanted time-dependence.

Returns

Prints success messages if all validations pass.

Return type

None

Raises

ValueError – If any of the above checks fail, with an explanatory message.

4.5.4 Tables package

The goal of this package is to allow the user to display on screen the points and forces defined on example for visualization. The bodies are automatically displayed from the JointSystem class. Maybe it would be good to move the bodies under Table module for consistency.

<code>multibody.tables.points_table</code>	Build a pandas DataFrame listing all defined points in the system.
<code>multibody.tables.force_table</code>	Build a consolidated pandas DataFrame of all force entries.

`multibody.tables.points_table`

points_table(Initial_Points)

Build a pandas DataFrame listing all defined points in the system.

Parameters

Initial_Points (*dict*) – Dictionary with two entries:

- "GR"
[list of array-like, shape (2,)] Ground points, each given as [x, z].
- "BD"
[dict of {int: list of array-like, shape (2,)}] Body-defined points.
Keys are body indices (>0), values are lists of points [x_rel, z_rel]
relative to that body's center of gravity.

Returns

df_points – Table with columns:

- Body
[int] Body index (0 for ground points, >0 for body-defined points).
- PointType
[str] Either "GR" (ground) or "BD" (body-defined).
- PointID
[int] Index of the point within its category list.
- Rel coordinates
[array-like, shape (2,)] The [x, z] coordinates (absolute for ground,
relative for body points).

Return type

pandas.DataFrame

`multibody.tables.force_table`

force_table(Force)

Build a consolidated pandas DataFrame of all force entries.

Parameters

Force (*dict of lists*) – Dictionary mapping force categories to their definitions:

- **PointsBD**
[list of lists] Each entry is [body, ptID, Fx, Fy, Mz].
- **CG**
[list of lists] Each entry is [body, Fx, Fy, Mz].
- **TensionSpring**
[list of tuples] Each entry is ((ptStr1, ptStr2), [l0, stiffness]).
- **TensionDamper**
[list of tuples] Each entry is ((ptStr1, ptStr2), damping_value).
- **TorsionSpring**
[list of tuples] Each entry is ([body1, body2], [theta0, stiffness]).
- **TorsionDamper**
[list of tuples] Each entry is ([body1, body2], damping_value).

Returns

df_forces – Table with columns:

- **ForceType**
[str] One of “PointsBD”, “CG”, “TensionSpring”, “Tension-Damper”, “TorsionSpring”, “TorsionDamper”.
- **BodiesConnected**
[str] Comma-separated body IDs (e.g. “2” or “1,3”).
- **PointType**
[str] Point category code (e.g. “BD”, “CG”, “BD,BD”, “GR,BD”, “JO”).
- **PointID**
[str] Numeric point IDs (or “NA” if none).
- **ForceComponents**
[list or None] [Fx, Fy, Mz] for point-based forces, else None.
- **SpringParameters**
[list or None] [l0, stiffness] for springs, else None.
- **DamperParameters**
[scalar or None] Damping coefficient for dampers, else None.

Return type
pandas.DataFrame

4.5.5 ExtForcesManager package

The ExtForcesManager handles coupling between the multibody core and external-force models (e.g. MoorDyn lines, wave loads, etc.). The multibody framework provides state variables such as position and velocity, whereas the external modules provide applied forces.

<i>multibody.ExternalForcesManager</i>	Orchestrates several adapters that talk to third-party libraries (MoorDyn, FAST, CFD, ...).
--	---

ExternalForcesManager

class ExternalForcesManager(mbd_sys)

Bases: `object`

Orchestrates several adapters that talk to third-party libraries (MoorDyn, FAST, CFD, ...). Each adapter must implement:

```
.initialize(t0, q0, qd0) # optional .update(t, q, qd) # advance internal state
.forces() -> list[CartForce] # Cartesian forces in body frames
```

register(adapter)

Attach an adapter instance that follows the contract above.

Return type

`None`

get_adapter(name)

Return the adapter object previously registered under name. Raises `KeyError` if no such adapter exists.

Parameters

name (*str*)

initialize(t0, q0, qd0)

Call initialize method if needed for some reason (Optional)

Parameters

- `t0` (*float*)
- `q0` (*_Buffer* / *_SupportsArray*[*dtype*[*Any*]] / *_NestedSequence*[*_SupportsArray*[*dtype*[*Any*]]] / *bool* / *int* / *float* / *complex* / *str* / *bytes* / *_NestedSequence*[*bool* / *int* / *float* / *complex* / *str* / *bytes*])
- `qd0` (*_Buffer* / *_SupportsArray*[*dtype*[*Any*]] / *_NestedSequence*[*_SupportsArray*[*dtype*[*Any*]]] / *bool* / *int* / *float* / *complex* / *str* / *bytes* / *_NestedSequence*[*bool* / *int* / *float* / *complex* / *str* / *bytes*])

end()

Calls the `.end()` method on each adapter if it exists. (Optional)

init_plot()

Calls `.init_plot()` on each adapter if it exists. (Optional)

generalized_forces(`t`, `main_num_vars`)

Generate the generalized-coordinate force vector ΔRHS from external Cartesian forces.

This method:

1. Calls each registered adapter's `update(t, q, qd, main_num_vars)` to advance its internal state.
2. Retrieves Cartesian forces via `adapter.forces()`, which must return a list of `CartForce` namedtuples with fields:

- `pType` (*str*):
 - 'CG' for forces applied at a body's center of gravity (no moment generated).
 - 'BD' for forces applied at a body-defined point (moment computed about the y-axis).
- `body` (*int*): 1-based child body index in the joint system.
- `localID` (*int*): index into `MBD.BDpoints_func[body]` for 'BD'; ignored for 'CG'.

- `vector` (array_like of float, shape (2,)): planar force components `[Fx, Fz]` in the local body frame.
3. For each `CartForce`, computes a 3-element Cartesian force/moment vector `[Fx, Fz, moment]`, where `moment = 0` for 'CG', and `moment = -(rel_x*Fz - rel_z*Fx)` for 'BD'.
 4. Assembles these into a global Cartesian vector `Fcart` of size `(3 * n_bodies, 1)`.
 5. Applies the numeric velocity-transformation matrix `R_func`: `Qgen = R_func(*main_num_vars).T @ Fcart`.

Parameters

- `t` (*float*) – Current time for which external forces should be evaluated.
- `main_num_vars` (*array_like of float*) – Concatenated numeric values of generalized coordinates (Q), velocities (QD), and any additional parameters required by the lambdified kinematic functions.

Returns

The generalized-force vector to add to the multibody RHS. If no external forces are provided, returns a zero vector.

Return type

`numpy.ndarray`, shape `(n_q,)`

Notes

- Adapters must be registered via `register(adapter)` and must implement:
 - `update(t, q, qd, main_num_vars)`
 - `forces()` -> `list[CartForce]`
- `CartForce` is defined as:

```
>>> CartForce = namedtuple("CartForce",
...                          ["pType", "body", "localID", "vector",
...                          "↪"])
```

- The `body` index is 1-based and corresponds to the child index in `JointSystem.joints`.
- For `pType=='BD'`, `localID` must reference a valid point from `MBD.BDpoints_func[body]`.

4.5.6 YAML parser package (M4E v2.0)

The `yaml_parser` package translates `*.model.yaml` and `*.simulation.yaml` input files into the `SimpleNamespace` expected by `MbdSystem.from_example()`. This allows users to define multibody systems declaratively without writing Python example modules.

Both files are required and must share the same base path (e.g. `my_model.model.yaml` and `my_model.simulation.yaml`).

The entry point is `load_yaml_as_example()`. Internally it instantiates `YAML2Example`, which:

1. anchors the system to a fixed body or synthesises a world anchor,
2. builds a directed body graph and assigns topological indices,
3. emits joints in parent-child order,
4. assembles the kinematic lists expected by `JointSystem`,
5. reads mass and out-of-plane inertia I_y per body,
6. builds initial conditions from declared positions and velocities,
7. parses spring-dampers (`rsdas` / `tsdas`), body loads and motors,
8. validates the model with the `multibody.checks` package.

The simulation file (`*.simulation.yaml`) provides `time_step`, `end_time` and optional `gravity`; these are appended to the namespace as `TimeStep`, `tspan` and `g`.

Note

The parser enforces a 2-D constraint: all body CG Y-coordinates must be zero and orientations must be aligned with the out-of-plane axis. Alternatively, `helper.py` can be modified to specify what are the working axis, updating the variable `AXIS_2D`. In the future, this should be an input parameter. Models imported from 3-D CAD tools may need adjustment before parsing.

Minimum model file structure

```
model:
  bodies:
    - name: ground
      fixed: true
      location: [0, 0, 0]
    - name: link
      location: [0, 0, -0.5]
      mass: 1.0
      inertia:
        moments: [0.083, 0.083, 0.001]
  joints:
    - name: hinge
      type: revolute
      body1: ground
      body2: link
      location: [0, 0, 0]
```

Minimum simulation file structure

```
simulation:
  time_step: 0.01
  end_time: 10.0
  gravity: [0, 0, -9.81]
```

Usage

```
from multibody import load_yaml_as_example, MbdSystem

ex = load_yaml_as_example("yaml_examples/double_pendulum")
MBD = MbdSystem.from_example(ex)
```

<i>multibody.load_yaml_as_example</i>	One-liner wrapper: ex = load_yaml_as_example("myfile.yaml")
<i>multibody.yaml_parser.yaml_adapter.YAML2Example</i>	Convert .model.yaml to the lists required by MBDSysyem.from_example().

`multibody.load_yaml_as_example`

```
load_yaml_as_example(path, *, animation_on=True, SaveMovieOn=None,
                      plotTstep=10)
```

One-liner wrapper: `ex = load_yaml_as_example("myfile.yaml")`

Return type

`SimpleNamespace`

Parameters

- `path (str / Path)`
- `animation_on (bool)`
- `SaveMovieOn (str / None)`
- `plotTstep (int)`

`YAML2Example`

```
class YAML2Example(yaml_path)
```

Bases: `object`

Convert .model.yaml to the lists required by `MBDSysyem.from_example()`.

Parameters

`yaml_path (str / Path)`

```
ensure_world_and_floats()
```

Anchor system using any declared fixed body. If none exist, synthesize a world anchor at origin.

```
build_body_graph()
```

```
assign_body_indices()
```

emit_joint_order()

build_lists()

read_mass_and_inertia()

Build arrays m_0 and J_0 (I_y in the B-frame) aligned with the child (body2) order of `self.joint_records`.

Steps per body: (a) Read ‘mass’ (b) Build I_C from YAML (about COM unless declared otherwise) (c) Get (R_{BC} , r_{BC} , `about_com`) from `_centroidal_transform` (d) If YAML inertia is about the B-frame (not COM): convert it to COM via

parallel-axis inverse: $I_{B@COM} = I_{B@B} - m * (||r||^2 I - r r^T)$,
where $r = r_{BC}$ (B→C in B). If it is already about COM: keep as-is.

(e) Ensure inertia is expressed in the B-frame at COM: $I_{B@COM} = R_{BC} I_C R_{BC}^T$

(f) Extract $J_y = (I_{B@COM})[1,1]$ for 2D (out-of-plane about Y).

parse_spring_dampers()

Populate `Initial_Points` + `Force` from the YAML `spring_dampers` block.

parse_ic()

Build initial conditions vector `self.ic = [Q ; QD]` following `JointSystem` order. XY plane; rotation about Z.

parse_body_loads()

Parse Chrono YAML `body_loads` (from `self.raw.body_loads`) and populate `self.Force`.

Populates:

- `self.Force[“PointsBD”]`: dicts with {“body”, “localID”, “vector”:[F_x, F_z]}
- `self.Force[“CG”]`: dicts with {“body”, “Mz”} (includes CG torques and $r \times F$ from forces)

parse_motors()

as_example_module(animation_on, SaveMovieOn, plotTstep)

Return a namespace ready for `MBDSystem.from_example()`.

Return type

`SimpleNamespace`

4.5.7 Linearization package (M4E v2.0)

The `linearization` package linearizes the nonlinear equations of motion about an equilibrium point, assembles frequency-domain system matrices when external force adapters are registered, and provides a compiled time integrator for the linearized system.

Two usage levels are provided:

Level 1 — symbolic linearization only

`linearize_mbd()` takes an assembled `MbdSystem` and an equilibrium configuration and returns a `LinearizedMBD` dataclass whose four fields (`Mbar`, `Cbar`, `Kbar`, `Fbar`) are NumPy-ready callables. The linearization solves

$$\bar{M}(q_0) \delta \ddot{q} + \bar{C}(q_0) \delta \dot{q} + \bar{K}(q_0) \delta q = \bar{F}$$

where $\bar{M} = B^T M B$ and $B = R|_{q_0}$.

Level 2 — frequency-domain assembly with external adapters

`LinearizationManager` wraps Level 1 and coordinates one or more `LinearizationAdapter` objects that supply frequency-dependent added mass, radiation damping, hydrostatic stiffness and wave excitation.

The typical workflow is:

```
from multibody import MbdSystem
from multibody.linearization.linearization_main import LinearizationManager
from multibody.linearization.hydro_linear_mckf import HydroLinearMCKF

MBD = MbdSystem.from_example(ex)
q0 = MBD.ic[:len(MBD.Q)]

lm = LinearizationManager(MBD, q0, mainNumVars, m0, J0)
lm.register(HydroLinearMCKF(MBD, (mainNumVars, m0, J0), omega_r=omega, body_
↪inputs=body_inputs, ...))
```

(continues on next page)

(continued from previous page)

```

fd = lm.assemble_frequency_domain(omega)    # FrequencyDomainTotals
lm.compile_operating_point(omega0)
sol = lm.integrate_linear_system(tspan=200.0, dt=0.05)

```

`integrate_linear_system` returns a `scipy.integrate.OdeSolution` object.

Custom adapters

Any object satisfying the *LinearizationAdapter* protocol can be registered. The protocol requires:

- a `name` attribute (unique string),
- a `frequency_domain_MCKF(omega)` method returning (M, C, K, F) where F is a dict with keys "dc" (symbolic or numeric DC force) and "phasor" (complex excitation).

Hydrodynamic coupling — regular waves

HydroLinearMCKF implements the adapter protocol for linear potential-flow hydrodynamics under regular (monochromatic) waves. It:

- runs or loads a BEM solution via *Capytaine*,
- provides frequency-dependent added mass $A(\omega)$ and radiation damping $B(\omega)$ matrices,
- computes hydrostatic stiffness K_{hs} and buoyancy forces,
- returns complex wave excitation phasors (Froude-Krylov + diffraction) scaled by wave amplitude.

Problem statement

The module solves the linearized equations of motion for a floating multibody system subject to a single regular wave of angular frequency ω_0 and amplitude A . Working in the Cartesian space of all bodies and projecting onto joint coordinates through the velocity transformation matrix $R_0 = B|_{q_0}$, the assembled frequency-domain system is

$$[\bar{M} + \bar{A}(\omega)] \delta \ddot{q} + [\bar{C} + \bar{B}(\omega)] \delta \dot{q} + [\bar{K} + \bar{K}_{hs}] \delta q = \text{Re} \left(R_0^T \hat{F}(\omega) e^{-i\omega t} \right)$$

where:

- $\bar{M} = R_0^T M_{\text{rigid}} R_0$ — rigid-body inertia in joint coordinates,

- $\bar{A}(\omega) = R_0^T A(\omega) R_0$ — frequency-dependent added mass projected onto joint coordinates,
- $\bar{B}(\omega) = R_0^T B(\omega) R_0$ — radiation damping projected onto joint coordinates,
- $\bar{K}$ and $\bar{K}_{hs}$ — structural and hydrostatic stiffness in joint coordinates,
- $\hat{F}(\omega)$ — complex excitation phasor (Froude-Krylov + diffraction) at wave amplitude A , wave direction 0 rad (head-on).

The Cartesian state per body uses the planar ordering $[F_x, F_z, M_y]$; 6-DOF BEM results are reduced to this 3-DOF subset before projection.

The time-domain signal reconstructed by `assemble_time_forces_from_linear_solution()` is

$$F(t) = \text{Re}\left(\hat{F}(\omega_0) e^{-i\omega_0 t}\right) r(t) - B(\omega_0) \dot{x}(t) - K_{hs} x(t) + F_b$$

where $r(t)$ is an optional cosine ramp and F_b is the static buoyancy force.

Hydrodynamic coefficient convention

All dimensional BEM quantities (added mass, radiation damping, excitation force, hydrostatic stiffness) are stored with the units produced by Capytaine and are proportional to the fluid density ρ used in the BEM solve. When the simulation density differs from the density stored in the `.nc` file, every coefficient is rescaled by $\rho_{\text{user}}/\rho_{\text{nc}}$. This is a linear proportional rescaling — it is not the same non-dimensionalization that tools such as WEC-Sim apply.

Limitations

- Linear regime only. The theory assumes small-amplitude waves and small motions about the equilibrium q_0 . Results degrade for large wave amplitudes or large excursions from equilibrium.
- Regular (monochromatic) waves. Only a single wave frequency and direction (0 rad, head-on) are supported. Irregular sea states require superposition outside this module.
- Dimensional coefficients. Added mass, damping, stiffness, and excitation forces are kept dimensional. Non-dimensionalization (e.g. by $\rho g A$) is not performed automatically.
- 2-D planar embedding. The full 6-DOF BEM solution is reduced to the in-plane degrees of freedom $[F_x, F_z, M_y]$. Out-of-plane forces are discarded.

- Potential flow only. Viscous drag (Morison equation) is not included.
- Equilibrium-point linearization. The velocity transformation matrix R_0 is evaluated once at q_0 . It does not update as the system evolves.

<code>multibody.linearize_mbd</code>	Linearize $B(q)^T M B(q) \ddot{q} = B(q)^T (f - M \dot{B}(q, \dot{q}) \dot{q})$ about $(q_0, \dot{q}_0)$ with $\ddot{q}_0 = 0$, using your class' symbolic fields.
<code>multibody.linearization. linearize_eom.LinearizedMBD</code>	
<code>multibody.linearization. linearization_main. LinearizationManager</code>	Final workflow:
<code>multibody.linearization. linearization_main. LinearizationAdapter</code>	Adapter returns frequency-domain M, C, K plus a force dict containing:
<code>multibody.linearization. hydro_linear_mckf.HydroLinearMCKF</code>	Frequency-domain hydrodynamics adapter with force dict convention:

`multibody.linearize_mbd`

linearize_mbd(MBD, q0, m0, J0, print_sym_matrices=False)

Linearize $B(q)^T M B(q) \ddot{q} = B(q)^T (f - M \dot{B}(q, \dot{q}) \dot{q})$ about $(q_0, \dot{q}_0)$ with $\ddot{q}_0 = 0$, using your class' symbolic fields.

Expected attributes on MBD (names follow your codebase):

MBD.Q -> list[Sym] : q symbols MBD.QD -> list[Sym] : qd symbols
 MBD.R -> sym.Matrix : B(q) (cartesian-velocity map) MBD.RD -> sym.Matrix
 : dotB(q, qd) MBD.M -> sym.Matrix : Cartesian mass/inertia (block-diag)
 MBD.ForceAllCombined -> sym.Matrix (Cartesian forces f). If absent and assume_f_constant=True,

we set f as a symbol vector (treated as constant).

Returns

$$M_0 * \ddot{q} + (C + dF/d\dot{q}) * \dot{q} + K * q = F_{\text{ext}}$$

Return type

LinearizedEOM with $M_0, K_q, K_{\dot{q}}$ so that

LinearizedMBD

class **LinearizedMBD**(Mbar, Cbar, Kbar, Fbar)

Bases: object

Parameters

- **Mbar** (*Callable*[[...], *ndarray*])
- **Cbar** (*Callable*[[...], *ndarray*])
- **Kbar** (*Callable*[[...], *ndarray*])
- **Fbar** (*Callable*[[...], *ndarray*])

Mbar: *Callable*[..., *ndarray*]

Cbar: *Callable*[..., *ndarray*]

Kbar: *Callable*[..., *ndarray*]

Fbar: *Callable*[..., *ndarray*]

LinearizationManager

class **LinearizationManager**(mbd_sys, q0, mainNumVars, m0, J0, *,
print_sym_matrices=False)

Bases: object

Final workflow:

- 1) `assemble_frequency_domain( )`: builds joint-space $M()$, $C()$, K , $Fhat()$ and `eq_residual` - DC handled via adapter-provided $F["dc"]$ (symbolic) -> contributes to `eq_residual`, K_{dc} , C_{dc} - phasor excitation handled via $F["phasor"]$ -> contributes only to forcing term (mapped with $R0.T$)
- 2) `compile_operating_point( 0)`: caches $M0, C0, K0, Fhat0, eq_residual$ and `ramp_T`
- 3) time integration uses compiled constants:

$$M0 \ddot{q} + C0 \dot{q} + K0 q = eq_residual + ramp(t) * Re\{Fhat0 e^{i \omega t}\}$$

Parameters

- **mbd_sys** (*Any*)

- `q0` (*npt.ArrayLike*)
- `mainNumVars` (*npt.ArrayLike*)
- `m0` (*npt.ArrayLike*)
- `J0` (*npt.ArrayLike*)
- `print_sym_matrices` (*bool*)

register(adapter)

Register a linearization adapter. Adapters must have unique names.

Return type

`None`

Parameters

adapter (*LinearizationAdapter*)

assemble_frequency_domain(omega)

Return type

`FrequencyDomainTotals`

Parameters

omega (*float* / *_Buffer* / *_SupportsArray[dtype[Any]]* / *_NestedSequence[_SupportsArray[dtype[Any]]]* / *bool* / *int* / *complex* / *str* / *bytes* / *_NestedSequence[bool* / *int* / *float* / *complex* / *str* / *bytes]*)

static ramp(t, T)

Return type

`float`

Parameters

- **t** (*float*)
- **T** (*float*)

compile_operating_point(omega0, *, eq_tol=1e-08, eq_mode='zero_if_small', ramp_T=0.0)

Return type

`CompiledOperatingPoint`

Parameters

- **omega0** (*float*)
- **eq_tol** (*float*)
- **eq_mode** (*str*)
- **ramp_T** (*float*)

has_compiled()

Return type

`bool`

force_time(t)

Evaluate the time-domain forcing at time `t` using the compiled operating point and the ramp function.

Return type

`ndarray`

Parameters

t (*float*)

integrate_linear_system(*, `tspan`, `dt=None`, `method='RK45'`, `omega0=None`,
`eq_tol=1e-08`, `eq_mode='zero_if_small'`,
`**solver_kwargs`)

Parameters

- **tspan** (*float*)
- **dt** (*float / None*)
- **method** (*str*)
- **omega0** (*float / None*)
- **eq_tol** (*float*)
- **eq_mode** (*str*)

LinearizationAdapter

class LinearizationAdapter(*args, **kwargs)

Bases: Protocol

Adapter returns frequency-domain M,C,K plus a force dict containing:

- F[“dc”]: SymPy Matrix (ncart,1) DC Cartesian force as a function of q,qd (typically q only)
- F[“phasor”]: complex ndarray phasor excitation (n ,ncart,1) or (ncart,1)

The manager:

- uses only F[“phasor”] for forcing (mapped by R0.T)
- linearizes only F[“dc”] into eq_residual and K_dc, C_dc (via Jacobians of $R^T F_{dc}$)

name: str

frequency_domain_MCKF(omega)

Return type

Tuple[ndarray, ndarray, ndarray, Dict[str, Any]]

Parameters

omega (*float / ndarray*)

HydroLinearMCKF

class HydroLinearMCKF(mbd_sys, numericalValuesTuple, is_2D=False, *, data=None, omega_r=None, body_inputs=None, wave_amplitude=1.0, equilibrium_pos=None, **kwargs)

Bases: object

Frequency-domain hydrodynamics adapter with force dict convention:

returns (M(), C(), K, F) where $F = \{\text{“dc”}: F_{dc_sym}, \text{“phasor”}: \text{Fhat}()\}$

- F_{dc_sym} is a SymPy Cartesian (3N,1) DC force vector (buoyancy) that depends on q via $\theta_{i,}$.
- $\text{Fhat}()$ is complex excitation phasor in Cartesian (3N,1) or (n ,3N,1).

Cartesian ordering per body: [Fx, Fz, My].

Parameters

- **mbd_sys** (*Any*)
- **numericalValuesTuple** (*tuple*)
- **is_2D** (*bool*)
- **data** (*Optional[Any]*)
- **omega_r** (*Optional[np.ndarray]*)
- **body_inputs** (*Optional[Dict[int, Any]]*)
- **wave_amplitude** (*float*)
- **equilibrium_pos** (*Optional[np.ndarray]*)
- **kwargs** (*Any*)

name = 'HydroLinearMCKF'

frequency_domain_MCKF(omega)

Returns

F["dc"] = SymPy Matrix (3N,1) F["phasor"] = complex ndarray,
shape (3N,1) or (n ,3N,1)

Return type

M(), C(), K, F where

Parameters

omega (*float / ndarray*)

assemble_time_forces_from_linear_solution(t, y, mbd_sys, mainNumVars, *,
omega0, R0, ramp_T=0.0,
include_buoyancy=True,
return_forces_inJC=False)

Assemble time-domain hydrodynamic forces using joint states from the linear integrator. This is a post-processing step that maps the integrated joint trajectories to Cartesian forces using the frequency-domain MCKF data and the planar embedding defined by R0.

Return type

Dict[str, ndarray]

Parameters

- `t` (*ndarray*)
- `y` (*ndarray*)
- `mbd_sys` (*Any*)
- `mainNumVars` (*ndarray*)
- `omega0` (*float*)
- `R0` (*ndarray*)
- `ramp_T` (*float*)
- `include_buoyancy` (*bool*)
- `return_forces_inJC` (*bool*)

`static save_bem_data(save_dir, dataset, file_name)`

Return type

None

Parameters

- `save_dir` (*str*)
- `dataset` (*Any*)
- `file_name` (*str*)

MoorPy mooring adapter

The *MoorPyLinearMCKF* adapter couples MoorPy quasi-static / linearised mooring dynamics with the `LinearizationManager`. MoorPy is an optional dependency.

MoorPy mooring dynamics adapter

The MoorPy adapter adds quasi-static and linearised mooring dynamics to the *LinearizationManager* frequency-domain assembly. It implements the *LinearizationAdapter* protocol and is entirely optional — the base multibody package does not depend on MoorPy.

Architecture

M4E Initial_Points

GR anchors → MoorPy fixed points (type = 1)
 BD fairleads → MoorPy coupled points (type = -1, attached to body)

MoorPy input module (moorpy_inputs.py)

line_types (diameter, mass, stiffness, drag/added-mass coefficients)
 mooring_lines (name, type, length, segments, anchor ref, fairlead ref)
 point references (by M4E GR/BD IDs - no duplicated coordinates)

MoorPySystemBuilder

evaluates M4E body CG positions at q_0
 evaluates M4E BD/GR positions at q_0
 constructs MoorPy bodies, fixed points, coupled fairleads
 calls System.initialize() (catenary solve, internal DOFs only)
 calls System.updateSystemDynamicMatrices()

MoorPyLinearMCKF.frequency_domain_MCKF()

calls System.getSystemDynamicMatrices("coupled", lines_only=True)
 → M, A, B, K each ($n_{\text{CpldDOF}} \times n_{\text{CpldDOF}}$)
 $n_{\text{CpldDOF}} = 6 \cdot n_{\text{bodies}} + 3 \cdot n_{\text{attached_points}}$
 extracts body block with $\text{idx} = [6i+d \text{ for } i \text{ in range}(n_{\text{b}}) \text{ for } d \text{ in } (0,2,4)]$
 → M3, A3, B3, K3 each ($3 \cdot n_{\text{b}} \times 3 \cdot n_{\text{b}}$)
 maps into $M_{\text{cart}} = M3 + A3$, $C_{\text{cart}} = B3$, $K_{\text{cart}} = K3$
 embedded in ($3 \cdot n_{\text{m4e}} \times 3 \cdot n_{\text{m4e}}$) with zero rows for non-mooring bodies
 returns M_{cart} , C_{cart} , K_{cart} , $F\{\text{"dc"}: \text{SymPy}, \text{"phasor"}: \text{zeros}\}$

LinearizationManager

body Cartesian → M4E joint coordinates
 $M_q = R_0 \ M_{\text{cart}} \ R_0$

(continues on next page)

(continued from previous page)

```
Cq = R0 (C_cart R0 + M_cart RD0)
Kq = R0 K_cart R0
```

Architectural guarantees

The following statements are invariants of the implementation.

1. MoorPy performs the fairlead-to-body transformation. `Body.getDynamicMatrices` calls MoorPy's internal `translateMatrix3to6` using the body-local fairlead offset stored in `body.rPointRel`. The resulting body matrix already accounts for the moment arm.
2. MoorPy includes fairlead moment arms in body forces and moments. `Body.getForces` similarly reduces line end-forces and torques to the body reference point. The mean wrench returned by the adapter therefore includes translation-rotation coupling due to the fairlead offset.
3. MoorPy body-level stiffness includes geometric contributions. `K` returned by `getSystemDynamicMatrices` accounts for the change in catenary geometry under small body displacements, including the pretension contribution.
4. The M4E adapter does not reapply a fairlead transformation. No additional `H` matrix is applied. Doing so would double-count the fairlead offset that MoorPy already applied internally.
5. Reduction uses local DOF indices `[0, 2, 4]`. MoorPy's 6-DOF body ordering is `[x, y, z, roll, pitch, yaw]`. M4E planar DOFs correspond to indices 0 (surge), 2 (heave), 4 (pitch). For `nb` mooring bodies:

```
idx = [6*i + d for i in range(nb) for d in (0, 2, 4)]
```

6. MoorPy body reference points coincide with M4E CGs. `System.addBody` is called with `rCG = np.zeros(3)` so the MoorPy body reference frame origin coincides with the CG. No reference-point translation is required in the adapter.
7. `q0` is externally supplied and remains unchanged. The adapter reads `q0` once at construction time. MoorPy's equilibrium solver is not invoked; body positions are held fixed at the prescribed `q0`.
8. Mean mooring force is optional. `include_mean_force=False` (default) returns a zero

SymPy DC force. When `True`, the mean line-induced body wrench at `q0` is returned as a constant symbolic vector for equilibrium-residual checking. It does not modify `q0` and does not double-count stiffness already present in `K`.

9. MoorPy is an optional dependency. `import multibody` never triggers a MoorPy import. Only instantiating *MoorPyLinearMCKF* raises an `ImportError` if MoorPy is not installed.
10. Experimental limitations of MoorPy's dynamic matrices are identified. See the [Known limitations](#) section below.

Matrix semantics

Symbol	Units	Description	MoorPy source
<code>M_cart</code>	kg	Line structural inertia + line added mass ($M + A$)	<code>M, A</code> from <code>getSystemDynamicMatrices</code>
<code>C_cart</code>	kg/s	Mooring line viscous drag damping	<code>B</code> from <code>getSystemDynamicMatrices</code>
<code>K_cart</code>	N/m or N · m/rad	Mooring stiffness including geometric contribution	<code>K</code> from <code>getSystemDynamicMatrices</code>
<code>F["dc"]</code>	N or N · m	Mean line-induced body wrench at <code>q0</code> (optional)	<code>getForces(DOFtype="coupled", lines_only=True)</code>
<code>F["phasor"]</code>	N or N · m	Harmonic mooring excitation — zero (no external wave load)	n/a

Cartesian coordinate ordering per body: `[Fx, Fz, My]` (corresponding to DOFs `[x, z, pitch]`).

Known limitations

- Damping approximation. `B` (and hence `C_cart`) is computed assuming unit-amplitude motion of all line nodes. For physically accurate mooring damping, RAOs must be provided to `line.updateLumpedMass` before calling `updateSystemDynamicMatrices`. This limitation is inherited from MoorPy's `updateSystemDynamicMatrices` API.
- Cross-body `M/A/B` not assembled. `getSystemDynamicMatrices` assembles cross-body coupling only for `K`. Off-diagonal `M`, `A`, and `B` blocks between different mooring bodies

are zero. Cross-body K coupling is preserved.

- `nCpldDOF` includes attached fairlead points. For a system with one coupled body and one attached fairlead point, `getSystemDynamicMatrices` returns a 9×9 matrix (6 body DOFs + 3 attached-point DOFs). The reduction indices `[0, 2, 4]` correctly address the body block only. The attached-point block is not used.
- BD-to-BD mooring lines not supported. The current schema only allows lines from a GR anchor to a BD fairlead. Body-to-body lines (both endpoints on M4E bodies) are not supported and will fail schema validation.
- MoorPy dev branch required. `System.updateSystemDynamicMatrices` and `System.getSystemDynamicMatrices` are not present in MoorPy 1.1.0 (PyPI). The adapter requires MoorPy installed from the dev branch:

```
pip install git+https://github.com/NatLabRockies/MoorPy.git@dev
```

or the local editable install used during development:

```
pip install -e /path/to/MoorPy
```

Numerical validation procedure

The following five-step chain verifies the adapter end-to-end. It is implemented in `Examples_linearization/single_body_moorpy/main_linearization.py` and executed automatically when the example is run.

```
Step 1 MoorPy full body matrix
      ms.getSystemDynamicMatrices("coupled", lines_only=True)
      → M_mp, A_mp, B_mp, K_mp    shape (nCpldDOF × nCpldDOF)

Step 2 Independent manual reduction
      idx = [0, 2, 4]
      M3 = (M_mp + A_mp)[np.ix_(idx, idx)]
      C3 = B_mp[np.ix_(idx, idx)]
      K3 = K_mp[np.ix_(idx, idx)]

Step 3 Adapter Cartesian output
      adapter.frequency_domain_MCKF( ) → M_cart, C_cart, K_cart
```

(continues on next page)

(continued from previous page)

```

assert np.allclose(M_cart, M3)    # embedded in ncart×ncart zeros
assert np.allclose(C_cart, C3)
assert np.allclose(K_cart, K3)

```

Step 4 Manager joint-coordinate output

```

totals = manager.assemble_frequency_domain( )
M_joint = totals.M,  C_joint = totals.C,  K_joint = totals.K

```

Step 5 Independent R0 X_cart R0

```

assert np.allclose(R0.T @ M_cart @ R0,  M_joint - M_mbd)
assert np.allclose(R0.T @ C_cart @ R0,  C_joint - C_mbd)
assert np.allclose(R0.T @ K_cart @ R0,  K_joint - K_mbd)

```

Usage example

```

import numpy as np
from multibody import MbdSystem
from multibody.linearization.linearization_main import LinearizationManager
from multibody.linearization._moorpy_schema import from_module, validate_
    ↳moorpy_inputs
from multibody.linearization.moorpy_linear_mckf import MoorPyLinearMCKF

from Examples_linearization.single_body_moorpy import M4E_inputs as ex
from Examples_linearization.single_body_moorpy import moorpy_inputs as mp_
    ↳mod

# Parse and validate mooring topology
moorpy_inputs = from_module(mp_mod)
validate_moorpy_inputs(moorpy_inputs, ex.Initial_Points)

# Build MBD system and operating point
mbd_sys = MbdSystem.from_example(ex)
q0 = (mbd_sys.ic - ex.ic)[: len(mbd_sys.Q)]
mainNumVars = ex.ic.copy()

```

(continues on next page)

(continued from previous page)

```

# Instantiate adapter and register with manager
adapter = MoorPyLinearMCKF(mbd_sys, q0, mainNumVars, moorpy_inputs)
manager = LinearizationManager(mbd_sys, q0, mainNumVars, ex.m0, ex.J0)
manager.register(adapter)

# Assemble frequency-domain totals
totals = manager.assemble_frequency_domain(omega=0.5)

# Inspect joint-coordinate matrices
nq = manager.nq
K_joint = np.asarray(totals.K).reshape(nq, nq)
print("K_joint =", K_joint)

```

Minimum working example: Examples_linearization/single_body_moorpy/

New modules added

```

source/multibody/linearization/
    _moorpy_schema.py           # Input schema dataclasses and validation
    _moorpy_system_builder.py   # Constructs MoorPy System at q0
    moorpy_linear_mckf.py       # LinearizationAdapter implementation

Examples_linearization/single_body_moorpy/
    M4E_inputs.py              # MBD topology (one floating body)
    moorpy_inputs.py           # Mooring line type and topology
    main_linearization.py       # Runnable example with validation checks

tests/cases/linearization/
    test_moorpy_schema.py       # 33 schema + validation tests
    test_manager_adapter.py     # 24 manager/adapter protocol tests
    test_moorpy_full.py         # 39 point mapping, reduction, coupling tests

```

API reference

<code>multibody.linearization. moorpy_linear_mckf.MoorPyLinearMCKF</code>	Mooring dynamics adapter for <i>LinearizationManager</i> .
<code>multibody.linearization. _moorpy_schema.MoorPyInputs</code>	Complete MoorPy mooring-system inputs for one scenario.
<code>multibody.linearization. _moorpy_schema.LineType</code>	Properties for one mooring line type.
<code>multibody.linearization. _moorpy_schema.MooringLine</code>	Definition of a single mooring line.
<code>multibody.linearization. _moorpy_schema.PointRef</code>	A reference to one point in M4E <code>Initial_Points</code> .
<code>multibody.linearization. _moorpy_schema.from_module</code>	Build a <code>MoorPyInputs</code> from a <code>moorpy_inputs.py</code> module.
<code>multibody.linearization. _moorpy_schema. validate_moorpy_inputs</code>	Validate inputs against the M4E <code>Initial_Points</code> structure.

MoorPyLinearMCKF

```
class MoorPyLinearMCKF(mbd_sys, q0, mainNumVars, moorpy_inputs, *,
                        include_mean_force=False)
```

Bases: `object`

Mooring dynamics adapter for *LinearizationManager*.

Constructs a MoorPy system at the M4E operating point `q0`, extracts body-level dynamic matrices, reduces them to M4E planar Cartesian coordinates `[x, z, pitch]`, and assembles the global body-Cartesian matrices.

Parameters

- **mbd_sys** (Any) – Compiled *MbdSystem*.
- **q0** (Union[_Buffer, _SupportsArray[dtype[Any]], _NestedSequence[_SupportsArray[dtype[Any]]], bool, int, float, complex, str, bytes, _NestedSequence[bool | int | float | complex | str | bytes])) – Joint-coordinate equilibrium vector, shape (nq,).

- **mainNumVars** (Union[_Buffer, _SupportsArray[dtype[Any]], _NestedSequence[_SupportsArray[dtype[Any]]], bool, int, float, complex, str, bytes, _NestedSequence[bool | int | float | complex | str | bytes])) – Full numeric parameter vector for the M4E model. **q** and **qd** slots are overwritten internally.
- **moorpy_inputs** (Any) – Validated *MoorPyInputs*.
- **include_mean_force** (bool) – When True, populate **F["dc"]** with the constant mean body wrench due to mooring lines at **q0**. When False (default), **F["dc"]** is a zero SymPy matrix.

Raises

ImportError – If MoorPy is not installed (raised at instantiation, not at import).

Notes

q0 is read once during construction and is never modified. The MoorPy equilibrium solver is not invoked — bodies remain fixed at the prescribed **q0**.

name: str = 'MoorPyLinearMCKF'

frequency_domain_MCKF(omega)

Return mooring matrices in M4E body-Cartesian coordinates.

All MoorPy matrices are frequency-independent. **omega** is accepted for API compatibility but does not change the returned values.

Parameters

omega (Union[float, _Buffer, _SupportsArray[dtype[Any]], _NestedSequence[_SupportsArray[dtype[Any]]], bool, int, complex, str, bytes, _NestedSequence[bool | int | float | complex | str | bytes])) – Frequency [rad/s]. Scalar or 1-D array.

Return type

Tuple[ndarray, ndarray, ndarray, Dict[str, Any]]

Returns

- **M_cart** (ndarray, shape (3·nb_m4e, 3·nb_m4e)) – Mooring inertia = line structural mass + line added mass.

- `C_cart` (ndarray, shape $(3 \cdot \text{nb_m4e}, 3 \cdot \text{nb_m4e})$) – Mooring damping (unit-amplitude approximation, see limitations).
- `K_cart` (ndarray, shape $(3 \cdot \text{nb_m4e}, 3 \cdot \text{nb_m4e})$) – Mooring stiffness including geometric stiffness from pretension.
- `F` (dict) – "dc" — constant SymPy Matrix $(3 \cdot \text{nb_m4e}, 1)$. "phasor" — complex ndarray $(3 \cdot \text{nb_m4e}, 1)$ of zeros. Mooring lines produce no harmonic external excitation.

Notes

M4E Cartesian coordinate ordering per body: `[Fx, Fz, My]`. Non-mooring M4E bodies receive zero blocks. Cross-body mooring K terms are preserved; M/A/B cross-body terms are zero due to a limitation in MoorPy's assembly (see module docs).

property moorpy_system: Any

The underlying `moorpy.System` object.

property reduction_indices: ndarray

6-DOF indices used to reduce MoorPy matrices to M4E planar DOFs.

property m4e_cartesian_rows: ndarray

Global Cartesian row indices for the mooring bodies.

MoorPyInputs

class MoorPyInputs(line_types, mooring_lines, depth=200.0, rho=1025.0, g=9.81)

Bases: `object`

Complete MoorPy mooring-system inputs for one scenario.

Typically constructed via `from_module()` from a `moorpy_inputs.py` example file.

Parameters

- **line_types** (Dict[str, *LineType*]) – Dict mapping line-type name to its *LineType* properties.
- **mooring_lines** (List[*MoorLine*]) – Ordered list of *MoorLine* definitions.

- **depth** (float) – Water depth [m]. Passed to `moorpy.System(depth=...)`.
- **rho** (float) – Water density [kg/m³].
- **g** (float) – Gravitational acceleration [m/s²].

line_types: Dict[str, [LineType](#)]

mooring_lines: List[[MooringLine](#)]

depth: float = 200.0

rho: float = 1025.0

g: float = 9.81

[LineType](#)

```
class LineType(diameter, mass_per_length, axial_stiffness, drag_coefficient=1.2,  
               added_mass_coefficient=1.0, drag_coefficient_axial=0.4,  
               added_mass_coefficient_axial=0.0)
```

Bases: `object`

Properties for one mooring line type.

All fields map directly to a MoorPy `System.addLineType` call followed by updating the resulting dict.

Parameters

- **diameter** (float) – Volume-equivalent diameter [m]. Maps to MoorPy `d`.
- **mass_per_length** (float) – Linear mass density [kg/m]. Maps to MoorPy `mass`.
- **axial_stiffness** (float) – Axial (extensional) stiffness EA [N]. Maps to MoorPy `EA`.
- **drag_coefficient** (float) – Transverse Morison drag coefficient `Cd` [-]. Maps to MoorPy `Cd`.
- **added_mass_coefficient** (float) – Transverse added-mass coefficient `Ca` [-]. Maps to MoorPy `Ca`.

- **drag_coefficient_axial** (float) – Tangential (axial) drag coefficient CdAx [-]. Maps to MoorPy CdAx.
- **added_mass_coefficient_axial** (float) – Tangential (axial) added-mass coefficient CaAx [-]. Maps to MoorPy CaAx.

diameter: float

mass_per_length: float

axial_stiffness: float

drag_coefficient: float = 1.2

added_mass_coefficient: float = 1.0

drag_coefficient_axial: float = 0.4

added_mass_coefficient_axial: float = 0.0

MooringLine

```
class MooringLine(name, line_type, unstretched_length, num_segments, anchor,
                  fairlead)
```

Bases: object

Definition of a single mooring line.

Parameters

- **name** (str) – Unique identifier for this line.
- **line_type** (str) – Key into the `line_types` dict of the parent *MoorPyInputs*.
- **unstretched_length** (float) – Unstretched (natural) length of the line [m].
- **num_segments** (int) – Number of finite-element segments to discretise the line into.
- **anchor** (*PointRef*) – Reference to the anchor point (must be `kind == "GR"`).
- **fairlead** (*PointRef*) – Reference to the fairlead point (must be `kind == "BD"`).

```
name: str

line_type: str

unstretched_length: float

num_segments: int

anchor: PointRef

fairlead: PointRef
```

PointRef

```
class PointRef(kind, point_id, body_id=0)
```

Bases: `object`

A reference to one point in M4E `Initial_Points`.

Parameters

- **kind** (`str`) – "GR" for a ground/anchor point or "BD" for a body-defined (fairlead) point.
- **point_id** (`int`) – 0-based index into `Initial_Points["GR"]` (when `kind == "GR"`) or into `Initial_Points["BD"][body_id]` (when `kind == "BD"`).
- **body_id** (`int`) – 1-based body number. Required (and validated) when `kind == "BD"`; ignored when `kind == "GR"`.

```
kind: str

point_id: int

body_id: int = 0
```

`multibody.linearization._moorpy_schema.from_module`

```
from_module(module)
```

Build a *MoorPyInputs* from a `moorpy_inputs.py` module.

The module must expose two module-level variables:

- `line_types`: `dict[str, dict]` — each inner dict has keys matching *LineType* field names.
- `mooring_lines`: `list[dict]` — each dict has keys matching *MooringLine* field names, with `anchor` and `fairlead` values that are dicts matching *PointRef* field names.

Optional module-level floats `depth`, `rho`, and `g` are forwarded to *MoorPyInputs*.

Parameters

`module` (`ModuleType`) – An imported `moorpy_inputs` module.

Return type

MoorPyInputs

`multibody.linearization._moorpy_schema.validate_moorpy_inputs`

`validate_moorpy_inputs`(`inputs`, `initial_points`)

Validate inputs against the M4E `Initial_Points` structure.

Return type

`None`

Parameters

- **`inputs`** (*MoorPyInputs*)
- **`initial_points`** (*dict*)

Checks performed

- All line-type property values are positive where required.
- Every line name is unique.
- Every line references a known line type.
- `unstretched_length > 0` and `num_segments >= 1`.
- Anchors reference "GR" points at valid indices.
- Fairleads reference "BD" points at valid body and point indices.

type inputs

MoorPyInputs

param inputs

The *MoorPyInputs* object to check.

type initial_points

dict

param initial_points

The M4E `Initial_Points` dict (must contain "GR" list and "BD" dict).

raises ValueError

On the first validation failure encountered, with a descriptive message.

4.5.8 Validation

Validation has been migrated from standalone scripts to a self-contained pytest suite under `tests/`. See the [Tests](#) section for the full description of the four test suites (MATLAB symbolic, MATLAB energy, Chrono regression, YAML parser, and linearization) and instructions for running them.

A

Acc (MbdSystem attribute), 88
 Acc_func (MbdSystem attribute), 89
 add_joint() (JointSystem method), 92
 added_mass_coefficient (LineType attribute), 135
 added_mass_coefficient_axial (LineType attribute), 135
 anchor (MooringLine attribute), 136
 animate_multibody() (in module multibody.plotting), 100
 as_example_module() (YAML2Example method), 114
 assemble_frequency_domain() (LinearizationManager method), 120
 assemble_time_forces_from_linear_solution() (HydroLinearMCKF method), 123
 assign_body_indices() (YAML2Example method), 113
 axial_stiffness (LineType attribute), 135

B

BDpoints_func (MbdSystem attribute), 89
 BDvel_func (MbdSystem attribute), 89
 bodies() (in module multibody.checks), 102
 Body_col (JointSystem attribute), 91
 Body_col (MbdSystem attribute), 88
 Body_col (VelocityTransformation attribute), 95

body_id (PointRef attribute), 136
 BodyDataSym (MbdSystem attribute), 88
 build_body_graph() (YAML2Example method), 113
 build_lists() (YAML2Example method), 114

C

Cbar (LinearizedMBD attribute), 119
 CGpoints_func (MbdSystem attribute), 89
 CGvel_func (MbdSystem attribute), 89
 compile_operating_point() (LinearizationManager method), 120
 compute_grounded() (VelocityTransformation method), 96
 compute_non_grounded() (VelocityTransformation method), 96
 coordinate_finder() (JointSystem method), 92

D

depth (MoorPyInputs attribute), 134
 diameter (LineType attribute), 135
 display_table() (JointSystem method), 92
 drag_coefficient (LineType attribute), 135

- `drag_coefficient_axial` (LineType attribute), 135
- ## E
- `emit_joint_order()` (YAML2Example method), 113
- `end()` (ExternalForcesManager method), 109
- `Energy` (MbdSystem attribute), 89
- `Energy_func` (MbdSystem attribute), 89
- `ensure_world_and_floats()` (YAML2Example method), 113
- `ExternalForcesManager` (class in multibody), 108
- ## F
- `fairlead` (MooringLine attribute), 136
- `Fbar` (LinearizedMBD attribute), 119
- `Force` (MbdSystem attribute), 88
- `Force_All` (MbdSystem attribute), 89
- `force_table()` (in module multibody.tables), 106
- `force_time()` (LinearizationManager method), 121
- `ForcesPointsSym` (MbdSystem attribute), 88
- `frequency_domain_MCKF()` (HydroLinearMCKF method), 123
- `frequency_domain_MCKF()` (LinearizationAdapter method), 122
- `frequency_domain_MCKF()` (MoorPyLinearMCKF method), 132
- `from_data()` (JointSystem class method), 92
- `from_example()` (MbdSystem class method), 90
- `from_module()` (in module multibody.linearization._moorpy_schema), 136
- ## G
- `g` (MbdSystem attribute), 88
- `g` (MoorPyInputs attribute), 134
- `generalized_forces()` (ExternalForcesManager method), 109
- `get_adapter()` (ExternalForcesManager method), 108
- `graph` (MbdSystem attribute), 89
- `GRpoints_func` (MbdSystem attribute), 89
- `gVec` (MbdSystem attribute), 88
- ## H
- `has_compiled()` (LinearizationManager method), 121
- `HydroLinearMCKF` (class in multibody.linearization.hydro_linear_mckf), 122
- ## I
- `ic` (MbdSystem attribute), 88
- `init_plot()` (ExternalForcesManager method), 109
- `Initial_Points` (MbdSystem attribute), 88
- `initialize()` (ExternalForcesManager method), 108
- `integrate()` (MbdSystem method), 90
- `integrate_linear_system()` (LinearizationManager method), 121
- ## J
- `J` (MbdSystem attribute), 89
- `joint_system` (MbdSystem attribute), 88

[joint_to_child_cg](#) (MbdSystem attribute), [88](#)
[JointLoc](#) (MbdSystem attribute), [88](#)
[JointLoc_func](#) (MbdSystem attribute), [89](#)
[joints](#) (JointSystem attribute), [91](#)
[joints](#) (MbdSystem attribute), [87](#)
[Joints](#) (VelocityTransformation attribute), [94](#)
[JointSystem](#) (class in multibody), [91](#)
[JointtoChildCG](#) (VelocityTransformation attribute), [94](#)
K
[Kbar](#) (LinearizedMBD attribute), [119](#)
[kind](#) (PointRef attribute), [136](#)
L
[line_type](#) (MooringLine attribute), [136](#)
[line_types](#) (MoorPyInputs attribute), [134](#)
[LinearizationAdapter](#) (class in multibody.linearization.linearization_main), [122](#)
[LinearizationManager](#) (class in multibody.linearization.linearization_main), [119](#)
[linearize_mbd\(\)](#) (in module multibody), [118](#)
[LinearizedMBD](#) (class in multibody.linearization.linearize_eom), [119](#)
[LineType](#) (class in multibody.linearization._moorpy_schema), [134](#)
[load_yaml_as_example\(\)](#) (in module multibody), [113](#)
M
[M](#) (MbdSystem attribute), [89](#)

[m](#) (MbdSystem attribute), [89](#)
[m4e_cartesian_rows](#) (MoorPyLinearMCKF property), [133](#)
[mass_per_length](#) (LineType attribute), [135](#)
[Mbar](#) (LinearizedMBD attribute), [119](#)
[MbdSystem](#) (class in multibody), [86](#)
[mooring_lines](#) (MoorPyInputs attribute), [134](#)
[MooringLine](#) (class in multibody.linearization._moorpy_schema), [135](#)
[moorpy_system](#) (MoorPyLinearMCKF property), [133](#)
[MoorPyInputs](#) (class in multibody.linearization._moorpy_schema), [133](#)
[MoorPyLinearMCKF](#) (class in multibody.linearization.moorpy_linear_mckf), [131](#)
[name](#) (HydroLinearMCKF attribute), [123](#)
[name](#) (LinearizationAdapter attribute), [122](#)
[name](#) (MooringLine attribute), [136](#)
[name](#) (MoorPyLinearMCKF attribute), [132](#)
[NBodies](#) (VelocityTransformation attribute), [95](#)
[NDOF](#) (JointSystem attribute), [91](#)
[NDOF](#) (MbdSystem attribute), [88](#)
[NDoF](#) (VelocityTransformation attribute), [95](#)
[normalize_prismatic\(\)](#) (in module multibody), [93](#)
[num_segments](#) (MooringLine attribute), [136](#)
P
[parent_cg_to_joint](#) (MbdSystem attribute), [88](#)

- ParentCGtoJoint (VelocityTransformation attribute), 94
 parse_body_loads() (YAML2Example method), 114
 parse_ic() (YAML2Example method), 114
 parse_motors() (YAML2Example method), 114
 parse_spring_dampers() (YAML2Example method), 114
 paths (MbdSystem attribute), 89
 plot_multibody_system() (in module multibody.plotting), 99
 plt_template() (in module multibody.plotting), 101
 point_id (PointRef attribute), 136
 PointRef (class in multibody.linearization._moorpy_schema), 136
 Points_All (MbdSystem attribute), 89
 points_force_finder() (in module multibody), 96
 points_forces() (in module multibody.checks), 104
 points_table() (in module multibody.tables), 106
 Pos (MbdSystem attribute), 88
 Pos_func (MbdSystem attribute), 89
 prismatic_direction (MbdSystem attribute), 88
 prismatic_direction (VelocityTransformation attribute), 95
- ## Q
- Q (MbdSystem attribute), 88
 QD (MbdSystem attribute), 88
 QDD (MbdSystem attribute), 88
- ## R
- R (MbdSystem attribute), 88
 R_func (MbdSystem attribute), 89
 ramp() (LinearizationManager static method), 120
 RD (MbdSystem attribute), 88
 RD_func (MbdSystem attribute), 89
 read_mass_and_inertia() (YAML2Example method), 114
 ReducedM_func (MbdSystem attribute), 89
 reduction_indices (MoorPyLinearMCKF property), 133
 Reference_frame_Origin (MbdSystem attribute), 88
 Reference_frame_Origin (VelocityTransformation attribute), 95
 register() (ExternalForcesManager method), 108
 register() (LinearizationManager method), 120
 rho (MoorPyInputs attribute), 134
 Right_side_func (MbdSystem attribute), 89
- ## S
- save_bem_data() (HydroLinearMCKF static method), 124
 symvars_definition() (in module multibody), 98
 systems_energy() (in module multibody), 98
- ## T
- t_update (MbdSystem attribute), 89
 Type (VelocityTransformation attribute), 94
 types (MbdSystem attribute), 87

U

`unstretched_length` (`MooringLine` attribute), [136](#)

V

`validate_moorpy_inputs()`
(in module `multibody.linearization.__moorpy__schema`),
[137](#)

`Vel` (`MbdSystem` attribute), [88](#)

`Vel_func` (`MbdSystem` attribute), [89](#)

`VelocityTransformation` (class in `multibody`), [94](#)

Y

`YAML2Example` (class in `multibody.yaml_parser.yaml_adapter`),
[113](#)